

TRƯỜNG ĐẠI HỌC NGOẠI NGỮ - TIN HỌC TP. HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN

KHÓA LUẬN TỐT NGHIỆP

THEO DÕI LƯỒNG GIAO THÔNG

GIẢNG VIÊN HƯỚNG DẪN: ThS. Tôn Quang Toại

SINH VIÊN THỰC HIỆN: Phan Minh Tân – 21DH111670

Nguyễn Xuân Thịnh – 21DH114152

TP. HỒ CHÍ MINH – 08-2024

TRƯỜNG ĐẠI HỌC NGOẠI NGỮ - TIN HỌC TP. HỒ CHÍ MINH

KHOA CÔNG NGHỆ THÔNG TIN

KHÓA LUẬN TỐT NGHIỆP

THEO DÕI LƯỖNG GIAO THÔNG

GIẢNG VIÊN HƯỚNG DẪN: ThS. Tôn Quang Toại

SINH VIÊN THỰC HIỆN: Phan Minh Tân – 21DH111670

Nguyễn Xuân Thịnh – 21DH114152

TP. HỒ CHÍ MINH – 08-2024

LỜI CẢM ƠN

Để hoàn thành bài khóa luận này, chúng em xin chân thành cảm ơn thầy ThS. Tôn Quang Toại, người đã tận tình hướng dẫn và hỗ trợ chúng em trong suốt quá trình học tập và nghiên cứu. Sự chỉ bảo và động viên của thầy là nguồn động lực lớn giúp chúng em hoàn thành công việc này.

Chúng em cũng xin gửi lời cảm ơn đến tất cả các thầy cô trong Khoa Công nghệ Thông tin, Trường Đại học Ngoại ngữ - Tin học TP. Hồ Chí Minh, đã truyền đạt kiến thức và kỹ năng quý báu cho chúng em trong suốt thời gian học tập tại trường.

Mặc dù đã cố gắng hết sức để hoàn thiện bài khóa luận này một cách hoàn chỉnh nhất, nhưng do hạn chế về kiến thức và kinh nghiệm, chúng em không thể tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự góp ý của các thầy cô và bạn bè để bài khóa luận được hoàn thiện hơn.

Chúng em xin chân thành cảm ơn !

TP. Hồ Chí Minh, tháng 08 năm 2024

Sinh viên thực hiện

LỜI CAM ĐOAN

Chúng em xin cam đoan số liệu và kết quả nghiên cứu trong khóa luận này là trung thực và chưa hề được sử dụng để bảo vệ một học vị nào. Mọi sự giúp đỡ cho việc thực hiện khóa luận này đã được cảm ơn và các thông tin trích dẫn trong khóa luận đều được chỉ rõ nguồn gốc rõ ràng và được phép công bố.

TP. Hồ Chí Minh, tháng 8 năm 2024

Sinh viên thực hiện

TÓM TẮT KHÓA LUẬN

Trong bối cảnh đô thị hóa và sự gia tăng nhanh chóng của các phương tiện giao thông, vấn đề quản lý và giám sát luồng giao thông trở nên cấp thiết. Khóa luận này nghiên cứu và phát triển một hệ thống theo dõi luồng giao thông thông minh, ứng dụng công nghệ thị giác máy tính và học sâu nhằm phát hiện, đếm và phân loại các loại phương tiện giao thông như ô tô, xe buýt, xe tải, đồng thời xác định hướng di chuyển của từng phương tiện.

Để giải quyết bài toán này, chúng em đã nghiên cứu và so sánh một số phương pháp phổ biến trong lĩnh vực thị giác máy tính và học sâu như YOLO và các phương pháp xử lý ảnh truyền thống. Mỗi phương pháp đều có ưu và nhược điểm riêng, tùy thuộc vào mục đích sử dụng và điều kiện môi trường.

Trong việc theo dõi và nhận diện các đối tượng di chuyển, khóa luận cũng thực hiện so sánh giữa hai phương pháp nổi bật là SORT và BYTETrack. SORT (Simple Online and Realtime Tracking) là một phương pháp theo dõi đơn giản và hiệu quả, phù hợp với các ứng dụng thời gian thực nhưng có thể bị giảm hiệu suất trong điều kiện phức tạp. BYTETrack, mặt khác, là một cải tiến mới với khả năng theo dõi các đối tượng mất dấu tạm thời, giúp tăng độ chính xác trong các tình huống giao thông phức tạp và đa dạng.

Phương pháp đề xuất trong khóa luận là sử dụng YOLOv8, một phiên bản ổn định nhất của họ mô hình YOLO, được thiết kế để cải thiện độ chính xác và hiệu suất phát hiện đối tượng. YOLOv8 kết hợp các cải tiến về kiến trúc mạng, tối ưu hóa quá trình suy diễn và cơ chế học sâu mới, giúp tăng cường khả năng nhận diện và theo dõi các phương tiện giao thông trong điều kiện thực tế đa dạng.

Kết quả thực nghiệm cho thấy, hệ thống đề xuất đạt được độ chính xác cao trong việc phát hiện và phân loại các loại phương tiện giao thông. Hệ thống cũng hoạt động hiệu quả trong các điều kiện ánh sáng và thời tiết khác nhau, đảm bảo tính ứng dụng cao trong thực tiễn. Khóa luận này đã góp phần nâng cao an toàn giao thông và tối ưu hóa việc quản lý giao thông đô thị.

MỤC LỤC

LỜI CẢM ƠN	I
LỜI CAM ĐOAN	II
TÓM TẮT KHÓA LUẬN	III
MỤC LỤC.....	I
DANH MỤC CÁC KÝ HIỆU, CHỮ VIẾT TẮT	V
DANH MỤC CÁC BẢNG.....	VI
DANH MỤC HÌNH ẢNH	VII
DANH MỤC BIỂU ĐỒ.....	VIII
Chương 1. Giới thiệu bài toán.....	1
1.1 Câu hỏi nghiên cứu.....	1
1.2 Tầm quan trọng	2
1.2.1 Tính mới.....	2
1.2.2 Tính hữu dụng.....	3
1.3 Giới hạn nghiên cứu	3
1.4 Bố cục khóa luận	4
Chương 2. Lý thuyết nền tảng	6
2.1 Thị giác máy tính.....	6
2.1.1 Quá trình hình thành ảnh.....	6
2.1.2 Dữ liệu ảnh.....	7
2.1.3 Các thao tác trên dữ liệu ảnh.....	8
2.1.4 Một số bài toán về thị giác máy tính.....	10
2.2 Deep learning	12
2.2.1 Perceptron	12

2.2.2	Mạng neuron đa tầng	12
2.2.3	Mạng neuron tích chập.....	13
2.2.4	Phép toán Convolution.....	13
2.2.5	Phép toán Padding.....	13
2.2.6	Phép toán Stride	14
2.3	Một số kiến trúc mạng neuron tích chập hiện đại	14
2.3.1	Mạng VGG.....	14
2.3.2	Mạng ResNet	15
2.3.3	Mạng Vision Transformer.....	15
2.4	Các phương pháp đánh giá bài toán	16
2.4.1	Average Precision (AP)	16
2.4.2	Mean Average Precision (mAP)	16
2.4.3	Intersection over Union (IoU).....	17
2.4.4	Multiple Object Tracking Accuracy (MOTA).....	17
2.4.5	Multiple Object Tracking Precision (MOTP).....	18
2.4.6	Higher Order Tracking Accuracy (HOTA)	18
2.4.7	ID F1 Score (IDF1).....	18
2.4.8	ID Switches (IDs)	19
Chương 3.	Các công trình liên quan	20
3.1	Bộ lọc Kalman (Kalman Filter): Ước lượng trạng thái tối ưu trong thế giới nhiễu 20	
3.2	Thuật toán Hungarian Association	27
3.3	Thuật toán SORT (2017).....	29
3.3.1	Detection	31
3.3.2	Mô hình ước lượng	32

3.3.3	Kết hợp dữ liệu.....	34
	Algorithm 1 Mã giả kết hợp SORT và YOLO.....	38
3.4	ByteTrack	39
3.4.1	Giới thiệu	39
3.4.2	Ý tưởng chính của ByteTrack.....	39
3.4.3	Đổi mới chính của ByteTrack.....	39
3.4.4	Các bước của thuật toán ByteTrack.....	39
3.4.5	Các thành phần trong ByteTrack	42
	Algorithm 1 Mã giả của BYTE.....	43
3.4.7	Kết luận	43
3.5	So sánh giữa ByteTrack và SORT	44
3.6	Mô hình Darknet-19	46
3.7	Mô hình Darknet-53	47
3.8	Mô hình CSPNet	47
3.9	Mô hình EfficientDet.....	47
3.10	Mô hình EfficientNet-L2.....	48
3.11	Mô hình YOLO	48
3.11.1	Giới thiệu	48
3.11.2	Cách YOLO hoạt động	49
3.11.3	Anchor box.....	51
3.11.4	Hàm tính IoU	52
3.11.5	Hàm loss.....	52
3.11.6	Dự đoán Bounding box.....	53
3.11.7	Non-max suppression.....	55
3.11.8	Quá trình phát triển	56

Chương 4. Giải pháp đề xuất	66
4.1 Phân tích dữ liệu	66
4.1.1 Dữ liệu test	66
4.1.2 Dữ liệu huấn luyện	66
4.2 Mô hình	68
4.2.1 Backbone	68
4.2.2 Neck	69
4.2.3 Head	69
4.3 Huấn luyện model trên tập dữ liệu được tạo	69
4.4 Mô tả giải pháp thiết kế	70
4.4.1 ROI	70
4.4.2 Phát hiện đối tượng	71
4.4.3 Truy vết đối tượng	71
4.4.4 Đếm đối tượng khi vào ROI	72
4.5 Đánh giá	74
4.6 Thực nghiệm	75
KẾT LUẬN	77
Những điều đã làm được	77
Những điều chưa làm được	77
Hướng phát triển	77
TÀI LIỆU THAM KHẢO	79
PHỤ LỤC	81
Các thành phần thuật toán SORT	81

DANH MỤC CÁC KÝ HIỆU, CHỮ VIẾT TẮT

STT	KÝ HIỆU, CHỮ VIẾT TẮT	Ý NGHĨA
1	YOLO	You Only Look Once
2	SORT	Simple Online and Realtime Tracking
3	ROI	Region of Interest
4	IoU	Intersection over Union
5	CNN	Convolutional Neural Network
6	MLP	Multilayer Perceptron
7	GAN	Generative Adversarial Network
8	VAE	Variational Autoencoder
9	MOTA	Multiple Object Tracking Accuracy
10	MOTP	Multiple Object Tracking Precision
11	AP	Average Precision
12	HOTA	Higher Order Tracking Accuracy
13	mAP	Mean Average Precision
14	IDF1	ID F1 Score
15	IDs	ID Switches

DANH MỤC CÁC BẢNG

Bảng 1. Phương trình cập nhật thời gian.....	23
Bảng 2. Các giá trị đo lường	26
Bảng 3. Tính giá trị xk	26
Bảng 4. So sánh hiệu suất của các bộ theo dõi và phát hiện đối tượng	32
Bảng 5. Kết quả so sánh các mô hình YOLO	74

DANH MỤC HÌNH ẢNH

Hình 1. Ảnh minh họa theo dõi luồng giao thông.....	1
Hình 2. Ảnh minh họa theo dõi luồng giao thông.....	2
Hình 3. Biểu đồ hội tụ của bộ lọc Kalman.....	26
Hình 4. Biểu diễn trực quan của giải thuật Hungarian.....	27
Hình 5. Các bước thực hiện giải thuật Hungarian.....	29
Hình 6. Quy trình theo dõi đối tượng sử dụng Kalman Filter và Hungarian Algorithm	30
Hình 7. Quy trình theo dõi đối tượng sử dụng bộ lọc Kalman và khớp IOU	31
Hình 8. Quy trình cập nhật và theo dõi đối tượng qua các khung hình	35
Hình 9. Quá trình theo dõi đối tượng qua các khung hình trong không gian	36
Hình 10. Quy trình của BYTETrack.....	42
Hình 11. Xe sắp bị che khuất (SORT)	45
Hình 12. Xe bị đổi id sau khi bị cây che khuất (SORT)	45
Hình 13. Xe sắp bị che khuất (BYTETRACK)	46
Hình 14. Xe vẫn giữ được ID (BYTETRACK).....	46
Hình 15. Kiến trúc mô hình.....	49
Hình 16. Quy trình phát hiện đối tượng sử dụng mạng YOLO	49
Hình 17. Cấu trúc đầu ra của mạng YOLO cho phát hiện đối tượng	50
Hình 18. Cơ chế chọn bounding box trong mạng YOLO dựa trên IoU.....	51
Hình 19. Hàm tính IoU.....	52
Hình 20. Phương trình tính toán bounding box trong YOLO.....	54
Hình 21. Hiệu ứng của Non-Max Suppression trong phát hiện đối tượng	55
Hình 22. Quá trình phát triển của YOLO.....	56
Hình 23. So sánh hiệu suất phát hiện đối tượng trên MS COCO	59
Hình 24. So sánh thời gian huấn luyện giữa YOLOv4-custom và YOLOv5s.....	61
Hình 25. So sánh hiệu suất và độ trễ của các mô hình YOLO trên MS COCO	62
Hình 26. So sánh hiệu suất phát hiện đối tượng của các mô hình YOLO trên MS COCO.....	63
Hình 27. Kiến trúc mạng với các bước mở rộng độ sâu và chiều rộng	64

Hình 28. Kiến trúc chi tiết của mô hình YOLO.....	68
Hình 29. Giám sát luồng giao thông tại ngã tư sử dụng ROI	70
Hình 30. Giám sát luồng giao thông sử dụng model YOLO để phát hiện đối tượng	71
Hình 31. Giám sát luồng giao thông sử dụng BYTETracker để truy vết đối tượng.	71
Hình 32. Giám sát luồng giao thông sử dụng BYTETracker để truy vết đối tượng.	72
Hình 33. Đếm đối tượng khi vào ROI	72
Hình 34. Đếm đối tượng khi vào ROI	73
Hình 35. Giao diện chính	75
Hình 36. Chức năng detection trên ảnh.....	75
Hình 37. Chức năng tracking	76
Hình 38. Chức năng vẽ vùng đếm.....	76

DANH MỤC BIỂU ĐỒ

Biểu đồ 1. Quy trình tổng quát.....	66
-------------------------------------	----

Chương 1. Giới thiệu bài toán

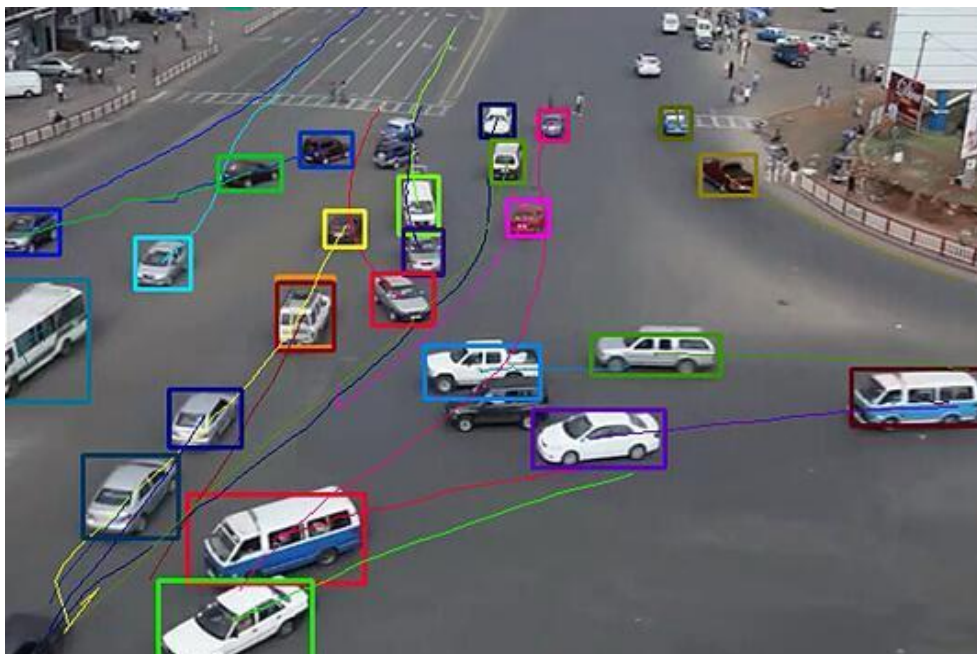
1.1 Câu hỏi nghiên cứu

Bài toán nghiên cứu nhằm phát triển một hệ thống theo dõi luồng giao thông thông minh ứng dụng công nghệ thị giác máy tính và deep learning tiên tiến. Hệ thống có khả năng phát hiện, đếm và phân loại chính xác các loại phương tiện giao thông, bao gồm ô tô, xe bus, xe tải, v.v., đồng thời xác định hướng di chuyển của từng phương tiện. Nhờ vậy, hệ thống góp phần nâng cao an toàn giao thông và tối ưu hóa việc quản lý giao thông đô thị hiệu quả.

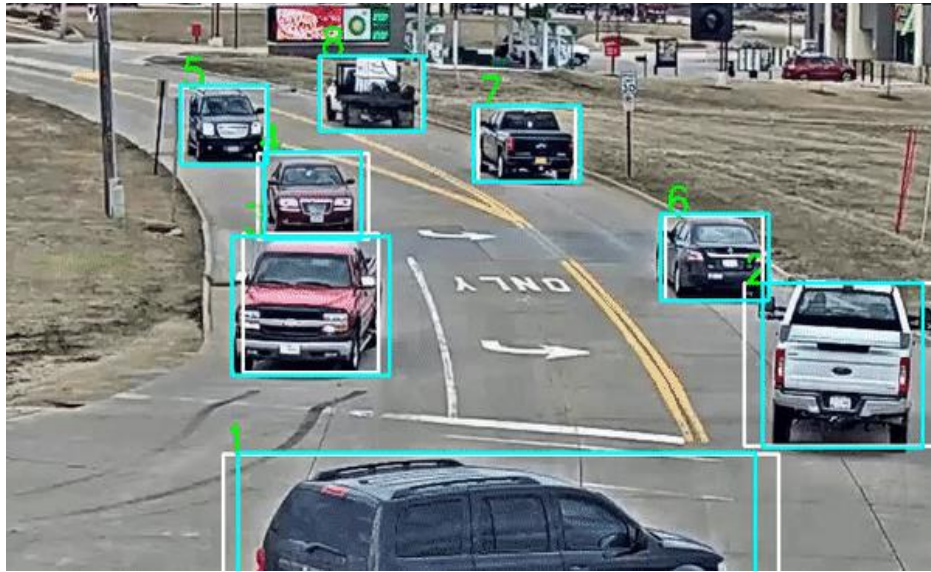
Input cho hệ thống bao gồm dữ liệu video được thu thập từ các camera giám sát giao thông đặt tại các điểm giao thông trong thành phố. Các video này có độ phân giải cao, được quay trong điều kiện ánh sáng và thời tiết khác nhau để đảm bảo tính đa dạng và thực tế.

Output của hệ thống là số lượng, loại phương tiện và hướng đi được phát hiện trong mỗi khung hình.

Một số ví dụ minh họa:



Hình 1. Ảnh minh họa theo dõi luồng giao thông



Hình 2. Ảnh minh họa theo dõi luồng giao thông

1.2 Tầm quan trọng

1.2.1 Tính mới

Về dữ liệu, chúng tôi sử dụng một tập dữ liệu giao thông được thu thập từ nhiều camera đặt tại các vị trí khác nhau trên các tuyến đường chính và phụ trong thành phố. Dữ liệu này không chỉ phong phú về mặt số lượng mà còn đa dạng về các điều kiện ánh sáng và thời tiết, giúp mô hình có khả năng hoạt động tốt trong nhiều hoàn cảnh khác nhau. Điều này đảm bảo rằng hệ thống có thể nhận diện và theo dõi phương tiện một cách chính xác hơn, ngay cả trong các tình huống thời tiết xấu hoặc điều kiện ánh sáng yếu.

Về phương pháp, trong nghiên cứu này, chúng tôi tập trung vào YOLOv8, phiên bản ổn định của họ mô hình YOLO, với những bước tiến đáng kể về kiến trúc và hiệu năng. YOLOv8 được thiết kế để mang đến khả năng phát hiện đối tượng vượt trội, đặc biệt là với các vật thể ở khoảng cách xa và kích thước nhỏ, đồng thời tối ưu hóa hiệu quả tính toán cho quá trình suy diễn nhanh hơn. YOLOv8 bao gồm các cải tiến như kiến trúc mạng nâng cao, tối ưu hóa quá trình suy diễn và cơ chế học sâu mới, giúp tăng cường độ chính xác và hiệu suất của mô hình.

1.2.2 Tính hữu dụng

- Vấn đề thực tế:
 - Quản lý và giám sát luồng giao thông tại các giao lộ là một vấn đề quan trọng trong đô thị hiện đại.
 - Việc theo dõi số lượng và loại xe ra vào giao lộ giúp cung cấp thông tin hữu ích cho việc:
 - **Đánh giá tình trạng giao thông:** Xác định thời gian cao điểm, mức độ ùn tắc và các vấn đề tiềm ẩn.
 - **Lập kế hoạch giao thông:** Cung cấp dữ liệu cho việc thiết kế hệ thống đèn tín hiệu, quy hoạch làn đường, và tối ưu hóa luồng giao thông.
- Ứng dụng hệ thống: Hệ thống quản lý giao thông thông minh (ITS), hệ thống giám sát an ninh, ...

1.3 Giới hạn nghiên cứu

Trong nghiên cứu này, chúng tôi đặt ra một số giới hạn và giả định cụ thể để hình thành phạm vi và định hướng cho đề tài theo dõi luồng giao thông bằng mô hình YOLOv8. Những giới hạn này giúp tập trung vào các mục tiêu cụ thể và đảm bảo rằng kết quả nghiên cứu có thể đạt được và đáng tin cậy.

- Giới hạn về loại phương tiện được phân tích:

Nghiên cứu chỉ tập trung vào việc nhận diện và phân loại các phương tiện chính như ô tô, xe buýt, và xe tải. Các loại phương tiện đặc biệt như xe hai bánh, người đi bộ không nằm trong phạm vi của đề tài này.

- Điều kiện giao thông:

Mô hình được huấn luyện và kiểm thử dựa trên dữ liệu thu thập trong điều kiện giao thông bình thường và đông đúc. Các tình huống giao thông khẩn cấp hoặc tai nạn không được xem xét trong phạm vi nghiên cứu này.

- Giới hạn về điều kiện môi trường:

Dữ liệu được sử dụng trong nghiên cứu chủ yếu được thu thập trong điều kiện ánh sáng tốt và thời tiết khô ráo. Điều kiện thời tiết xấu như mưa to, sương mù, hoặc tuyết rơi có thể ảnh hưởng đến khả năng nhận diện của mô hình và không được đưa vào phân tích trong đề tài này.

- Độ phân giải và chất lượng dữ liệu:

Giả định rằng tất cả dữ liệu video được sử dụng có độ phân giải cao (ít nhất là Full HD - 1080p). Dữ liệu hình ảnh chất lượng thấp hơn có thể ảnh hưởng đến kết quả của mô hình và không được phân tích trong nghiên cứu này.

- Độ chính xác của mô hình:

Mặc dù mục tiêu là đạt được độ chính xác cao, nhưng nghiên cứu này chấp nhận rằng mô hình có thể không hoàn toàn chính xác 100% trong mọi tình huống do giới hạn của thuật toán và dữ liệu huấn luyện.

1.4 Bố cục khóa luận

Nội dung của khóa luận được trình bày với các phần chính như sau:

Chương 1: Giới thiệu yêu cầu bài toán, loại dữ liệu được sử dụng. Đưa ra các đặc điểm của đề tài như tính mới, tính hữu dụng. Cuối cùng là đưa ra các giới hạn của đề tài.

Chương 2: Xây dựng nền tảng lý thuyết cần thiết cho đề tài, giới thiệu về thị giác máy tính và học sâu (deep learning), cùng với một vài mạng neuron tích chập (CNNs) hiện. Tiếp theo, trình bày các phương pháp đánh giá hiệu quả của mô hình của đề tài. Bên cạnh đó là các lý thuyết chuyên sâu như bộ lọc Kalman (Kalman Filter), thuật toán Hungarian Association, và thuật toán SORT, cùng với cách kết hợp SORT với mô hình hiện tại để cải thiện hiệu quả theo dõi đối tượng trong video.

Chương 3: Đưa ra quá trình phát triển của YOLO, chi tiết lý thuyết của các mô hình YOLO (v1-v8) và các mô hình backbone được sử dụng.

Chương 4: Phân tích chi tiết hơn về dữ liệu đầu vào (video, ảnh được lấy từ video và chú thích ảnh theo định dạng YOLO). Giải thích kỹ hơn về mô hình, kiến trúc của YOLOv8 và cuối cùng là thực nghiệm.

Kết luận: Báo cáo lại những điều đã đạt được và chưa đạt được trong đề tài. Đưa ra hướng phát triển cho đề tài, cách khắc phục những điều chưa đạt được.

Chương 2. Lý thuyết nền tảng

2.1 Thị giác máy tính

2.1.1 Quá trình hình thành ảnh

Quá trình hình thành ảnh trong thị giác máy tính mô tả cách ánh sáng từ thế giới thực được chuyển đổi thành tín hiệu số mà máy tính có thể xử lý. Quá trình này thường được minh họa bằng mô hình camera pinhole, một mô hình đơn giản nhưng hiệu quả để giải thích cơ chế hoạt động của máy ảnh.

1. Mô hình camera pinhole:

- **Pinhole:** Một lỗ nhỏ được khoét trên một tấm chắn sáng.
- **Ánh sáng:** Ánh sáng từ vật thể đi qua pinhole và chiếu lên một màn hình đặt phía sau.
- **Hình ảnh:** Ánh sáng tạo ra một hình ảnh ngược chiều và thu nhỏ của vật thể trên màn hình.

2. Quá trình hình thành ảnh:

- **Ánh sáng phản chiếu:** Vật thể phản chiếu ánh sáng từ nguồn sáng (mặt trời, đèn,...).
- **Đi qua ống kính:** Ánh sáng phản chiếu từ vật thể đi qua ống kính của máy ảnh.
- **Tập trung ánh sáng:** Ống kính hội tụ ánh sáng vào một điểm trên cảm biến, tạo thành hình ảnh ngược chiều và thu nhỏ của vật thể.
- **Cảm biến ảnh:** Cảm biến ảnh (ví dụ: CMOS hoặc CCD) ghi nhận cường độ ánh sáng tại mỗi điểm trên bề mặt cảm biến, tạo ra các giá trị số tương ứng với màu sắc và cường độ ánh sáng tại mỗi điểm.
- **Chuyển đổi sang tín hiệu số:** Các giá trị số từ cảm biến ảnh được xử lý và chuyển đổi thành tín hiệu số, được lưu trữ dưới dạng file ảnh.

3. Hệ tọa độ ảnh:

- **Hệ tọa độ ảnh:** Hệ tọa độ được sử dụng để mô tả vị trí của các điểm ảnh trong ảnh, thường được biểu diễn bằng các giá trị nguyên (x, y) tương ứng với hàng và cột của điểm ảnh.
- **Hệ tọa độ thế giới:** Hệ tọa độ được sử dụng để mô tả vị trí của các điểm trong thế giới thực, thường được biểu diễn bằng các giá trị thực (X, Y, Z) .
- **Hệ tọa độ camera:** Hệ tọa độ được sử dụng để mô tả vị trí của các điểm trong không gian 3 chiều tương đối với camera, thường được biểu diễn bằng các giá trị thực (X_c, Y_c, Z_c) .

4. Biến đổi hình học:

- **Biến đổi hình học:** Các phép biến đổi toán học được áp dụng để thay đổi vị trí, kích thước hoặc hình dạng của ảnh, ví dụ như:
- **Tịnh tiến:** Di chuyển ảnh theo một hướng nhất định.
- **Quay:** Xoay ảnh quanh một điểm.
- **Co giãn:** Thay đổi kích thước của ảnh.
- **Biến dạng:** Biến đổi hình dạng của ảnh theo một hàm biến đổi.

2.1.2 Dữ liệu ảnh

Ảnh kỹ thuật số là tập hợp các điểm ảnh (pixel) được sắp xếp theo hàng và cột, mỗi pixel chứa thông tin về màu sắc và cường độ ánh sáng tại vị trí đó trong ảnh.

Dữ liệu trong ảnh có thể được phân loại theo các khía cạnh sau:

- **Kênh màu:** Mô tả màu sắc của mỗi điểm ảnh, ví dụ như RGB (hệ màu cơ bản), Grayscale (thang độ xám cho ảnh đen trắng), HSV (hệ màu dựa trên sắc độ, độ bão hòa, và giá trị), ...
- **Độ phân giải:** Số lượng điểm ảnh tạo nên ảnh (ví dụ: 1920x1080) ảnh hưởng đến kích thước và độ chi tiết của ảnh. Độ phân giải không gian (ví dụ: 300 PPI) thể hiện mức độ chi tiết và khả năng phân biệt các điểm lân cận.
- **Định dạng:** Cách lưu trữ và nén dữ liệu ảnh. JPEG phổ biến cho ảnh chụp với mức nén cao, PNG cho ảnh đồ họa với độ trong suốt và nén không mất

dữ liệu, BMP cho ảnh đơn giản không nén. Ngoài ra còn các định dạng phổ biến như là GIF, SVG, TIFF, ...

2.1.3 Các thao tác trên dữ liệu ảnh

Xử lý ảnh là một phần quan trọng trong thị giác máy tính, cho phép chúng ta biến đổi và phân tích dữ liệu ảnh để trích xuất thông tin hữu ích hoặc cải thiện chất lượng ảnh. Dưới đây là một số thao tác cơ bản trên dữ liệu ảnh:

1. Thao tác điểm ảnh (Pixel-level Operations):

- **Thay đổi độ sáng/độ tương phản:** Điều chỉnh mức độ sáng tối và độ tương phản của ảnh. Ví dụ: Tăng độ sáng để nhìn rõ hơn các chi tiết trong ảnh chụp thiếu sáng, hoặc giảm độ tương phản để làm mềm ảnh.
- **Cân bằng trắng (White Balance):** Hiệu chỉnh màu sắc của ảnh để ánh sáng trắng trông tự nhiên. Ví dụ: Chỉnh sửa màu sắc của ảnh chụp dưới ánh đèn huỳnh quang để màu sắc trông tự nhiên hơn.
- **Thay đổi gam màu (Color Mapping):** Ánh xạ màu sắc từ không gian màu này sang không gian màu khác. Ví dụ: Chuyển đổi ảnh từ RGB sang Grayscale để phân tích các chi tiết dựa trên cường độ sáng.
- **Lọc nhiễu (Noise Reduction):** Loại bỏ nhiễu khỏi ảnh để cải thiện chất lượng. Ví dụ: Loại bỏ nhiễu hạt (grain) trong ảnh chụp thiếu sáng hoặc nhiễu do cảm biến ảnh kém chất lượng.
- **Lọc tuyến tính (Linear Filtering):** Sử dụng phép tích chập (convolution) với bộ lọc tuyến tính để làm mịn ảnh, làm sắc nét ảnh, hoặc phát hiện cạnh. Ví dụ: Sử dụng bộ lọc Gaussian để làm mịn ảnh, hoặc sử dụng bộ lọc Laplacian để phát hiện cạnh.
- **Lọc phi tuyến (Nonlinear Filtering):** Sử dụng các hàm phi tuyến để loại bỏ nhiễu, bảo toàn chi tiết, hoặc thực hiện các hiệu ứng đặc biệt. Ví dụ: Sử dụng bộ lọc median để loại bỏ nhiễu muối tiêu (salt and pepper), hoặc sử dụng bộ lọc bilateral để bảo toàn chi tiết và loại bỏ nhiễu.

2. Xử lý hình thái học (Morphological Operations):

- **Giãn nở (Dilation):** Mở rộng các vùng sáng trong ảnh. Ví dụ: Sử dụng giãn nở để làm đầy các lỗ nhỏ trong ảnh hoặc mở rộng các đường viền của vật thể.
- **Co lại (Erosion):** Thu hẹp các vùng sáng trong ảnh. Ví dụ: Sử dụng co lại để loại bỏ các điểm ảnh nhỏ, làm mịn biên của vật thể.
- **Mở (Opening):** Loại bỏ các chi tiết nhỏ và làm mịn biên của vật thể. Ví dụ: Sử dụng mở để loại bỏ các điểm ảnh nhỏ và làm mịn biên của vật thể.
- **Đóng (Closing):** Lấp đầy các lỗ nhỏ và kết nối các vùng gần nhau. Ví dụ: Sử dụng đóng để lấp đầy các lỗ nhỏ trong vật thể hoặc kết nối các đường viền bị đứt gãy.

3. Biến đổi hình học (Geometric Transformations):

- **Tịnh tiến (Translation):** Di chuyển ảnh theo một hướng nhất định. Ví dụ: Di chuyển ảnh lên trên hoặc xuống dưới, sang trái hoặc sang phải.
- **Quay (Rotation):** Xoay ảnh quanh một điểm. Ví dụ: Xoay ảnh 90 độ, 180 độ hoặc bất kỳ góc nào khác.
- **Co giãn (Scaling):** Thay đổi kích thước của ảnh. Ví dụ: Phóng to hoặc thu nhỏ ảnh.
- **Biến dạng (Warping):** Biến đổi hình dạng của ảnh theo một hàm biến đổi. Ví dụ: Biến đổi hình dạng của ảnh để tạo hiệu ứng cong, méo hoặc các hiệu ứng đặc biệt khác.

4. Phân đoạn ảnh (Image Segmentation):

- **Phân đoạn dựa trên ngưỡng (Thresholding):** Chia ảnh thành các vùng dựa trên mức cường độ ánh sáng. Ví dụ: Tách nền trắng khỏi chữ đen trong một bức ảnh.
- **Phân đoạn dựa trên vùng (Region-based Segmentation):** Nhóm các điểm ảnh có tính chất tương đồng thành các vùng. Ví dụ: Tách các vật thể trong ảnh dựa trên màu sắc hoặc kết cấu.

- **Phân đoạn dựa trên cạnh (Edge-based Segmentation):** Phát hiện các cạnh trong ảnh và sử dụng chúng để phân chia ảnh thành các vùng. Ví dụ: Phát hiện biên của vật thể trong ảnh để phân đoạn chúng.

5. Trích xuất đặc trưng (Feature Extraction):

- **Trích xuất các đặc trưng hình dạng:** Diện tích, chu vi, độ tròn, độ dài, độ rộng,...
- **Trích xuất các đặc trưng kết cấu:** Độ mịn, độ nhám, hướng,...
- **Trích xuất các đặc trưng màu sắc:** Trung bình, độ lệch chuẩn, histogram,...

2.1.4 Một số bài toán về thị giác máy tính

Thị giác máy tính là một lĩnh vực rộng lớn với nhiều bài toán phức tạp và đa dạng. Dưới đây là một số bài toán cơ bản và phổ biến trong thị giác máy tính:

1. Phân loại ảnh (Image Classification):

- **Mục tiêu:** Xác định lớp (category) của một ảnh, ví dụ: chó, mèo, xe hơi, phong cảnh, ...
- **Ứng dụng:** Xây dựng hệ thống phân loại ảnh tự động, hỗ trợ tìm kiếm ảnh, phân tích nội dung hình ảnh, ...
- **Phương pháp:** Các mô hình học sâu (Deep Learning) như Convolutional Neural Network (CNN) thường được sử dụng cho bài toán này.

2. Phát hiện đối tượng (Object Detection) [1]:

- **Mục tiêu:** Phát hiện và xác định vị trí của các đối tượng trong một ảnh, đồng thời phân loại chúng, ví dụ: phát hiện và phân loại người, xe hơi, xe máy, ...
- **Ứng dụng:** Hệ thống giám sát an ninh, xe tự lái, robot, ...
- **Phương pháp:** Các mô hình học sâu như YOLO, Faster R-CNN, SSD,... thường được sử dụng cho bài toán này.

3. Phân đoạn ảnh (Image Segmentation):

- **Mục tiêu:** Chia một ảnh thành các vùng (region) khác nhau, mỗi vùng đại diện cho một đối tượng hoặc một phần của đối tượng, ví dụ: tách nền khỏi vật thể, tách các vật thể khác nhau trong ảnh, ...
 - **Ứng dụng:** Xử lý ảnh y tế, thực hiện các tác vụ chỉnh sửa ảnh, tự động hóa các tác vụ trong sản xuất, ...
 - **Phương pháp:** Có nhiều phương pháp phân đoạn ảnh, từ các phương pháp truyền thống như phân đoạn dựa trên ngưỡng, phân đoạn dựa trên vùng, phân đoạn dựa trên cạnh, đến các phương pháp học sâu như U-Net, Mask R-CNN,...
4. Nhận diện khuôn mặt (Face Recognition):
- **Mục tiêu:** Nhận dạng khuôn mặt của một người từ ảnh hoặc video.
 - **Ứng dụng:** Hệ thống kiểm soát an ninh, xác thực danh tính, ứng dụng giải trí, ...
 - **Phương pháp:** Các mô hình học sâu được huấn luyện trên một lượng lớn dữ liệu khuôn mặt để nhận dạng khuôn mặt chính xác.
5. Theo dõi đối tượng (Object Tracking):
- **Mục tiêu:** Theo dõi chuyển động của một đối tượng trong video, ví dụ: theo dõi xe cộ, người đi bộ, ...
 - **Ứng dụng:** Hệ thống giám sát an ninh, xe tự lái, robot, ...
 - **Phương pháp:** Có nhiều phương pháp theo dõi đối tượng, từ các phương pháp truyền thống như Kalman Filter, Particle Filter, đến các phương pháp học sâu như Deep SORT, TrackRCNN,...
6. Phân tích nội dung ảnh (Image Content Analysis):
- **Mục tiêu:** Trích xuất thông tin từ nội dung ảnh, ví dụ: nội dung của bức ảnh (chủ đề, cảnh, ...), cảm xúc thể hiện trong ảnh, ...
 - **Ứng dụng:** Xây dựng hệ thống tìm kiếm ảnh, phân tích cảm xúc, hỗ trợ thiết kế, ...
 - **Phương pháp:** Sử dụng các kỹ thuật xử lý ngôn ngữ tự nhiên (NLP) kết hợp với thị giác máy tính để phân tích nội dung ảnh.

7. Tạo ảnh (Image Generation):

- **Mục tiêu:** Tạo ra các bức ảnh mới dựa trên một tập dữ liệu ảnh hoặc dựa trên một mô tả văn bản.
- **Ứng dụng:** Xây dựng hệ thống tạo ảnh tự động, hỗ trợ thiết kế, chỉnh sửa ảnh, ...
- **Phương pháp:** Các mô hình học sâu như Generative Adversarial Network (GAN), Variational Autoencoder (VAE), ...

2.2 Deep learning

2.2.1 Perceptron

Perceptron là một mô hình học máy đơn giản nhất trong mạng neuron, được giới thiệu bởi Frank Rosenblatt vào năm 1958. Perceptron cơ bản bao gồm các thành phần chính sau:

- **Input :** Một vector các giá trị đầu vào.
- **Weights :** Các trọng số tương ứng với mỗi giá trị đầu vào.
- **Bias :** Một giá trị bù được thêm vào tổng trọng số của đầu vào.
- **Activation Function :** Xử lý tổng các đầu vào nhân với trọng số cộng với bias để đưa ra kết quả đầu ra.

Perceptron được sử dụng để phân loại các mẫu dữ liệu đầu vào thành các lớp khác nhau.

2.2.2 Mạng neuron đa tầng

Mạng neuron đa tầng (Multilayer Perceptron - MLP) là một dạng mở rộng của perceptron đơn giản, bao gồm nhiều lớp neuron:

- **Input Layer :** Nhận dữ liệu đầu vào.
- **Hidden Layers:** Xử lý và trích xuất các đặc trưng từ dữ liệu đầu vào.
- **Output Layer:** Đưa ra kết quả cuối cùng sau khi dữ liệu đã được xử lý qua các lớp ẩn.

MLP có khả năng học các mối quan hệ phức tạp hơn trong dữ liệu so với perceptron đơn giản. Nó sử dụng các hàm kích hoạt phi tuyến như ReLU, Sigmoid, Tanh để học và biểu diễn các đặc trưng phi tuyến của dữ liệu.

2.2.3 Mạng neuron tích chập

Mạng neuron tích chập (Convolutional Neural Network - CNN) là một loại mạng neuron được thiết kế đặc biệt để xử lý dữ liệu có cấu trúc lưới như hình ảnh:

- **Convolutional Layers:** Áp dụng các phép toán tích chập lên đầu vào để trích xuất các đặc trưng.
- **Pooling Layers:** Giảm kích thước của dữ liệu đầu vào để giảm bớt số lượng tham số và tính toán.
- **Fully Connected Layers:** Kết nối tất cả các neuron của lớp trước với lớp sau, giống như trong MLP.

CNN thường được sử dụng trong các ứng dụng như nhận diện hình ảnh, phân loại ảnh, phát hiện đối tượng và nhiều lĩnh vực khác liên quan đến xử lý hình ảnh.

2.2.4 Phép toán Convolution

Phép toán Convolution (Tích chập) là một phép toán giữa hai hàm số, thường được sử dụng để trích xuất các đặc trưng từ dữ liệu đầu vào. Trong CNN, phép toán này được thực hiện bằng cách:

- **Kernel :** Một ma trận nhỏ di chuyển trên toàn bộ dữ liệu đầu vào.
- **Stride :** Khoảng cách mà kernel di chuyển trên dữ liệu đầu vào.
- **Padding :** Thêm các giá trị vào biên của dữ liệu đầu vào để giữ nguyên kích thước của đầu ra sau phép toán convolution.

Kết quả của phép toán convolution là một ma trận đặc trưng thể hiện các đặc trưng không gian của dữ liệu đầu vào.

2.2.5 Phép toán Padding

Phép toán Padding là việc thêm các giá trị vào biên của dữ liệu đầu vào trước khi thực hiện phép toán convolution. Có hai loại padding phổ biến:

- **Valid Padding:** Không thêm giá trị nào vào biên, kích thước đầu ra sẽ nhỏ hơn kích thước đầu vào.
- **Same Padding:** Thêm giá trị vào biên để kích thước đầu ra bằng với kích thước đầu vào.

Padding giúp kiểm soát kích thước của đầu ra và bảo toàn thông tin ở biên của dữ liệu đầu vào.

2.2.6 Phép toán Stride

Phép toán Stride (Bước dịch chuyển) xác định khoảng cách mà kernel di chuyển trên dữ liệu đầu vào trong phép toán convolution. Một stride nhỏ (thường là 1) sẽ tạo ra đầu ra có kích thước lớn hơn, trong khi một stride lớn sẽ giảm kích thước đầu ra. Stride giúp kiểm soát mức độ giảm kích thước của dữ liệu qua các lớp convolution và pooling trong CNN.

2.3 Một số kiến trúc mạng neuron tích chập hiện đại

2.3.1 Mạng VGG

Mạng VGG (Visual Geometry Group) là một kiến trúc mạng neuron tích chập (CNN) được giới thiệu bởi nhóm nghiên cứu tại Đại học Oxford vào năm 2014. Các mô hình VGG, đặc biệt là VGG16 và VGG19, nổi tiếng với việc sử dụng các lớp convolution với kích thước bộ lọc nhỏ 3x3 và các lớp fully connected lớn ở cuối mạng. Các đặc điểm chính của mạng VGG bao gồm:

- **Sử dụng các lớp convolution nhỏ:** Các lớp convolution 3x3 được xếp chồng lên nhau để tăng độ sâu của mạng và trích xuất các đặc trưng phức tạp.
- **Sử dụng các lớp pooling:** Các lớp Max-Pooling 2x2 được sử dụng để giảm kích thước không gian của dữ liệu và tăng tính bất biến của mô hình đối với các thay đổi nhỏ trong đầu vào.
- **Các lớp fully connected:** Các lớp fully connected ở cuối mạng để đưa ra dự đoán cuối cùng.

Mạng VGG đã đạt được kết quả ấn tượng trong các cuộc thi như ImageNet và được sử dụng rộng rãi trong nhiều ứng dụng thị giác máy tính như phân loại ảnh, nhận diện đối tượng và phân đoạn ảnh.

2.3.2 Mạng ResNet

Mạng ResNet (Residual Networks) được giới thiệu bởi Kaiming He và các cộng sự vào năm 2015 và đã mang lại sự đột phá trong việc huấn luyện các mạng neuron rất sâu. Các đặc điểm chính của mạng ResNet bao gồm:

- **Residual Blocks (Khối dư thừa):** Mỗi residual block bao gồm các lớp convolution cùng với các kết nối dư thừa (skip connections) cho phép thông tin đi qua các lớp mà không cần thay đổi. Điều này giúp giảm hiện tượng mất mát gradient và cải thiện quá trình huấn luyện.
- **Huấn luyện các mạng sâu hơn:** Nhờ các residual block, mạng ResNet có thể huấn luyện các mô hình rất sâu mà không gặp phải hiện tượng vanishing gradient, giúp đạt được độ chính xác cao hơn trong các bài toán phức tạp.
- **Kiến trúc đa dạng:** ResNet có nhiều biến thể với độ sâu khác nhau như ResNet-18, ResNet-34, ResNet-50, ResNet-101 và ResNet-152, cho phép lựa chọn mô hình phù hợp với yêu cầu của từng bài toán cụ thể.

Mạng ResNet đã đạt được nhiều thành tựu trong các cuộc thi như ImageNet và được sử dụng rộng rãi trong nhiều ứng dụng thị giác máy tính như phân loại ảnh, nhận diện đối tượng, phân đoạn ảnh và các bài toán khác.

2.3.3 Mạng Vision Transformer

Mạng Vision Transformer (ViT) là một kiến trúc mạng neuron mới được giới thiệu vào năm 2020 bởi Dosovitskiy et al. ViT áp dụng kiến trúc Transformer, vốn được phát triển cho các bài toán xử lý ngôn ngữ tự nhiên, vào bài toán thị giác máy tính. Các đặc điểm chính của mạng Vision Transformer bao gồm:

- **Sử dụng các patch của ảnh:** Ảnh đầu vào được chia thành các patch nhỏ (ví dụ 16x16 pixel), và mỗi patch được biến đổi thành một vector bằng cách sử dụng một lớp embedding.
- **Áp dụng kiến trúc Transformer:** Các vector embedding từ các patch được đưa qua các lớp Transformer encoder, giúp mô hình học được các mối quan hệ không gian phức tạp giữa các phần của ảnh.
- **Không cần các lớp convolution:** Khác với các mô hình CNN truyền thống, ViT không sử dụng các lớp convolution, mà hoàn toàn dựa vào các lớp Transformer để trích xuất các đặc trưng từ ảnh.

Vision Transformer đã đạt được kết quả ấn tượng trên nhiều bộ dữ liệu thị giác máy tính và mở ra một hướng tiếp cận mới trong việc áp dụng các kiến trúc mạng tiên tiến từ xử lý ngôn ngữ tự nhiên vào thị giác máy tính.

2.4 Các phương pháp đánh giá bài toán

2.4.1 Average Precision (AP)

- **Mô tả:** AP là chỉ số để đánh giá hiệu suất của mô hình phát hiện đối tượng bằng cách đo lường độ chính xác ở các mức độ khác nhau của độ nhạy [2].
- **Cách tính:** AP là diện tích dưới đường cong Precision-Recall (PR curve).
- **Công thức:**

$$AP = \sum_n (\text{Recall}(n) - \text{Recall}(n - 1)) \cdot \text{Precision} \quad (1)$$

- **Ý nghĩa:** AP càng cao thì mô hình phát hiện càng tốt, với giá trị AP từ 0 đến 1.

2.4.2 Mean Average Precision (mAP)

- **Mô tả:** mAP là trung bình của AP trên tất cả các lớp đối tượng.
- **Cách tính:** mAP được tính bằng cách lấy trung bình AP của tất cả các lớp.

- **Công thức:**

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i \quad (2)$$

- N là số lớp đối tượng.

- **Ý nghĩa:** mAP cung cấp một đánh giá tổng quan về hiệu suất của mô hình trên tất cả các lớp đối tượng.

2.4.3 Intersection over Union (IoU)

- **Mô tả:** IoU đo lường mức độ chồng lấn giữa hộp bao dự đoán và hộp bao thực tế [3, 2].
- **Cách tính:** IoU được tính bằng diện tích giao nhau chia cho diện tích hợp lại của hai hộp bao.
- **Công thức:**

$$IoU = \frac{\text{Diện tích giao nhau}}{\text{Diện tích hợp lại}} \quad (3)$$

- **Ý nghĩa:** IoU càng cao thì dự đoán càng chính xác, với giá trị từ 0 đến 1.

2.4.4 Multiple Object Tracking Accuracy (MOTA)

- **Mô tả:** MOTA đánh giá độ chính xác của hệ thống theo dõi nhiều đối tượng bằng cách tính toán lỗi phát hiện, lỗi liên kết và lỗi mất đối tượng.
- **Công thức:**

$$MOTA = 1 - \frac{\sum_t (FP_t + FN_t + IDS_t)}{\sum_t GT_t} \quad (4)$$

- FP_t : Số lượng phát hiện sai tại thời điểm t.
- FN_t : Số lượng đối tượng không phát hiện được tại thời điểm t.
- IDS_t : Số lượng lỗi liên kết (ID switches) tại thời điểm t.
- GT_t : Số lượng đối tượng thực tế tại thời điểm t.
- **Ý nghĩa:** MOTA càng cao thì hệ thống theo dõi càng chính xác, với giá trị từ $-\infty$ đến 1.

2.4.5 Multiple Object Tracking Precision (MOTP)

- **Mô tả:** MOTP đánh giá độ chính xác về không gian của các đối tượng theo dõi.
- **Công thức:**

$$\text{MOTP} = \frac{\sum_{i,t} d_{i,t}}{\sum_t c_t} \quad (5)$$

- $d_{i,t}$: Khoảng cách giữa đối tượng thực tế và đối tượng dự đoán tại thời điểm t .
- c_t : Số lượng đối tượng đúng tại thời điểm t .
- **Ý nghĩa:** MOTP càng thấp thì độ chính xác về không gian càng cao.

2.4.6 Higher Order Tracking Accuracy (HOTA)

- **Mô tả:** HOTA là một chỉ số đánh giá tổng quát hơn, kết hợp giữa MOTA và MOTP để đo lường cả độ chính xác về phát hiện và theo dõi.
- **Cách tính:** HOTA tính trung bình hình học của độ chính xác phát hiện và độ chính xác theo dõi.
- **Công thức:**

$$\text{HOTA} = \sqrt{\text{Detection Accuracy} \times \text{Tracking Accuracy}} \quad (6)$$

- **Ý nghĩa:** HOTA càng cao thì hệ thống theo dõi càng tốt, cung cấp một cách đánh giá cân bằng giữa MOTA và MOTP.

2.4.7 ID F1 Score (IDF1)

- **Mô tả:** IDF1 đo lường sự phù hợp giữa các dãy ID của đối tượng theo dõi.
- **Cách tính:** IDF1 là trung bình điều hòa của ID Precision và ID Recall.
- **Công thức:**

$$\text{IDF1} = \frac{2 \cdot \text{IDTP}}{2 \cdot \text{IDTP} + \text{IDFP} + \text{IDFN}} \quad (7)$$

- IDTP: Số lượng ID đúng (True Positives).
- IDFP: Số lượng ID sai (False Positives).
- IDFN: Số lượng ID bị bỏ sót (False Negatives).

- **Ý nghĩa:** IDF1 càng cao thì sự phù hợp ID càng tốt.

2.4.8 ID Switches (IDs)

- **Mô tả:** ID Switches đếm số lần ID của một đối tượng thay đổi trong quá trình theo dõi, phản ánh mức độ sai lệch trong việc liên kết đối tượng qua các khung hình.
- **Ý nghĩa:** Số lượng ID Switches càng thấp thì hệ thống theo dõi càng ổn định và chính xác

Chương 3. Các công trình liên quan

3.1 Bộ lọc Kalman (Kalman Filter): Ước lượng trạng thái tối ưu trong thế giới nhiễu

Rudolf Kalman, một nhà toán học và kỹ sư gốc Hungary, đã phát triển Lọc Kalman vào năm 1960. Thuật toán này là một phương pháp hiệu quả để ước lượng trạng thái của các hệ thống động, dựa trên các phép đo bị nhiễu.

Lọc Kalman đã trở thành một công cụ quan trọng trong nhiều lĩnh vực như điều khiển tự động, định vị, dự báo, và xử lý tín hiệu. Kalman đã nhận được nhiều giải thưởng và vinh danh cho những đóng góp của ông, và lọc Kalman là một di sản to lớn, tiếp tục được sử dụng rộng rãi và đóng góp đáng kể vào khoa học và công nghệ hiện đại.

Lọc Kalman là một tập hợp các phương trình toán học cung cấp một phương pháp tính toán hiệu quả (thuật toán đệ quy) để ước lượng trạng thái của một quá trình, theo cách giảm thiểu giá trị trung bình bình phương của sai số [4]. Bộ lọc này rất mạnh mẽ trong nhiều khía cạnh: nó hỗ trợ ước lượng trạng thái trong quá khứ, hiện tại và thậm chí tương lai, và nó có thể làm được điều đó ngay cả khi bản chất chính xác của hệ thống được mô hình hóa là chưa biết. Lọc Kalman rất hữu ích trong nhiều lĩnh vực, từ điều khiển tự động và định vị vệ tinh đến dự báo tài chính và xử lý tín hiệu.

Khi suy luận các phương trình cho lọc Kalman, chúng ta bắt đầu với mục tiêu tìm một phương trình tính toán ước lượng trạng thái posteriori $\hat{x}_k$ như là một tổ hợp tuyến tính của ước lượng priori $\hat{x}_k^-$ và một hiệu số có trọng số giữa phép đo thực tế z_k và dự đoán phép đo $H\hat{x}_k^-$ như được hiển thị bên dưới.

$$\hat{X}_k = K_k * Z_k + (1 - K_k) * \hat{X}_{k-1} \quad (8)$$

- $\hat{X}_k$: dự đoán hiện tại
- K_k : Lợi ích Kalman (Kalman Gain)
- Z_k : giá trị được đo
- $\hat{X}_{k-1}$: dự đoán trước đó

Theo phương trình trên, các chỉ số k là các trạng thái. Ở đây, chúng ta có thể coi nó như các khoảng thời gian rời rạc, ví dụ như $k = 1$ nghĩa là 1ms, $k = 2$ nghĩa là 2ms. Mục tiêu là tìm $\hat{X}_k$, ước lượng của tín hiệu x . Và ta cần tìm nó cho mỗi k tiếp theo. Ta có Z_k là giá trị đo lường. Và K_k được gọi là "Kalman Gain" (điểm mấu chốt của tất cả những điều này), và $\hat{X}_{k-1}$ là ước lượng của tín hiệu ở trạng thái trước đó.

Thành phần duy nhất chưa biết trong phương trình này là Kalman Gain. Bởi vì, chúng ta có các giá trị đo lường, và chúng ta đã có tín hiệu ước lượng trước đó.

Mặt khác, hãy giả sử K_k bằng 0.5, chúng ta sẽ có được gì? Đó là một phép trung bình đơn giản! Nói cách khác, ta nên tìm các hệ số K_k tối ưu hơn ở mỗi trạng thái. Kết luận là: "Lọc Kalman tìm ra hệ số trung bình tối ưu nhất cho mỗi trạng thái tiếp theo. Đồng thời, nó cũng "nhớ" một chút về các trạng thái trước đó."

Cơ chế hoạt động

Lọc Kalman hoạt động dựa trên hai ý tưởng chính:

1. **Mô hình hóa hệ thống:** Chúng ta cần một mô hình toán học mô tả hành vi của hệ thống. Mô hình này sẽ bao gồm các phương trình mô tả cách trạng thái của hệ thống thay đổi theo thời gian và cách các phép đo được tạo ra từ trạng thái này.
2. **Dự đoán và cập nhật:** Lọc Kalman sẽ sử dụng mô hình để dự đoán trạng thái của hệ thống trong tương lai dựa trên thông tin hiện tại. Sau đó, nó sẽ so sánh dự đoán này với phép đo thực tế và cập nhật ước lượng trạng thái dựa trên độ chính xác tương đối của dự đoán và phép đo.

Các bước trong thuật toán Lọc Kalman:

➤ Bước 1 – Xây dựng mô hình

Đây là bước quan trọng nhất. Trước hết, ta phải chắc chắn rằng các điều kiện lọc Kalman phù hợp với vấn đề hiện tại.

Khởi tạo ước lượng ban đầu cho trạng thái hệ thống và ma trận hiệp phương sai lỗi. Hai phương trình sau mô tả mô hình hệ thống trong Lọc Kalman:

$$x_k = A_{k-1}x_{k-1} + Bu_k + w_{k-1} \quad (9)$$

$$z_k = Hx_k + v_k \quad (10)$$

Theo công thức thứ nhất, giá trị x_k ước tính toán bằng cách sử dụng một phương trình ngẫu nhiên tuyến tính. Mỗi x_k là một tổ hợp tuyến tính của giá trị trước đó cộng với một tín hiệu điều khiển u_k và **process noise**.

Phương trình thứ hai cho biết bất kỳ giá trị đo lường nào (mà chúng ta không chắc về độ chính xác của nó) là một tổ hợp tuyến tính của giá trị tín hiệu và **measurement noise**.

Các biến ngẫu nhiên **process noise** và **measurement noise** đều được coi là Gaussian. Chúng được giả định là độc lập (với nhau), trắng và có phân phối xác suất chuẩn.

$$p(w) \sim N(0, Q)$$

$$p(v) \sim N(0, R)$$

Trong thực tế, ma trận hiệp phương sai của **process noise** và ma trận hiệp phương sai của **measurement noise** có thể thay đổi theo mỗi bước thời gian hoặc phép đo. Tuy nhiên, ở đây chúng ta giả định rằng chúng là hằng số.

Các thực thể **A**, **B** và **H** nhìn chung là các ma trận. Tuy nhiên, trong phần lớn các bài toán xử lý tín hiệu, người ta sử dụng các mô hình sao cho các thực thể này chỉ là các giá trị số. Ngoài ra, để đơn giản hóa, mặc dù các giá trị này có thể thay đổi giữa các trạng thái, nhưng phần lớn thời gian, có thể giả định rằng chúng là hằng số. Nếu hệ thống phù hợp với mô hình này (hầu hết các hệ thống đều phù hợp theo cách này),

thì điều duy nhất còn lại là ước lượng giá trị trung bình và độ lệch chuẩn của các hàm nhiễu w_{k-1} và v_k . Trong đời thực, không có tín hiệu nào là Gaussian thuần túy, nhưng chúng ta có thể giả định nó với một số xấp xỉ. Đây không phải là vấn đề lớn, bởi vì thuật toán Lọc Kalman cố gắng hội tụ vào các ước lượng chính xác, ngay cả khi các tham số nhiễu Gaussian được ước lượng kém.

➤ Bước 2 – Bắt đầu quá trình

Lọc Kalman ước lượng quá trình bằng cách sử dụng một dạng điều khiển hồi tiếp: bộ lọc ước lượng trạng thái của quá trình tại một thời điểm nào đó và sau đó thu thập hồi tiếp dưới dạng các phép đo (có nhiễu). Như vậy, các phương trình cho Lọc Kalman được chia thành hai nhóm: phương trình cập nhật thời gian (**time update**) và phương trình cập nhật phép đo (**measurement update**). Phương trình cập nhật thời gian chịu trách nhiệm dự đoán về phía trước (trong thời gian) các ước lượng trạng thái hiện tại và hiệp phương sai lỗi để thu được các ước lượng **priori** cho bước thời gian tiếp theo. Phương trình cập nhật phép đo chịu trách nhiệm cho hồi tiếp - tức là kết hợp phép đo mới vào ước lượng **priori** để thu được ước lượng **posteriori** được cải thiện.

Phương trình cập nhật thời gian cũng có thể được coi là phương trình dự đoán (prediction), trong khi phương trình cập nhật phép đo có thể được coi là phương trình hiệu chỉnh (correction).

Time Update (prediction)	Mesurement Update (correction)
$\hat{x}_k^- = A\hat{x}_{k-1} + Bu_k$ $P_k^- = AP_{k-1}A^T + Q$	$K_k = P_k^- H^T (HP_k^- H^T + R)^{-1}$ $\hat{x}_k = \hat{x}_k^- + K_k(z_k - H\hat{x}_k^-)$ $P_k = (1 - K_k H)P_k^-$

Bảng 1. Phương trình cập nhật thời gian

Mô hình hóa đã được thực hiện ở BƯỚC 1, do đó các ma trận **A**, **B** và **H** đã được xác định. Trong nhiều trường hợp, các ma trận **A**, **B** và **H** có thể được giả định là hằng số, và thậm chí có thể bằng 1.

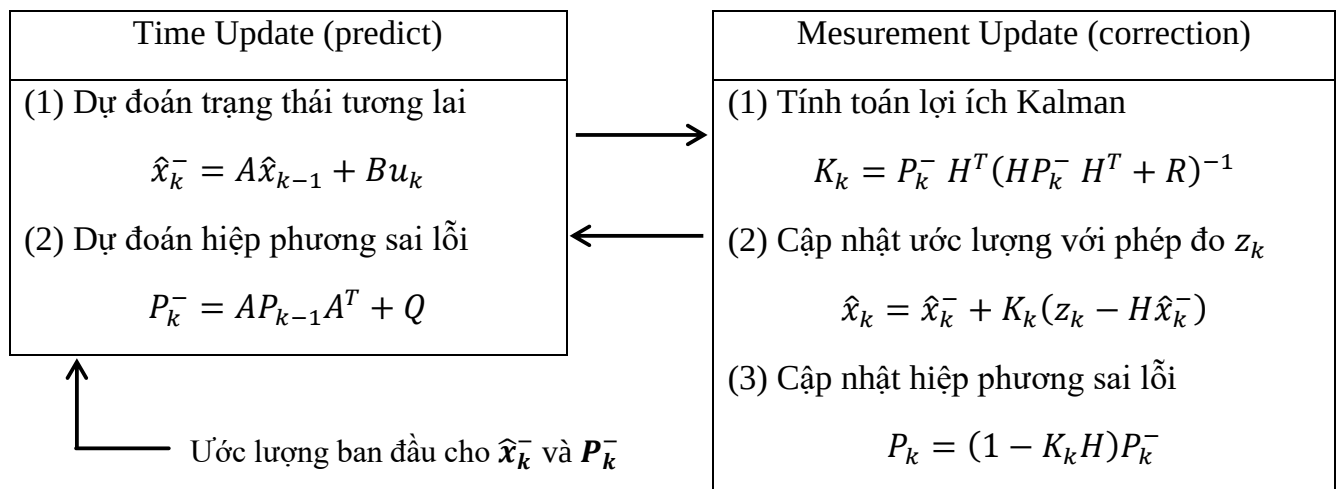
Bước đầu tiên trong quá trình cập nhật phép đo là tính toán lợi ích Kalman, K_k . Bước tiếp theo là thực hiện đo lường quá trình để thu được z_k , và sau đó tạo ra một ước lượng trạng thái posteriori bằng cách kết hợp phép đo để tính $\hat{x}_k$. Bước cuối cùng là thu được ước lượng hiệp phương sai lỗi posteriori P_k .

Trong việc triển khai thực tế của bộ lọc, hiệp phương sai nhiễu đo lường $\mathbf{R}$ thường được đo trước khi vận hành bộ lọc. Việc đo hiệp phương sai lỗi đo lường thường không khó để tìm bởi vì ta cần phải có khả năng đo lường quá trình (trong khi vận hành bộ lọc), vì vậy ta thường có thể thực hiện một số phép đo mẫu ngoại tuyến để xác định phương sai của nhiễu đo lường.

Việc xác định hiệp phương sai nhiễu quá trình $\mathbf{Q}$ thường khó khăn hơn vì ta thường không có khả năng quan sát trực tiếp quá trình đang được ước lượng. Trong trường hợp này, ta giả định rằng nhiễu quá trình có độ biến thiên lớn hơn bằng cách lựa chọn $\mathbf{Q}$ lớn hơn. Điều này tương đương với việc cho rằng mô hình hệ thống có nhiều sai số hơn so với thực tế. Chắc chắn trong trường hợp này, người ta hy vọng rằng các phép đo quá trình là đáng tin cậy.

➤ Bước 3 – Lặp lại

Sau khi đã thu thập tất cả thông tin cần thiết và bắt đầu quá trình, ta lặp lại các ước tính. Hãy nhớ rằng các ước tính trước đó sẽ là đầu vào cho trạng thái hiện tại.



Ở đây, $\hat{x}_k^-$ là ước lượng priori, có nghĩa là ước lượng sơ bộ trước khi cập nhật phép đo. Và P_k^- là hiệp phương sai lỗi priori. Ta sử dụng các giá trị priori này trong các phương trình cập nhật phép đo.

Trong các phương trình cập nhật phép đo, ta tìm $\hat{x}_k$ ước lượng của $\mathbf{x}$ tại thời điểm $\mathbf{k}$. Ngoài ra, chúng ta tìm P_k cần thiết cho ước lượng $\mathbf{k}+1$ (tương lai), cùng với $\hat{x}_k$. Lợi ích Kalman (K_k) được tính toán không cần thiết cho bước lặp tiếp theo, nhưng nó vẫn là một phần quan trọng của bộ phương trình này.

➤ Ví dụ

Bây giờ ta hãy thử ước lượng một hằng số ngẫu nhiên vô hướng, chẳng hạn như "điện áp" từ một nguồn. Vì vậy, hãy giả sử rằng nó có giá trị hằng số là $\mathbf{aV}$ (volt), nhưng tất nhiên sẽ có một số phép đo bị nhiễu ở trên và dưới $\mathbf{a}$ volt. Và giả sử rằng độ lệch chuẩn của nhiễu đo lường là 0.1 V.

Xây dựng mô hình:

$$\begin{aligned} x_k &= A_{k-1} + Bu_k + w_{k-1} & z_k &= Hx_k + v_k \\ x_k &= x_{k-1} + w_k & z_k &= x_k + v_k \end{aligned}$$

Như đã nói ở trên, các phương trình đã được rút gọn về một dạng rất đơn giản:

- Đây là một vấn đề tín hiệu một chiều, vì vậy mỗi thực thể trong mô hình của chúng ta là một giá trị số, không phải là một ma trận.
- Chúng ta không có tín hiệu điều khiển u_k nào như vậy, và nó nằm ngoài phạm vi của vấn đề.
- Vì tín hiệu là một giá trị hằng số, hằng số $\mathbf{A}$ chỉ bằng 1, bởi vì chúng ta đã biết rằng giá trị tiếp theo sẽ giống với giá trị trước đó.
- Giá trị $\mathbf{H} = 1$, bởi vì chúng ta biết rằng phép đo được tạo thành từ giá trị trạng thái và một số nhiễu. Bạn hiếm khi gặp phải các trường hợp thực tế mà $\mathbf{H}$ khác 1.

Và cuối cùng, chúng ta có các giá trị đo lường sau:

TIME (ms)	1	2	3	4	5	6	7	8	9	10
VALUE (V)	0.39	0.5	0.48	0.29	0.25	0.32	0.34	0.48	0.41	0.45

Bảng 2. Các giá trị đo lường

Bây giờ chúng ta sẽ tính giá trị $\hat{x}_k$ cho mỗi lần lặp

k	z_k	$\hat{x}_{k-1}$	P_k^-	Time update	Measurement update	$\hat{x}_k$	P_k
				$\hat{x}_k^- = \hat{x}_{k-1} = 0$ $P_k^- = P_{k-1} = 1$	$K_k = 1/(1 + 0.01)$ $= 0.99$ $\hat{x}_k = 0.99 * (0.39 - 0)$ $= 0.3861$ $P_k = (1 - 0.99) * 1$ $= 0.01$		
1	0.39	0	1			0.3861	0.01
2	0.5	0.3861	0.01			0.443	0.005
3	0.48	0.443	0.005			0.455	0.003
4	0.29	0.455	0.003			0.414	0.002
5	0.25	0.414	0.002			0.381	0.002
6	0.32	0.381	0.002			0.371	0.002
7	0.34	0.371	0.002			0.367	0.001
8	0.48	0.367	0.001			0.381	0.001
9	0.41	0.381	0.001			0.384	0.001
10	0.45	0.384	0.001			0.391	0.001

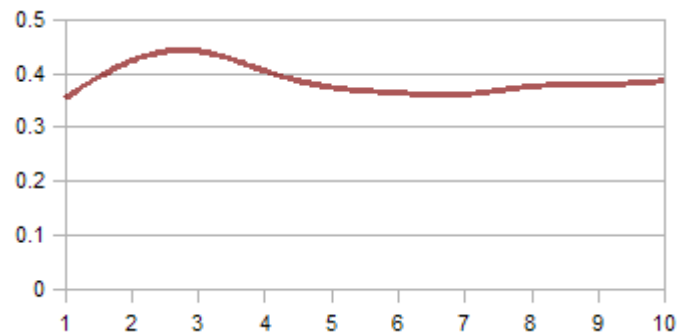
Bảng 3. Tính giá trị $\hat{x}_k$

Biểu đồ ở đây (bên phải) cho thấy thuật toán

Lọc Kalman hội tụ về giá trị điện áp thực. Ở đây, tôi hiển thị 10 lần lặp đầu tiên và chúng ta rõ ràng thấy dấu hiệu hội tụ. Sau khoảng 50 lần lặp, nó sẽ hội tụ tốt hơn.

Để cải thiện phép hội tụ trong ít bước hơn, bạn nên:

- Mô hình hóa hệ thống một cách tinh tế hơn.
- Ước lượng nhiễu chính xác hơn.



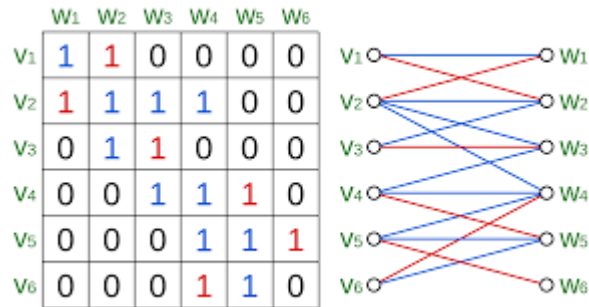
Hình 3. Biểu đồ hội tụ của bộ lọc Kalman

3.2 Thuật toán Hungarian Association

Giải thuật Hungarian, còn được gọi là giải thuật Kuhn-Munkres, là một phương pháp tối ưu để giải quyết bài toán gán (assignment problem) [5]. Bài toán gán là bài toán tìm cách gán một tập hợp công việc cho một tập hợp công nhân sao cho tổng chi phí là nhỏ nhất hoặc tổng hiệu quả là lớn nhất. Giải thuật này được Harold Kuhn phát triển năm 1955, dựa trên công trình của các nhà toán học Hungary là Dénes Kőnig và Jenő Egerváry. Sau đó, James Munkres đã cải tiến giải thuật này để đạt độ phức tạp thời gian là $\mathcal{O}(n^3)$.

Liên hệ với bài toán object tracking

Có n detection ($i = 1, 2, \dots, n$) và n track predicted ($j = 1, 2, \dots, n$). Để liên kết một detection i với một track j giả sử dựa vào 1 độ đo D - D là khoảng cách giữa i và j trong không gian vector. Bài toán đặt ra là cần liên kết mỗi detection với mỗi track tương ứng sao cho sai số của việc liên kết là nhỏ nhất.



Hình 4. Biểu diễn trực quan của giải thuật Hungarian

Các bước của Giải thuật Hungarian

Mô hình hóa lại bài toán như sau:

$$z = \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij} \rightarrow \min \quad (11)$$

Với:

$$\begin{cases} \sum_{i=1}^n x_{ij} = 1, i = 1, 2, \dots \\ \sum_{j=1}^n x_{ij} = 1, j = 1, 2, \dots \\ x_{ij} \geq 0 \end{cases} \quad (12)$$

Các số x_{ij} thỏa mãn các điều kiện trên gọi là một phương án phân công, hay ngắn gọn là một phương án, một phương án đạt cực tiểu của z được gọi là một phương án tối ưu hay lời giải của bài toán.

Dựa vào 2 định lý sau đây ta có thể tìm được x_{ij} :

- **[Định lý 1].** Giả sử ma trận chi phí của bài toán giao việc là không âm và có ít nhất n phần tử bằng 0. Hơn nữa nếu n phần tử 0 này nằm ở n hàng khác nhau và n cột khác nhau thì phương án giao cho người i thực hiện công việc tương ứng với số 0 này ở hàng i sẽ là phương án tối ưu (lời giải) của bài toán.
- **[Định lý 2].** Cho $C = [c_{ij}]$ là ma trận chi phí của bài toán giao việc (n người, n việc) và $X^* = [x_{ij}]$ là một lời giải (phương án tối ưu) của bài toán này. Giả sử C' là ma trận nhận được từ C bằng cách thêm số $\alpha \neq 0$ (dương hoặc âm) vào mỗi phần tử ở hàng r của C . Khi đó X^* cũng là lời giải của bài toán giao việc với ma trận chi phí C' .

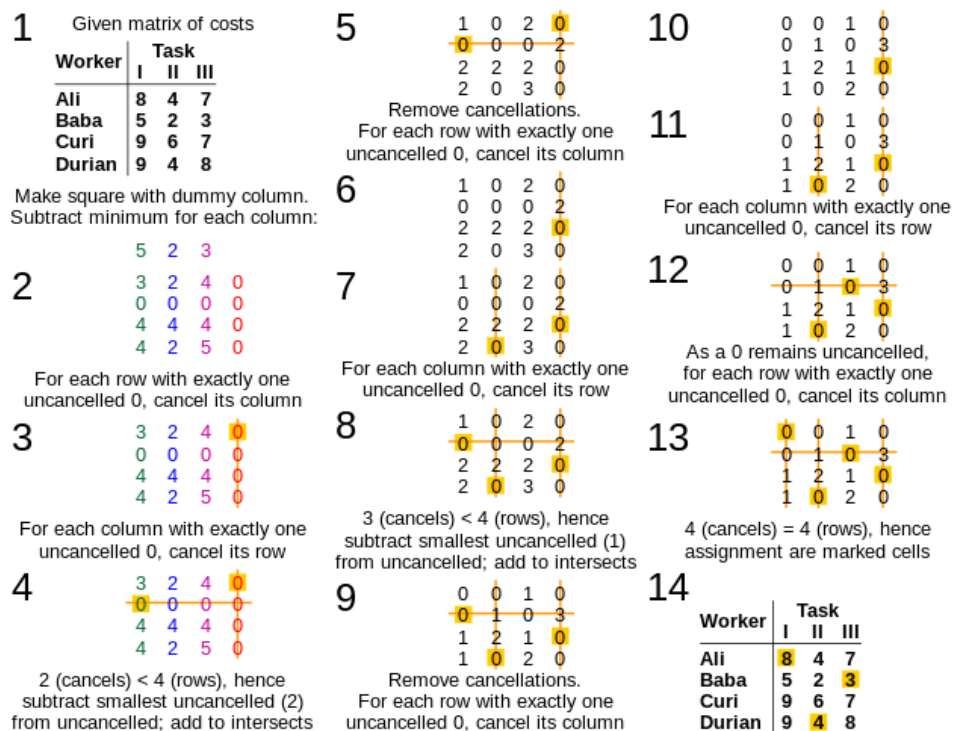
Thuật toán Hungarian dựa vào 2 định lý này, từ đó hình thành được hướng xử lý bài toán : Biến đổi ma trận (cộng trừ vào các hàng hoặc cột) để đưa về ma trận có n phần tử bằng 0 nằm ở các hàng và cột khác nhau, sau đó, lấy ra phương án tối ưu là các vị trí chứa các phần tử 0 này.

Cụ thể hơn, có thể chia thuật toán thành các bước sau:

- Bước 1 (Bước chuẩn bị). Trừ các phần tử trên mỗi hàng của C cho phần tử nhỏ nhất trên hàng đó, tiếp theo trừ các phần tử trên mỗi cột cho phần tử nhỏ nhất trên cột đó. Kết quả ta nhận được ma trận C' có tính chất: trên mỗi hàng, cột có ít nhất một phần tử 0 và bài toán giao việc với ma trận C' có cùng lời giải như bài toán với ma trận C .
- Bước 2: Vẽ một số tối thiểu các đường thẳng trên dòng và cột để đảm bảo mọi phần tử 0 đều được đi qua.
- Bước 3: Nếu có n đường thẳng được vẽ, kết thúc thuật toán và tiến hành phân công công việc. Nếu số đường thẳng được vẽ nhỏ hơn n ,

vẫn chưa tìm được phương án phân công tối ưu, tiến hành bước tiếp theo.

- Bước 4: Mỗi hàng (hoặc cột) có đường thẳng vẽ qua, ta gọi các hàng (cột) đó là các hàng (cột) thiết yếu. Các hàng (cột) còn lại là các hàng (cột) không thiết yếu. Tìm phần tử nhỏ nhất không nằm trong các hàng (cột) thiết yếu, tiến hành trừ mỗi hàng không thiết yếu cho phần tử nhỏ nhất ấy và cộng giá trị nhỏ nhất ấy cho cột thiết yếu. Ta được ma trận C'' có cùng lời giải với ma trận C' . Sau đó quay lại Bước 2.

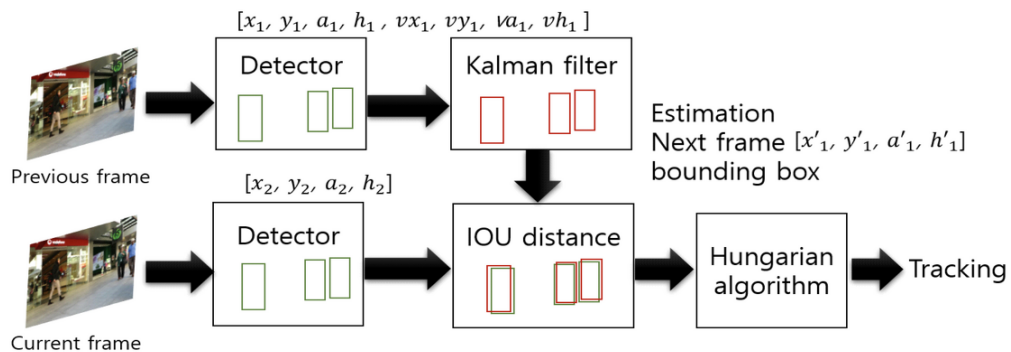


Hình 5. Các bước thực hiện giải thuật Hungarian

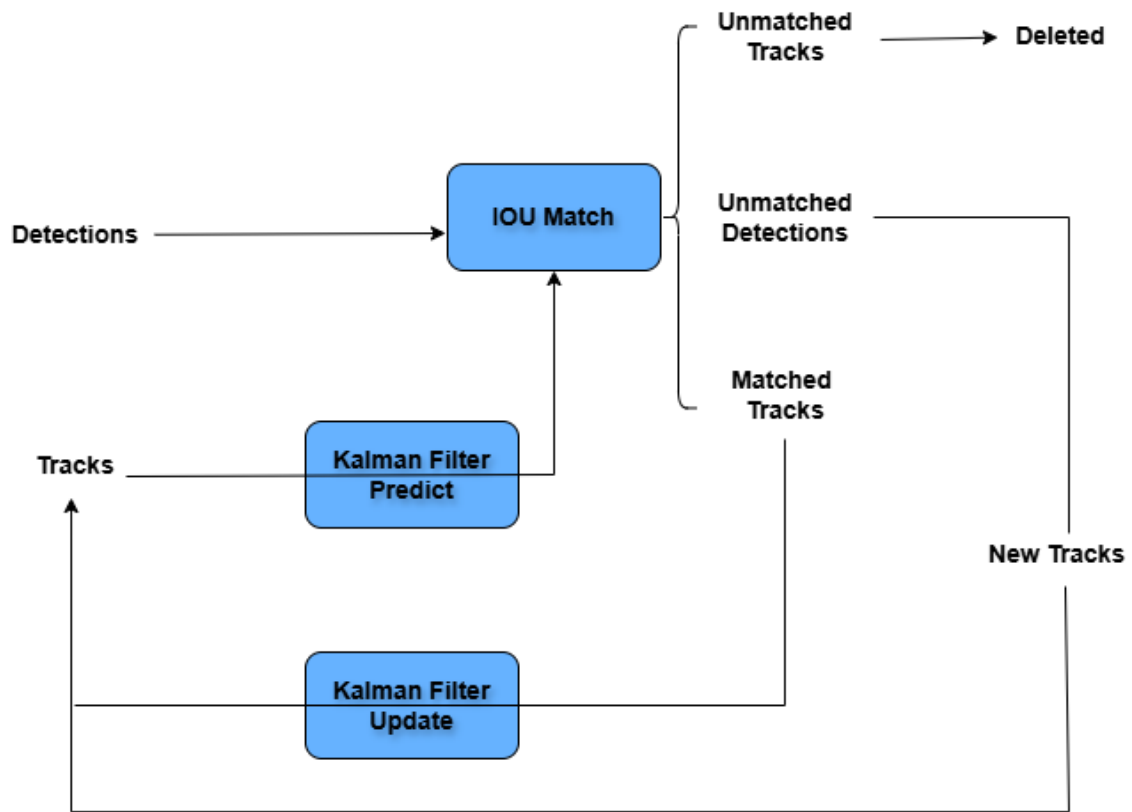
3.3 Thuật toán SORT (2017)

Thuật toán SORT (Simple Online and Realtime Tracking) , một thuật toán thuộc dạng Tracking-by-detection (hay Detection based Tracking) được giới thiệu lần đầu năm 2016, chỉnh sửa bổ sung v2 vào năm 2017, đề xuất giải pháp cho object tracking, đồng thời giải quyết cả 2 vấn đề mutiple object tracking và realtime object tracking [6, 7, 8, 9].

SORT tập trung vào vấn đề liên kết giữa các detection và track sau khi đã detect được từ frame, do đó, phần object detection có thể là bất cứ mô hình detector nào hiện nay như : YOLO (v1, v2, v3, v4, v5), SSD, RCNN (RCNN, Fast RCNN, Faster RCNN, Mask RCNN), CenterNet - Object as Point, Trong bài báo gốc, tác giả sử dụng detector là Faster Region CNN (FrCNN) với backbone là VGG16 để phát hiện đối tượng trong các khung hình và sau đó sử dụng lọc Kalman kết hợp với thuật toán Hungarian để theo dõi và dự đoán vị trí của các đối tượng trong các khung hình tiếp theo.



Hình 6. Quy trình theo dõi đối tượng sử dụng Kalman Filter và Hungarian Algorithm



Hình 7. Quy trình theo dõi đối tượng sử dụng bộ lọc Kalman và khớp IOU

3.3.1 Detection

Trong phần này, chúng ta sẽ tìm hiểu về bộ phát hiện (detector) tạo ra dữ liệu đầu vào cho thuật toán SORT. Nếu chúng ta giữ cho các thành phần theo dõi đơn giản, chúng ta sẽ lo ngại về sự suy giảm hiệu suất.

Trong SORT, các đầu vào là các hộp giới hạn (bounding boxes - bboxes) của các đối tượng được phát hiện bởi mô hình phát hiện đối tượng dựa trên mạng nơ-ron tích chập (CNN). Theo bài báo, Faster Region CNN (Faster R-CNN) được sử dụng làm khung phát hiện đối tượng

Tracker	Detector	Detection		Tracking	
		Recall	Precision	ID Sw	MOTA
MDP [12]	ACF	36.6	75.8	222	24.0
	FrRCNN(ZF)	46.2	67.2	245	22.6
	FrRCNN(VGG16)	50.1	76.0	178	33.5
Proposed	ACF	33.6	65.7	224	15.1
	FrRCNN(ZF)	41.3	72.4	347	24.0
	FrRCNN(VGG16)	49.5	77.5	274	34.0

Bảng 4. So sánh hiệu suất của các bộ theo dõi và phát hiện đối tượng

Từ kết quả so sánh hiệu suất của ba loại bộ phát hiện, bao gồm:

- Aggregate Channel Filter (ACF)
- Faster R-CNN (ZF) sử dụng ZF cho CNN
- Faster R-CNN (VGG16) sử dụng VGG16 cho CNN

Chất lượng phát hiện của các bộ phát hiện được cho là có ảnh hưởng đáng kể đến hiệu suất theo dõi. Trong so sánh hiệu suất giữa ACF và Faster R-CNN (VGG16), MOTA (một chỉ số về hiệu suất theo dõi) lần lượt là 15.1% và 34.0%. Điều này tương ứng với sự cải thiện lên đến 18.9% trong hiệu suất theo dõi do sự thay đổi bộ phát hiện được mô tả trong phần tóm tắt của bài báo.

3.3.2 Mô hình ước lượng

Nhiệm vụ của object tracking là dự đoán vị trí của đối tượng. Trong bài báo, phương pháp ước lượng chuyển động sử dụng lọc Kalman để dự đoán trạng thái hiện tại của đối tượng từ trạng thái của đối tượng ở thời điểm trước.

$$x = [u, v, s, r, \dot{u}, \dot{v}, \dot{s}]^T \quad (13)$$

- u, v : Tọa độ ngang, dọc của đối tượng.
- s, r : Diện tích và tỉ lệ khung hình của hộp giới hạn.
- $\dot{u}, \dot{v}$: Vận tốc tương ứng.
- $\dot{s}$: Tốc độ thay đổi diện tích của bounding box.

Các bước của lọc Kalman:

- Dự đoán (Prediction):

Giai đoạn này tính toán trạng thái dự kiến dựa trên trạng thái được ước lượng ở thời điểm trước và đầu vào hiện tại. Nghĩa là, từ trạng thái của đối tượng tại thời điểm trước đó và đầu vào hiện tại, dự đoán trạng thái mới.

- Hiệu chỉnh (Update):

Giai đoạn này tính toán trạng thái chính xác bằng cách kết hợp trạng thái dự đoán và trạng thái đo thực tế. Nghĩa là, từ trạng thái dự đoán và trạng thái đo được, tính toán trạng thái chính xác của đối tượng.

Ứng dụng trong object tracking:

Từ khung hình trước đó của video, trạng thái của đối tượng (gồm tọa độ bbox u, v tỉ lệ bbox s , và tỉ lệ chiều dài - chiều rộng r) được lấy qua object detector. Sau đó, trạng thái của đối tượng ở khung hình sau (gồm u', v', s') được dự đoán. (Giả định r là không đổi).

Nếu detection được kết nối với đối tượng, bbox được detection sẽ được sử dụng để cập nhật trạng thái của đối tượng qua bộ lọc Kalman, tối ưu hóa thành phần tốc độ. Nếu detection không được kết nối với đối tượng (unmatched), mô hình tốc độ tuyến tính sẽ được sử dụng để dự đoán trạng thái mà không có hiệu chỉnh.

Nếu unmatched liên tiếp trong T_{Loss} lần, ID của đối tượng sẽ bị xóa.

Chú ý:

Khi khởi tạo một track mới, giá trị hiệp phương sai của track nên để các giá trị lớn để thể hiện sự không chắc chắn của mô hình. Bên cạnh đó, các giá trị vận tốc ban đầu cũng được khởi tạo bằng 0.

Đặt ngưỡng T_{Loss} để lọc đi những track không được theo dõi nữa sau T_{Loss} frame. Trong bài báo, ngưỡng T_{Loss} được đặt bằng 1, do tác giả chú trọng vào vấn đề theo dõi frame-to-frame. Ngoài ra, giả định mô hình vận tốc không đổi cũng là một

giả định kém trong thực tế, vậy nên nếu đề T_{Loss} quá cao thậm chí có thể làm giảm độ chính xác theo dõi.

3.3.3 Kết hợp dữ liệu

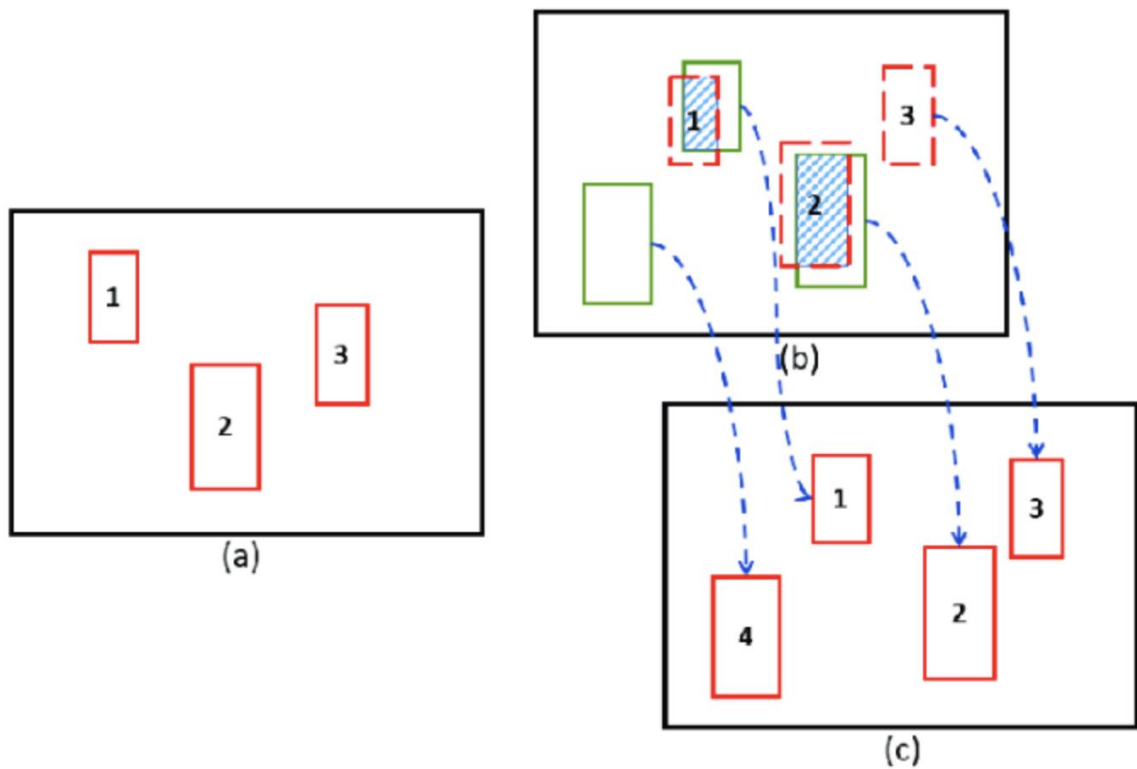
Data Association là một bước quan trọng trong phương pháp tracking-by-detection dựa trên Multiple Object Tracking (MOT) [8, 10]. Trong đó, thuật toán Hungarian thường được sử dụng như một phương pháp tối ưu hóa cho Data Association.

Bước này liên quan đến việc kết hợp các dự đoán từ Kalman filter với các đối tượng mới được phát hiện trong khung hình hiện tại.

Khoảng cách IoU (Intersection-over-Union) giữa các bounding box dự đoán và các bounding box phát hiện được sử dụng để tạo ra ma trận chi phí (cost matrix).

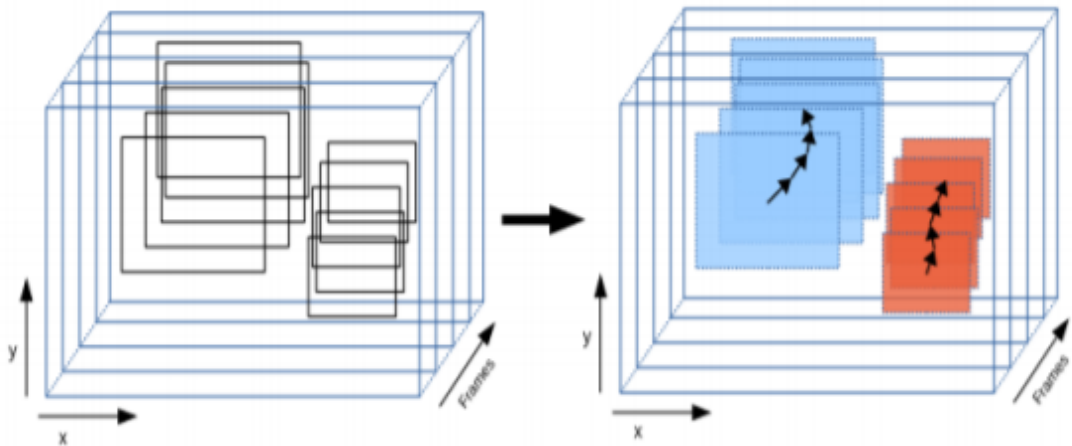
Thuật toán Hungarian sau đó được sử dụng để tối ưu hóa quá trình gán (assignment) giữa các track và các phát hiện mới. Điều này có nghĩa là thuật toán sẽ tìm cách ghép cặp các bounding box dự đoán với các bounding box phát hiện sao cho tổng chi phí là nhỏ nhất.

Để đảm bảo rằng chỉ những kết hợp có mức độ tương đồng đủ cao mới được chấp nhận, một ngưỡng IoU tối thiểu (IoU_min) được đặt ra. Nếu khoảng cách IoU giữa một phát hiện và một dự đoán nhỏ hơn ngưỡng này, kết hợp đó sẽ bị từ chối.



Hình 8. Quy trình cập nhật và theo dõi đối tượng qua các khung hình

- (a): Kết quả từ bộ lọc Kalman trước đó.
- (b): Các bounding box (hộp xanh lá cây) từ bộ phát hiện hiện tại.
- (c): Sau khi thực hiện ghép cặp (association) bằng thuật toán Hungarian, các bounding box được gán ID tương ứng.
- Toàn bộ quá trình IoU Tracker



Hình 9. Quá trình theo dõi đối tượng qua các khung hình trong không gian

Đánh giá

SORT được đánh giá trên bộ dữ liệu MOT benchmark với các chỉ số:

- MOTA ($\uparrow$): Độ chính xác theo dõi đối tượng.
- MOTP ($\uparrow$): Độ chính xác định vị đối tượng.
- FAF ($\downarrow$): Số lần báo động sai mỗi khung hình.
- MT ($\uparrow$): Số lượng đối tượng được theo dõi chủ yếu.
- ML ($\downarrow$): Số lượng đối tượng bị mất theo dõi.
- FP ($\downarrow$): Số lần phát hiện sai.
- FN ($\downarrow$): Số lần bỏ sót đối tượng.
- ID sw ($\downarrow$): Số lần thay đổi nhận diện.
- Frag ($\downarrow$): Số lần phân đoạn.

SORT đạt được hiệu suất cao với tốc độ 260Hz, kết hợp giữa tốc độ và độ chính xác, giúp nó phù hợp cho các ứng dụng thời gian thực.

Vấn đề của SORT

Thuật toán SORT, mặc dù đơn giản và hiệu quả, vẫn tồn tại một số vấn đề cần được xem xét kỹ lưỡng. Dưới đây là các điểm chính liên quan đến các vấn đề của SORT:

- ID Switches Cao: So sánh các chỉ số hiệu suất theo dõi trong bài báo SORT với các phương pháp khác, chúng ta thấy rằng hiện tượng đổi ID

(ID switch), nơi ID khác với đường đi trước đó, không tốt hơn so với các phương pháp tiên tiến nhất. Điều này có nghĩa là SORT có khả năng gặp khó khăn trong việc duy trì ID nhất quán cho các đối tượng qua các khung hình liên tiếp.

- **Không Sử Dụng Đặc Trưng Về Ngoại Hình:** SORT không sử dụng các đặc trưng về ngoại hình của đối tượng, điều này dẫn đến khó khăn trong việc xử lý các tình huống che khuất ngắn hạn và dài hạn. Các đặc trưng về ngoại hình có thể giúp phân biệt và nhận dạng lại các đối tượng khi chúng xuất hiện lại sau khi bị che khuất.

- **Xóa Track Nhanh:**

SORT được thiết kế để xóa các track một cách nhanh chóng vì hai lý do chính:

- Mô hình vận tốc không đổi là một dự báo kém về động lực thực sự của đối tượng.
- Nhận dạng lại đối tượng nằm ngoài phạm vi của nghiên cứu này.

Cụ thể, các track không được cập nhật trong hai khung hình liên tiếp sẽ bị xóa. Kết quả là, sau khi bbox của đối tượng tương ứng với đường đi hiện tại không được phát hiện trong 2 khung hình liên tiếp do che khuất, một track mới với ID mới sẽ được tạo cho bbox được phát hiện lại. Điều này dẫn đến hiện tượng đổi ID.

- **Xử Lý Che Khuất Ngắn Hạn:**

Mặc dù SORT không sử dụng các đặc trưng về ngoại hình, điều này làm cho nó khó xử lý các che khuất dài hạn, nhưng câu hỏi đặt ra là tại sao nó không được thiết kế để xử lý che khuất ngắn hạn. Một lý do có thể là thuật toán đơn giản của SORT chỉ có một điều kiện để xóa một track, và kết quả là nó không thể xử lý che khuất ngắn hạn để đối phó với các dương tính giả từ bộ phát hiện.

Cách kết hợp SORT và YOLO

Algorithm 1 Mã giả kết hợp SORT và YOLO

input:

- detections: Mảng chứa các bbox từ kết quả theo dõi SORT.
- frame: Khung hình hiện tại từ video.
- classnames: Danh sách các tên lớp đối tượng.

output:

- frame: Khung hình với các đối tượng được theo dõi, hiển thị nhãn và ID.
- track_history: Lịch sử di chuyển của các đối tượng.
- vehicle_counts: Số lượng xe trong các làn đường.

Mã giả:

1. Cập nhật các bbox từ kết quả theo dõi SORT
2. Với mỗi bbox trong kết quả theo dõi:
 3. Lấy $x1, y1, x2, y2, id$, và $class_{detect}$ của bbox
 4. Chuyển đổi $x1, y1, x2, y2, id, class_{detect}$ sang số nguyên
 5. Tính w, h từ $x2 - x1, y2 - y1$
 6. Tính cx, cy từ $(x1 + w // 2), (y1 + h // 2)$
 7. Lấy tên lớp của đối tượng từ $classnames[class_{detect}]$
 8. Nếu id không có trong $track_{history}$:
 9. Khởi tạo $track_{history}[id]$ là danh sách rỗng
 10. Thêm (cx, cy) vào $track_{history}[id]$
 11. Nếu độ dài $track_{history}[id] > 10$:
 12. Xóa điểm đầu tiên của $track_{history}[id]$
 13. Nếu id không có trong $track_{labels}$:
 14. Gán $track_{labels}[id] = class_{detect_name}$
 15. Vẽ đường giữa các điểm trong $track_{history}[id]$
 16. Với mỗi làn đường trong $lanes_{coordinates}$:
 17. Lấy $zone_{in}$ và $zone_{out}$ của làn đường
 18. Nếu (cx, cy) nằm trong $zone_{in}$:
 19. Nếu id không có trong $zone'_{counters}[lane]['in']$:
 20. Thêm id vào $zone'_{counters}[lane]['in']$
 21. Tăng $vehicle'_{counts}[lane]['in']$
 22. Nếu (cx, cy) nằm trong $zone_{out}$:
 23. Nếu id không có trong $zone'_{counters}[lane]['out']$:
 24. Thêm id vào $zone'_{counters}[lane]['out']$
 25. Tăng $vehicle'_{counts}[lane]['out']$
 26. Hiển thị nhãn $(class_{detect_name} - id)$ trên khung hình
 27. Vẽ bbox và tâm của đối tượng trên khung hình
 28. Hiển thị số lượng xe vào và ra cho mỗi làn đường trên khung hình

3.4 ByteTrack

3.4.1 Giới thiệu

ByteTrack: Theo dõi đối tượng đa mục tiêu bằng cách liên kết mỗi hộp phát hiện là một bài báo được trình bày tại ECCV2022 bởi Yifu Zhang và cộng sự. Nhờ vào khung làm việc toàn diện và tính đơn giản tương đối, BYTETrack đã được nhiều nhà nghiên cứu tiếp theo áp dụng cho các bộ theo dõi MOT của họ (ví dụ: Bot-SORT, SMILEtrack).

3.4.2 Ý tưởng chính của ByteTrack

Ý tưởng chính của BYTETrack rất đơn giản - giữ lại các hộp có điểm thấp không thuộc nền để thực hiện bước liên kết thứ cấp giữa khung hình trước và khung hình tiếp theo dựa trên sự tương đồng của chúng với các tracklet [11, 9, 10]. Điều này giúp cải thiện tính nhất quán theo dõi bằng cách giữ lại các hộp giới hạn liên quan mà nếu không sẽ bị loại bỏ do điểm tin cậy thấp (do che khuất hoặc thay đổi hình dạng). Khung làm việc tổng quát làm cho BYTETrack dễ dàng thích ứng với bất kỳ phương pháp phát hiện đối tượng nào (YOLO, RCNN) hoặc các thành phần liên kết thực thể (IoU, tương đồng đặc trưng).

3.4.3 Đổi mới chính của ByteTrack

Sự đổi mới chính của BYTETrack là giữ lại các hộp phát hiện có độ tin cậy thấp không thuộc nền mà thường bị loại bỏ sau khi lọc phát hiện ban đầu và sử dụng các hộp điểm thấp này cho một bước liên kết thứ cấp. Thông thường, các hộp phát hiện bị che khuất có điểm tin cậy thấp hơn ngưỡng, nhưng vẫn chứa một số thông tin về các đối tượng, khiến điểm tin cậy của chúng cao hơn các hộp thuần nền. Do đó, các hộp có điểm tin cậy thấp này vẫn có ý nghĩa để theo dõi trong giai đoạn liên kết.

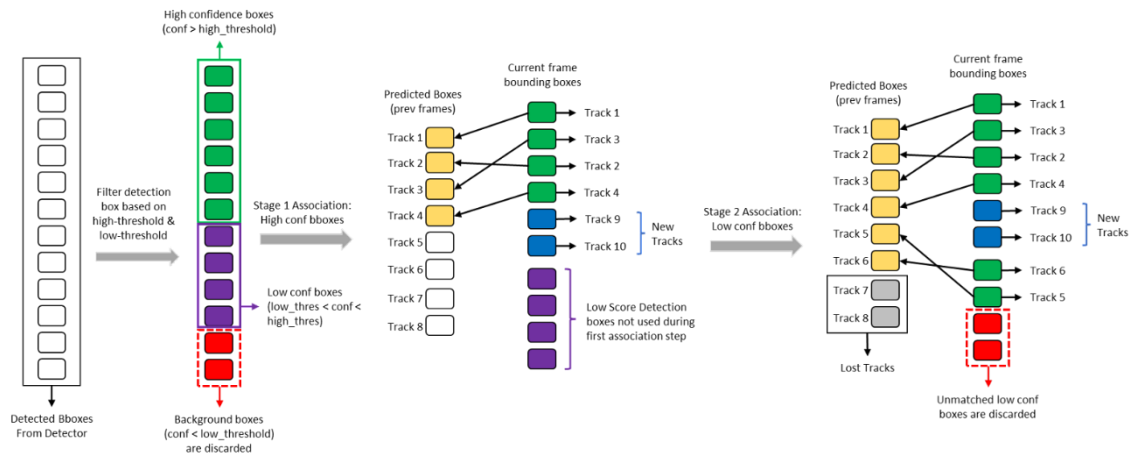
3.4.4 Các bước của thuật toán ByteTrack

Thuật toán BYTETrack thực hiện các bước sau:

- Bước 1 - Lọc các hộp phát hiện
 - **Đầu vào:** Các hộp phát hiện từ bộ phát hiện đối tượng.
 - **Quá trình:**

- Chia các hộp phát hiện thành ba loại dựa trên ngưỡng điểm cao (high threshold) và ngưỡng điểm thấp (low threshold):
 - **Hộp phát hiện có điểm cao (high confidence boxes):**
Điểm số lớn hơn ngưỡng điểm cao.
 - **Hộp phát hiện có điểm thấp (low confidence boxes):**
Điểm số nằm giữa ngưỡng điểm thấp và ngưỡng điểm cao.
 - **Hộp nền (background boxes):** Điểm số nhỏ hơn ngưỡng điểm thấp.
- **Kết quả:**
 - Giữ lại các hộp phát hiện có điểm cao và hộp phát hiện có điểm thấp.
 - Loại bỏ các hộp nền.
- Bước 2 - Liên kết giai đoạn 1 (Stage 1 Association)
- **Đầu vào:** Các hộp phát hiện có điểm cao từ bước 1 và các hộp dự đoán từ các khung hình trước.
- **Quá trình:**
 - Liên kết các hộp phát hiện có điểm cao với các hộp dự đoán từ các khung hình trước dựa trên sự tương đồng (IoU hoặc cosine similarity).
 - Sử dụng thuật toán Hungarian hoặc cascade matching để thực hiện liên kết.
- **Kết quả:**
 - Giữ lại các hộp phát hiện và hộp dự đoán không khớp để xử lý trong giai đoạn tiếp theo.
- Bước 3 - Liên kết giai đoạn 2 (Stage 2 Association)
- **Đầu vào:** Các hộp phát hiện có điểm thấp từ bước 1 và các hộp dự đoán còn lại từ bước 2.
- **Quá trình:**

- Liên kết các hộp phát hiện có điểm thấp với các hộp dự đoán còn lại từ các khung hình trước.
- Sử dụng thuật toán Hungarian hoặc cascade matching với ngưỡng thấp hơn.
- **Kết quả:**
 - Đánh dấu các hộp dự đoán không khớp là các track bị mất.
 - Loại bỏ các hộp phát hiện có điểm thấp không được liên kết (coi chúng như là hộp nền).
- Bước 4 - Khởi tạo các track mới
- **Đầu vào:** Các hộp phát hiện có điểm cao không được liên kết từ bước 2.
- **Quá trình:**
 - Sử dụng các hộp phát hiện có điểm cao không được liên kết để khởi tạo các track mới.
 - Các hộp dự đoán không được liên kết sẽ được lưu lại để xem xét khôi phục nếu đối tượng xuất hiện lại trong các khung hình tiếp theo.
- **Kết quả:**
 - Khởi tạo các track mới từ các hộp phát hiện có điểm cao không được liên kết.



Hình 10. Quy trình của BYTETrack

3.4.5 Các thành phần trong ByteTrack

Tổng quan về đoạn mã

- **STrack:** Lớp này đại diện cho một đối tượng đang được theo dõi. Nó chứa các thuộc tính và phương thức để dự đoán, cập nhật và khởi tạo đối tượng.
- **BYTETracker:** Lớp này quản lý các đối tượng đang được theo dõi, bao gồm việc cập nhật, kết hợp và quản lý các đối tượng bị mất và bị loại bỏ.

Algorithm 1 Mã giả của BYTE

```

1: Input: Một chuỗi video  $V$ ; bộ phát hiện đối tượng  $Det$ ; ngưỡng điểm phát hiện  $\tau$ 
2: Output: Các đường theo dõi  $T$  của video
3: Khởi tạo:  $T \leftarrow \emptyset$ 
4: for mỗi khung hình
5:    $D_k \leftarrow Det(f_k)$ 
6:    $D_{high} \leftarrow \emptyset$ 
7:    $D_{low} \leftarrow \emptyset$ 
8:   for mỗi  $d$  trong  $D_k$  do
9:     if  $d.score > \tau$  then
10:        $D_k \leftarrow D_{high} \cup \{d\}$ 
11:     else
12:        $D_{low} \leftarrow D_{low} \cup \{d\}$ 
13:     end if
14:   end for
15:   /* Dự đoán vị trí mới của các đường theo dõi */  $T$ 
16:   for mỗi  $t$  trong  $T$  do
17:      $t \leftarrow \text{KalmanFilter}(t)$ 
18:   end for
19:   /* Lần kết hợp đầu tiên */
20:   Kết hợp  $T$  và  $D_{high}$  sử dụng Similarity#1
21:    $D_{remain} \leftarrow$  Các hộp đối tượng còn lại từ  $D_{high}$ 
22:    $T_{remain} \leftarrow$  Các đường theo dõi còn lại từ  $T$ 
23:   /* Lần kết hợp thứ hai */
24:   Kết hợp  $T_{remain}$  và  $D_{low}$  sử dụng Similarity#2
25:    $T_{re-remain} \leftarrow$  Các đường theo dõi còn lại từ  $T_{remain}$ 
26:   /* Xóa các đường theo dõi không khớp */
27:    $T \leftarrow T \setminus T_{re-remain}$ 
28:   /* Khởi tạo các đường theo dõi mới */
29:   for mỗi  $d$  trong  $D_{remain}$  do
30:      $T \leftarrow T \cup \{d\}$ 
31:   end for
32: end for
33: return  $T$ 

```

3.4.7 Kết luận

ByteTrack là một phương pháp theo dõi đối tượng đa mục tiêu hiệu quả, đơn giản và dễ dàng thích ứng với nhiều phương pháp phát hiện đối tượng khác nhau. Bằng cách giữ lại các hộp phát hiện có điểm tin cậy thấp cho bước liên kết thứ cấp, ByteTrack cải thiện tính nhất quán và hiệu quả của quá trình theo dõi đối tượng trong video.

3.5 So sánh giữa ByteTrack và SORT

	Ưu điểm	Nhược điểm
SORT	• Đơn giản và nhanh chóng: Thuật toán đơn giản, dễ triển khai và có thể xử lý trong thời gian thực với tốc độ nhanh. • Tiết kiệm tài nguyên: Ít tốn tài nguyên tính toán hơn so với ByteTrack. • Hiệu quả trong điều kiện đơn giản: Hoạt động hiệu quả trong các điều kiện ít phức tạp và khi không có nhiều che khuất.	• Độ chính xác thấp hơn: Khả năng theo dõi không cao bằng ByteTrack trong các điều kiện phức tạp như đám đông hoặc khi đối tượng bị che khuất. • Dễ mất dấu đối tượng: Khi đối tượng bị che khuất hoặc nằm ngoài khung hình, SORT dễ bị mất dấu đối tượng hơn ByteTrack. • Giới hạn đặc điểm: Chỉ sử dụng vị trí và kích thước của hộp bao, không tận dụng được các đặc điểm phức tạp khác như hình dạng và màu sắc.
BYTE	• Độ chính xác cao: ByteTrack có khả năng theo dõi chính xác hơn trong các điều kiện phức tạp như đám đông hoặc khi đối tượng bị che khuất. • Giảm mất dấu đối tượng: Nhờ vào việc theo dõi cả các đối tượng không được phát hiện trong một số khung hình, giảm thiểu khả năng mất dấu đối tượng. • Khả năng phân loại: Sử dụng các mô hình học máy giúp phân loại và quyết định đối tượng cần tiếp tục theo dõi.	• Phức tạp và tốn tài nguyên: Thuật toán phức tạp hơn SORT và yêu cầu nhiều tài nguyên tính toán hơn. • Thời gian xử lý lâu hơn: Do sử dụng các thuật toán học máy, thời gian xử lý có thể chậm hơn so với các phương pháp đơn giản như SORT.

Kết luận

Mặc dù thuật toán ByteTrack phức tạp và sử dụng nhiều tài nguyên hơn SORT, ByteTrack đã khắc phục nhiều nhược điểm của SORT bằng cách sử dụng các kỹ thuật phức tạp hơn, bao gồm việc theo dõi các đối tượng không phát hiện, sử dụng nhiều đặc điểm phức tạp như kích thước, hình dạng, màu sắc và vị trí, và áp dụng các mô hình học máy để phân loại và quyết định đối tượng cần tiếp tục theo dõi. Điều này giúp ByteTrack duy trì độ chính xác cao hơn và giảm thiểu khả năng mất dấu đối tượng trong các điều kiện phức tạp như đám đông hoặc khi đối tượng bị che khuất, mà SORT thường gặp khó khăn.

Ví dụ :

SORT :



Hình 11. Xe sắp bị che khuất (SORT)



Hình 12. Xe bị đổi id sau khi bị cây che khuất (SORT)

BYTETRACK:



Hình 13. Xe sắp bị che khuất (BYTETRACK)



Hình 14. Xe vẫn giữ được ID (BYTETRACK)

Sau khi bị cây che khuất xe vẫn không đổi ID, cho thấy BYTETRACK đã làm tốt việc tracking.

3.6 Mô hình Darknet-19

Darknet-19 là một kiến trúc mạng nơ-ron sâu được thiết kế để xử lý các tác vụ nhận dạng và phát hiện đối tượng trong thời gian thực

Cấu trúc: Darknet-19 bao gồm 19 lớp tích chập (convolutional layers) và 5 lớp pooling. Nó sử dụng các hàm kích hoạt Leaky ReLU và các lớp batch normalization để cải thiện khả năng học.

Ứng dụng: Darknet-19 được sử dụng trong các phiên bản đầu tiên của YOLO (You Only Look Once), một trong những mô hình phát hiện đối tượng nhanh nhất và hiệu quả nhất.

3.7 Mô hình Darknet-53

Darknet-53 là phiên bản mở rộng và cải tiến của Darknet-19, được thiết kế để nâng cao hiệu suất và độ chính xác trong việc phát hiện đối tượng.

Cấu trúc: Darknet-53 có 53 lớp tích chập và sử dụng ResNet (Residual Network) để tạo ra các kết nối tắt (skip connections), giúp giảm thiểu vấn đề gradient vanishing và tăng tốc độ huấn luyện.

Ứng dụng: Darknet-53 được sử dụng trong YOLOv3, mang lại hiệu suất cao hơn đáng kể so với các phiên bản YOLO trước đó.

3.8 Mô hình CSPNet

CSPNet (Cross Stage Partial Network) là một kiến trúc mạng được thiết kế để cải thiện hiệu quả tính toán và tăng cường khả năng học của mô hình.

Cấu trúc: CSPNet chia khối tích chập thành hai phần, sau đó kết hợp chúng lại bằng cách sử dụng các kết nối tắt. Điều này giúp giảm tải tính toán và tăng cường khả năng học của mô hình.

Ứng dụng: CSPNet được sử dụng trong các phiên bản mới của YOLO (chẳng hạn như YOLOv4 và YOLOv5), cải thiện cả tốc độ và độ chính xác so với các phiên bản trước.

3.9 Mô hình EfficientDet

EfficientDet sử dụng EfficientNet làm backbone để trích xuất các đặc trưng từ ảnh đầu vào. EfficientNet là một mạng tích chập hiệu quả được tối ưu hóa dựa trên kích thước mạng, độ rộng và độ sâu, giúp tối ưu hóa cả tốc độ và độ chính xác.

EfficientDet sử dụng BiFPN (Bidirectional Feature Pyramid Network) để tổng hợp các đặc trưng từ các mức độ phân giải khác nhau của ảnh. BiFPN sử dụng các kết nối tắt hai chiều và trọng số có thể học để tổng hợp thông tin từ các tầng khác nhau, giúp mô hình học được những đặc trưng tốt hơn cho việc phát hiện đối tượng.

EfficientDet tích hợp CSPNet (Cross Stage Partial Network) để giảm tải tính toán và tăng cường khả năng học. CSPNet chia khối tích chập thành hai phần và kết hợp chúng lại bằng các kết nối tắt, giúp duy trì thông tin và tối ưu hóa quá trình học.

Head trong EfficientDet là các khối tích chập được thiết kế để dự đoán các bounding box và các lớp đối tượng. Các head này được tối ưu hóa để cải thiện độ chính xác của các dự đoán.

EfficientDet được sử dụng rộng rãi trong các ứng dụng phát hiện đối tượng yêu cầu sự cân bằng giữa hiệu suất và tài nguyên tính toán. Mô hình này được áp dụng trong nhiều lĩnh vực như giám sát an ninh, phân tích video, nhận dạng khuôn mặt, và các hệ thống tự động lái, nơi cần sự chính xác cao và khả năng xử lý nhanh.

3.10 Mô hình EfficientNet-L2

EfficientNet-L2 là một phiên bản mở rộng của EfficientNet, được thiết kế để cung cấp độ chính xác cao nhất trong các tác vụ nhận dạng hình ảnh.

Cấu trúc: EfficientNet-L2 áp dụng kỹ thuật compound scaling tương tự như các phiên bản khác của EfficientNet, nhưng với kích thước và độ sâu lớn hơn để tăng cường khả năng học của mô hình.

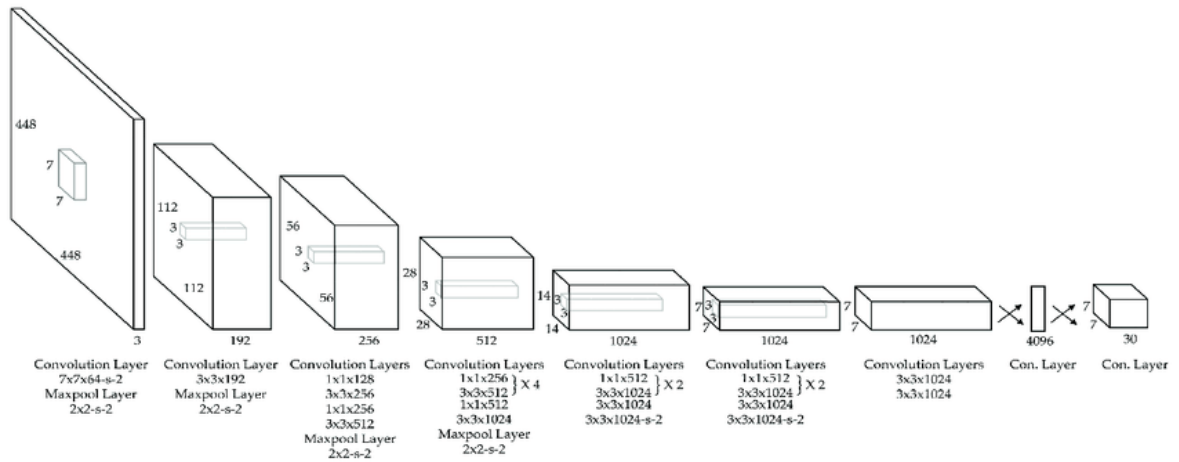
Ứng dụng: EfficientNet-L2 thường được sử dụng trong các ứng dụng yêu cầu độ chính xác cực cao, chẳng hạn như trong các cuộc thi nhận dạng hình ảnh (ImageNet).

3.11 Mô hình YOLO

3.11.1 Giới thiệu

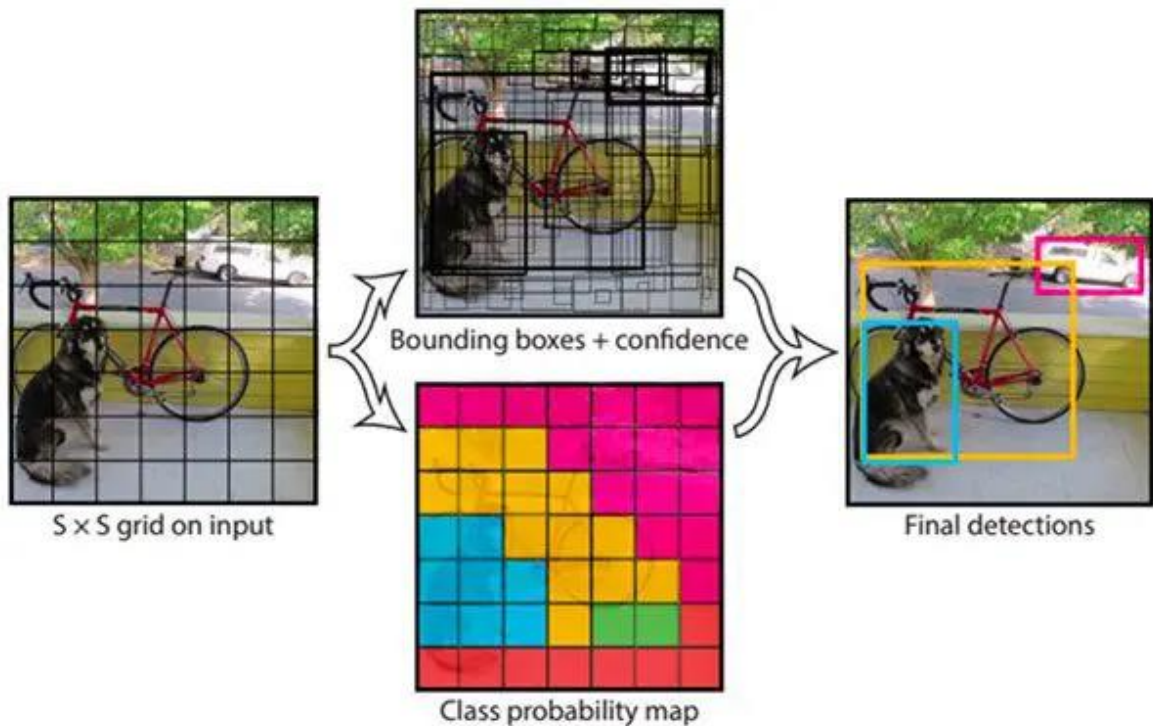
YOLO (You Only Look Once), được giới thiệu vào năm 2015 bởi Joseph Redmon và Ali Farhadi, là một thuật toán phát hiện đối tượng đột phá với khả năng xử lý real-time [3, 1]. Khác biệt với các phương pháp two-stage detection phổ biến thời bấy giờ, YOLO áp dụng single-stage detection (SSD), xem xét toàn bộ hình ảnh trong một lần duy nhất để đồng thời dự đoán bounding box và xác suất lớp của các đối tượng. Cách tiếp cận này đã mang đến hiệu suất vượt trội về tốc độ, cho phép YOLO xử lý hình ảnh với tốc độ khung hình cao, phù hợp cho các ứng dụng real-time như robot, xe tự lái và giám sát video. Bên cạnh tốc độ ấn tượng, YOLO cũng đạt được độ chính xác cao, cạnh tranh với các phương pháp two-stage detection phức tạp hơn.

Nhờ những ưu điểm vượt trội này, YOLO nhanh chóng trở thành một trong những mô hình được ưa chuộng và có ảnh hưởng nhất trong lĩnh vực thị giác máy tính, tạo tiền đề cho sự phát triển của các thể hệ mô hình object detection hiện đại.



Hình 15. Kiến trúc mô hình

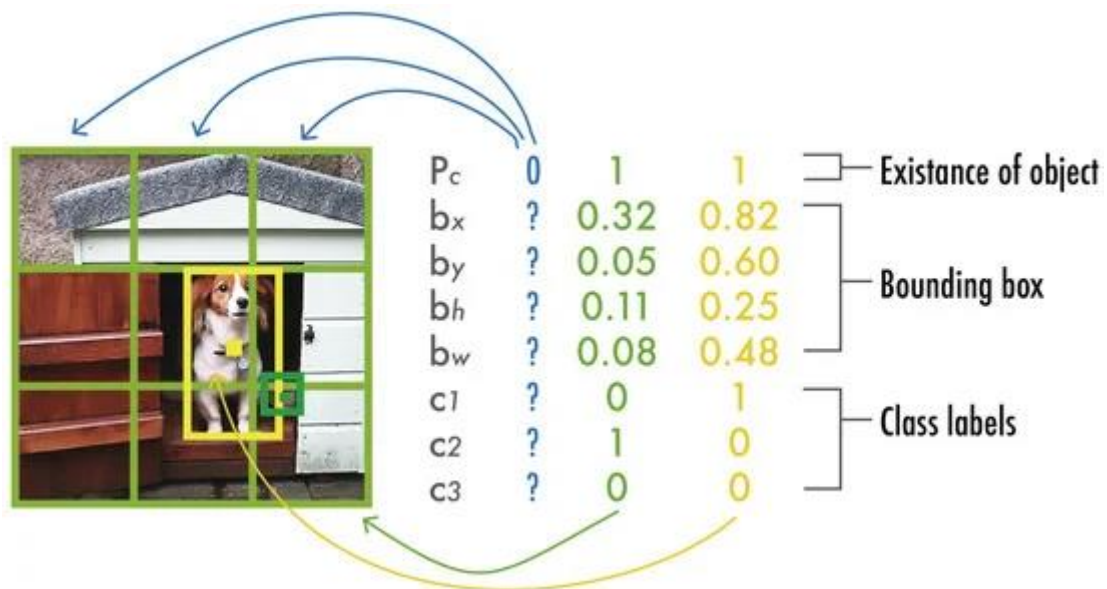
3.11.2 Cách YOLO hoạt động



Hình 16. Quy trình phát hiện đối tượng sử dụng mạng YOLO

YOLO đã hợp nhất các bước phát hiện đối tượng bằng cách phát hiện tất cả các bounding box đồng thời. Để thực hiện điều này, YOLO chia hình ảnh đầu vào thành một lưới $S \times S$ (thường thì sẽ là 3×3 , 7×7 , 9×9 ...) và dự đoán B bounding box cùng lớp, cùng với độ tin cậy cho C lớp khác nhau trên mỗi ô lưới. Mỗi dự đoán bounding box bao gồm 5 giá trị: (x, y, w, h, c) , trong đó c là độ tin cậy của bounding box, phản ánh mức độ tin tưởng của mô hình rằng bounding box chứa một đối tượng và độ chính xác của bounding box đó. Tọa độ x và y là tâm của bounding box so với ô lưới, còn w và h lần lượt là chiều rộng và chiều cao của bounding box so với toàn bộ hình ảnh. Đầu ra của YOLO là một tensor có kích thước $S \times S \times (B * 5 + C)$, tiếp theo là bước Non-Maximum Suppression (NMS) (tùy chọn) để loại bỏ các kết quả phát hiện trùng lặp.

Trong bài báo gốc về YOLO, các tác giả đã sử dụng tập dữ liệu PASCAL VOC, bao gồm 20 lớp đối tượng. Họ chia hình ảnh đầu vào thành lưới 7×7 và giới hạn mỗi ô lưới dự đoán tối đa 2 lớp. Do đó, đầu ra của mô hình là một tensor có kích thước $7 \times 7 \times 30$, với 30 là tổng số giá trị dự đoán cho mỗi ô lưới, bao gồm thông tin về bounding box và xác suất lớp.



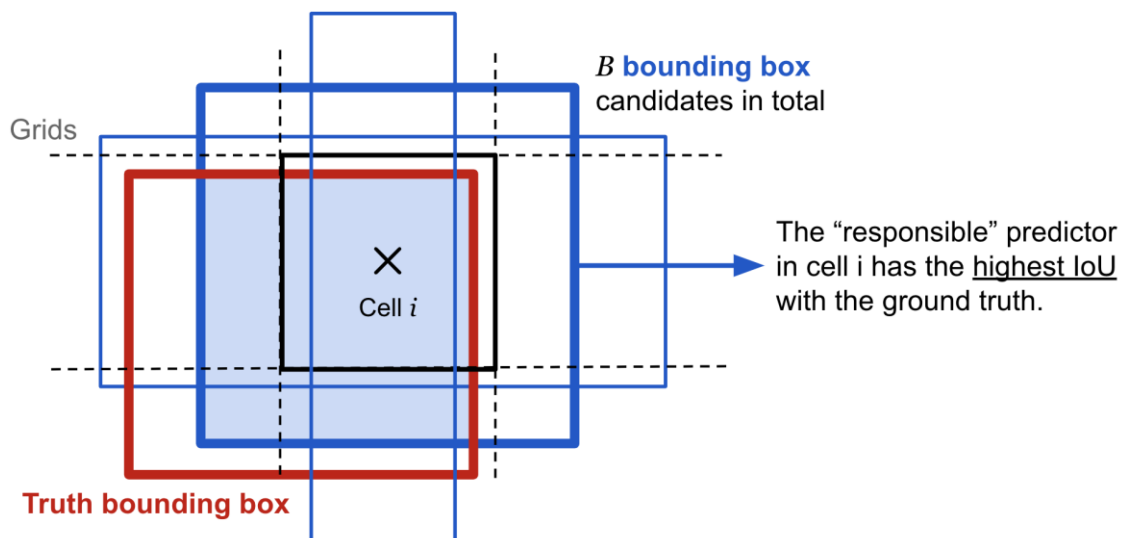
Hình 17. Cấu trúc đầu ra của mạng YOLO cho phát hiện đối tượng

3.11.3 Anchor box

Anchor box là một kỹ thuật quan trọng được giới thiệu từ YOLOv2 trở đi, giúp cải thiện đáng kể khả năng phát hiện đối tượng, đặc biệt là với các vật thể có kích thước và tỉ lệ khung hình khác nhau [3].

Để tìm được bounding box cho vật thể, YOLO sẽ cần các anchor box làm cơ sở ước lượng. Những anchor box này sẽ được xác định trước và sẽ bao quanh vật thể một cách tương đối chính xác. Sau này thuật toán regression bounding box sẽ tinh chỉnh lại anchor box để tạo ra bounding box dự đoán cho vật thể. Trong một mô hình YOLO:

- Mỗi một vật thể trong hình ảnh huấn luyện được phân bổ về một anchor box. Trong trường hợp có từ 2 anchor boxes trở lên cùng bao quanh vật thể thì ta sẽ xác định anchor box mà có IoU với ground truth bounding box là cao nhất.



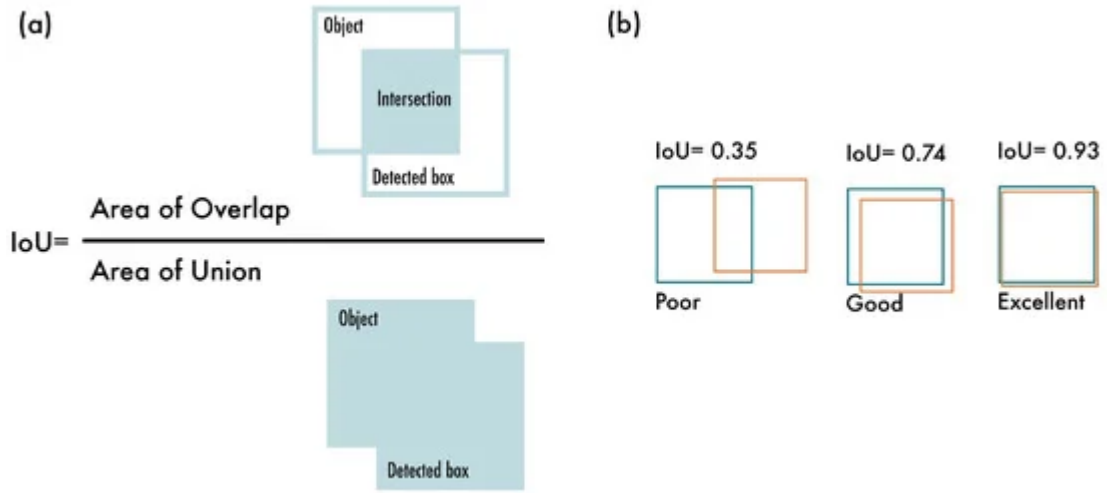
Hình 18. Cơ chế chọn bounding box trong mạng YOLO dựa trên IoU

- Mỗi một vật thể trong hình ảnh huấn luyện được phân bổ về một cell trên feature map mà chứa điểm midpoint của vật thể.

Như vậy khi xác định một vật thể ta sẽ cần xác định 2 thành phần gắn liền với nó là (**cell**, **anchor box**). Không chỉ riêng mình **cell** hoặc chỉ mình **anchor box**.

3.11.4 Hàm tính IoU

Intersection over Union (IoU): Mục tiêu của phát hiện đối tượng là xác định vị trí chính xác các đối tượng trong ảnh bằng cách dự đoán bounding box. IoU là tỷ lệ giữa diện tích giao nhau và diện tích hợp nhất của bounding box dự đoán và bounding box thực tế [3, 2]. Nó đo mức độ chồng chéo giữa bounding box dự đoán và bounding box thực tế. Tiêu chuẩn COCO xem xét nhiều ngưỡng IoU để đánh giá hiệu suất của mô hình ở các mức độ chính xác định vị khác nhau.



Hình 19. Hàm tính IoU

3.11.5 Hàm loss

Hàm lỗi trong YOLO được tính trên việc dự đoán và nhãn mô hình để tính. Cụ thể hơn nó là tổng độ lỗi của 3 thành phần con sau :

- $\mathcal{L}_{loc}$: Độ lỗi của dự đoán tọa độ tâm, chiều dài, rộng của boundary box (x, y, w, h) - Localization loss.
- $\mathcal{L}_{conf}$: Độ lỗi của việc dự đoán bounding box đó chứa object so với nhãn thực tế tại ô vuông đó - Confidence loss.
- $\mathcal{L}_{cls}$: Độ lỗi của việc dự đoán loại nhãn của object - Classification loss.

$$\mathcal{L}_{loc} = \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{obj} [((x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 + (\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2)] \quad (14)$$

$$\mathcal{L}_{\text{conf}} = \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \quad (15)$$

$$\mathcal{L}_{\text{cls}} = \sum_{i=0}^{S^2} \sum_{c \in \mathcal{C}} \mathbb{1}_i^{\text{obj}} (p_i(c) - \hat{p}_i(c))^2 \quad (16)$$

- λ_{coord} : trọng số điều chỉnh cho localization loss.
- S : kích thước của lưới (grid size).
- B : số lượng bounding box dự đoán cho mỗi ô lưới.
- $\mathbb{1}_i^{\text{obj}}$: hàm indicator có giá trị 0,1 nhằm xác định xem cell i có chứa vật thể hay không. Bằng 1 nếu chứa và 0 nếu không chứa.
- $\mathbb{1}_{ij}^{\text{obj}}$: cho biết bounding box thứ j của cell i có phải là bounding box của vật thể được dự đoán hay không? (xem hình ...).
- (x_i, y_i) : tọa độ trung tâm của bounding box dự đoán.
- (w_i, h_i) : chiều rộng và chiều cao của bounding box dự đoán.
- $(\hat{x}_i, \hat{y}_i, \hat{w}_i, \hat{h}_i)$: giá trị thực tế của bounding box.
- C_i : độ tin cậy của bounding box dự đoán.
- $\hat{C}_i$: độ tin cậy thực tế (1 nếu có đối tượng, 0 nếu không có đối tượng).
- λ_{noobj} : trọng số điều chỉnh cho confidence loss khi không có đối tượng.
- $p_i(c)$: xác suất dự đoán của lớp c .
- $\hat{p}_i(c)$: xác suất thực tế của lớp c .

3.11.6 Dự đoán Bounding box

Để dự báo bounding box cho một vật thể chúng ta dựa trên một phép biến đổi từ anchor box và cell [3, 12, 13].

YOLOv2 và YOLOv3 dự đoán bounding box sao cho nó sẽ không lệch khỏi vị trí trung tâm quá nhiều. Nếu bounding box dự đoán có thể đặt vào bất kỳ phần nào

của hình ảnh, như trong mạng regional proposal network, việc huấn luyện mô hình có thể trở nên không ổn định.

Cho một anchor box có kích thước (p_w, p_h) tại cell nằm trên feature map với góc trên cùng bên trái của nó là (c_x, c_y) , mô hình dự đoán 4 tham số (t_x, t_y, t_w, t_h) trong đó 2 tham số đầu là độ lệch (offset) so với góc trên cùng bên trái của cell và 2 tham số sau là tỷ lệ so với anchor box. Và các tham số này sẽ giúp xác định bounding box dự đoán b có tâm (b_x, b_y) và kích thước (b_w, b_h) thông qua hàm sigmoid và hàm exponential như các công thức bên dưới:

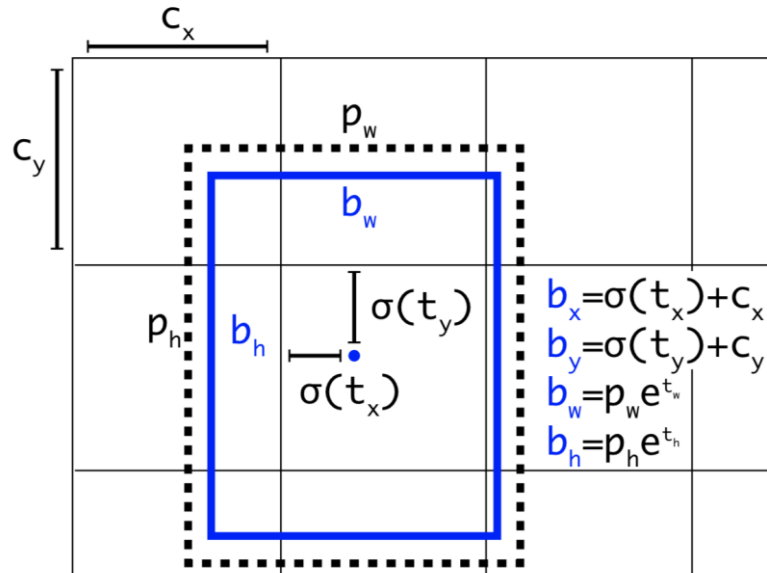
$$b_x = \sigma(t_x) + c_x \quad (17)$$

$$b_y = \sigma(t_y) + c_y \quad (18)$$

$$b_w = p_w e^{t_w} \quad (19)$$

$$b_h = p_h e^{t_h} \quad (20)$$

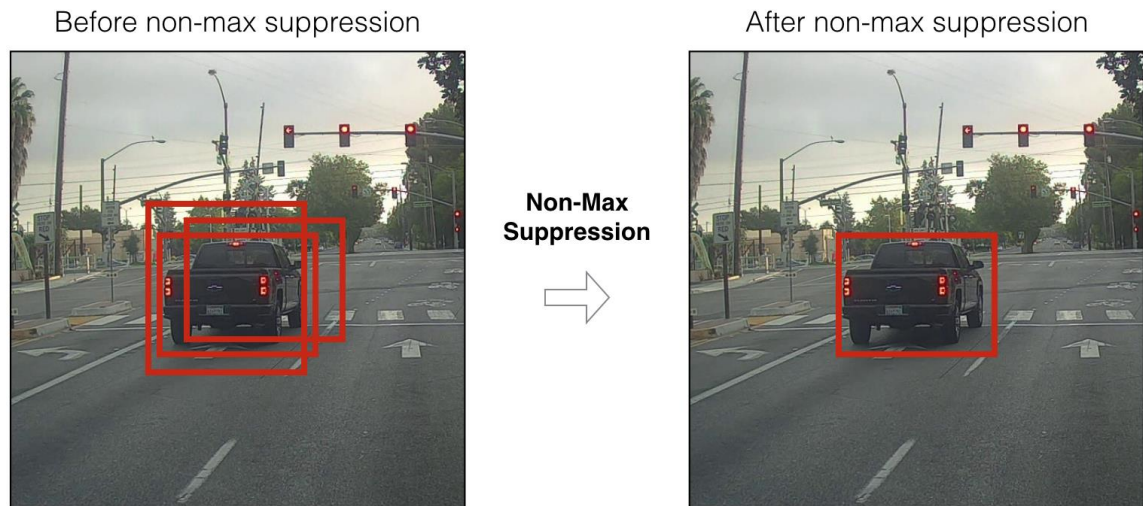
Ngoài ra do các tọa độ đã được hiệu chỉnh theo width và height của bức ảnh nên luôn có giá trị nằm trong ngưỡng $[0, 1]$. Do đó khi áp dụng hàm sigmoid giúp ta giới hạn được tọa độ không vượt quá xa các ngưỡng này.



Hình 20. Phương trình tính toán bounding box trong YOLO

3.11.7 Non-max suppression

Do thuật toán YOLO dự báo ra rất nhiều bounding box trên một bức ảnh nên đối với những cell có vị trí gần nhau, khả năng các khung hình bị overlap là rất cao. Trong trường hợp đó YOLO sẽ cần đến non-max suppression để giảm bớt số lượng các khung hình được sinh ra một cách đáng kể [3, 2].



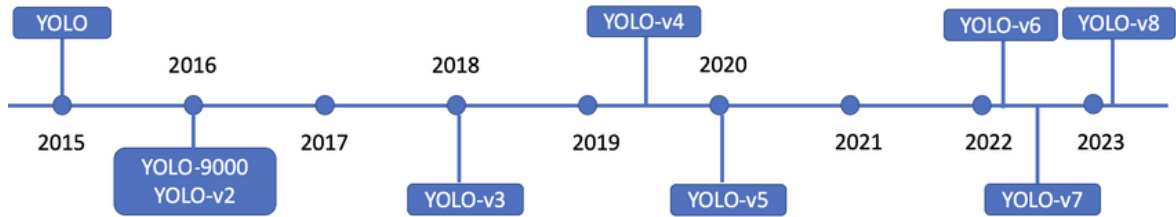
Hình 21. Hiệu ứng của Non-Max Suppression trong phát hiện đối tượng

Các bước của non-max suppression:

- Bước 1: Đầu tiên chúng ta sẽ tìm cách giảm bớt số lượng các bounding box bằng cách lọc bỏ toàn bộ những bounding box có xác suất chứa vật thể nhỏ hơn một ngưỡng threshold nào đó, thường là 0.5.
- Bước 2: Đối với các bounding box giao nhau, non-max suppression sẽ lựa chọn ra một bounding box có xác suất chứa vật thể là lớn nhất. Sau đó tính toán chỉ số giao thoa IoU với các bounding box còn lại.

Nếu chỉ số này lớn hơn ngưỡng threshold thì điều đó chứng tỏ 2 bounding boxes đang overlap nhau rất cao. Ta sẽ xóa các bounding có xác suất thấp hơn và giữ lại bounding box có xác suất cao nhất. Cuối cùng, ta thu được một bounding box duy nhất cho một vật thể.

3.11.8 Quá trình phát triển



Hình 22. Quá trình phát triển của YOLO

3.11.8.1 YOLO/YOLOv1

Phiên bản đầu tiên của YOLO đã tạo nên một bước đột phá trong lĩnh vực phát hiện đối tượng với khả năng nhận diện vật thể nhanh chóng và hiệu quả [3, 2, 1].

Tuy nhiên, phiên bản đầu tiên này cũng còn nhiều giới hạn:

- **Khó khăn trong việc phát hiện các vật thể nhỏ trong nhóm:**
Ví dụ, YOLO gặp khó khăn trong việc phát hiện từng người trong một đám đông tại sân vận động. Điều này là do mỗi ô lưới trong kiến trúc YOLO được thiết kế để phát hiện một đối tượng duy nhất.
- **Khả năng phát hiện hình dạng mới hoặc bất thường bị hạn chế:** YOLO có thể gặp khó khăn trong việc nhận diện các vật thể có hình dạng khác thường hoặc chưa từng xuất hiện trong tập dữ liệu huấn luyện.
- **Hàm mất mát chưa tối ưu:** Hàm mất mát được sử dụng để đánh giá hiệu suất phát hiện coi trọng các lỗi ở bounding box nhỏ và lớn như nhau, dẫn đến việc định vị không chính xác.

3.11.8.2 YOLOv2/YOLO9000:

Ra đời vào năm 2016, YOLOv2 được phát triển với mục tiêu nâng cấp YOLO trở nên tốt hơn, nhanh hơn và mạnh mẽ hơn [3, 2, 1]. Những cải tiến đáng chú ý bao gồm:

1. **Kiến trúc mới:** Thay thế kiến trúc ban đầu bằng Darknet-19, một mạng nơ-ron convolutional hiệu quả hơn với 19 lớp convolutional.
2. **Batch normalization:** Thêm lớp batch normalization giúp tăng hiệu suất thêm 2% mAP. Lớp batch normalization này có tác dụng điều chuẩn, ngăn chặn overfitting.
3. **Độ phân giải đầu vào cao hơn:** YOLOv2 sử dụng trực tiếp độ phân giải đầu vào 448x448 thay vì 224x224 như YOLOv1. Điều này giúp mô hình điều chỉnh bộ lọc để hoạt động tốt hơn với hình ảnh có độ phân giải cao. Phương pháp này đã tăng độ chính xác thêm 4% mAP sau khi được huấn luyện 10 epochs trên dữ liệu ImageNet.
4. **Sử dụng Anchor Boxes cho các lớp Convolutional:** Thay vì dự đoán trực tiếp tọa độ bounding box như YOLOv1, YOLOv2 đơn giản hóa bài toán bằng cách thay thế lớp fully connected bằng anchor boxes (bounding box được xác định trước). Mặc dù độ chính xác giảm nhẹ, recall (khả năng phát hiện vật thể) tăng thêm 7%, tạo dư địa cho những cải tiến tiếp theo.
5. **Gom cụm chiều (Dimensionality Clustering):** YOLOv2 tự động tìm anchor boxes bằng cách sử dụng thuật toán gom cụm K-means với $k=5$, thay vì lựa chọn thủ công. Phương pháp này mang lại sự cân bằng tốt giữa recall và precision (độ chính xác của bounding box) của mô hình.
6. **Đặc trưng chi tiết (Fine-grained features):** Dự đoán của YOLOv2 tạo ra feature map 13x13, đủ để phát hiện vật thể lớn. Để phát hiện vật thể nhỏ hơn, kiến trúc được sửa đổi bằng cách chuyển đổi feature map 26x26x512 thành 13x13x2048 và nối với feature map ban đầu. Cách tiếp cận này đã cải thiện hiệu suất mô hình thêm 1%.

3.11.8.3 YOLOv3

YOLOv3 được xây dựng dựa trên YOLOv2 với những cải tiến gia tăng đáng kể [3, 2, 1]. Thay đổi chính nằm ở kiến trúc mạng mới: Darknet-53. Đây là mạng nơ-

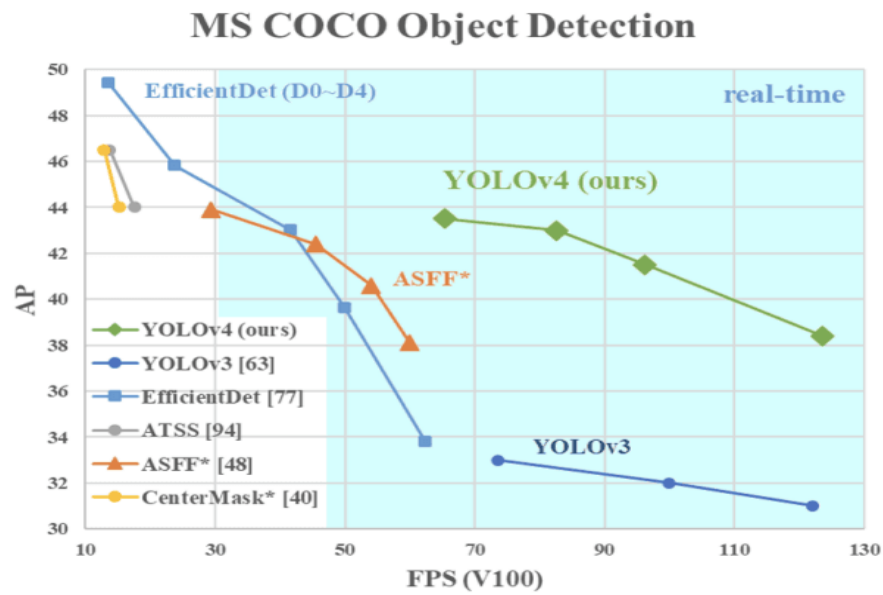
ron 106 lớp, kết hợp mạng upsampling và residual block. So với Darknet-19 (backbone của YOLOv2), Darknet-53 lớn hơn, nhanh hơn và chính xác hơn. Kiến trúc mới này mang lại nhiều lợi ích:

1. **Dự đoán Bounding Box tốt hơn:** YOLOv3 sử dụng mô hình hồi quy logistic để dự đoán objectness score (độ tin cậy của vật thể) cho mỗi bounding box, giúp tăng độ chính xác.
2. **Dự đoán lớp chính xác hơn:** Thay vì softmax như YOLOv2, YOLOv3 sử dụng các bộ phân loại logistic độc lập để dự đoán lớp của bounding box. Cách tiếp cận này hữu ích khi xử lý các miền phức tạp với nhãn chồng chéo (ví dụ: Người → Cầu thủ bóng đá). Softmax giới hạn mỗi bounding box chỉ có một lớp, điều không phải lúc nào cũng đúng.
3. **Dự đoán chính xác hơn ở các scale khác nhau:** YOLOv3 thực hiện ba dự đoán ở các scale khác nhau cho mỗi vị trí trong hình ảnh đầu vào, hỗ trợ upsampling từ các lớp trước đó. Chiến lược này cho phép thu thập thông tin chi tiết và ngữ nghĩa có ý nghĩa hơn cho hình ảnh đầu ra chất lượng cao hơn.

3.11.8.4 YOLOv4

YOLOv4, ra mắt với mục tiêu tối ưu hóa tốc độ và độ chính xác cho phát hiện đối tượng, đã vượt trội so với các phiên bản trước và các mô hình object detection tiên tiến khác [3, 2, 1].

Hình ảnh minh họa cho thấy YOLOv4 vượt trội YOLOv3 về tốc độ (FPS) thêm 10% và độ chính xác thêm 12%.



Hình 23. So sánh hiệu suất phát hiện đối tượng trên MS COCO

Được thiết kế đặc biệt cho hệ thống sản xuất và tối ưu hóa cho tính toán song song, YOLOv4 sử dụng CSPDarknet53 làm backbone. Đây là mạng nơ-ron convolutional với 29 lớp convolutional, sử dụng bộ lọc 3x3 và khoảng 27.6 triệu tham số.

So với YOLOv3, kiến trúc này bổ sung các cải tiến sau:

- **Khối Spatial Pyramid Pooling (SPP):** Mở rộng đáng kể trường tiếp nhận, tách biệt các đặc trưng ngữ cảnh liên quan nhất mà không ảnh hưởng đến tốc độ mạng.
- **PANet:** Thay thế Feature Pyramid Network (FPN) trong YOLOv3, YOLOv4 sử dụng PANet để tổng hợp tham số từ các mức phát hiện khác nhau, tăng cường luồng thông tin và độ chính xác.
- **Data augmentation:** Sử dụng kỹ thuật mosaic kết hợp bốn hình ảnh huấn luyện, cùng với phương pháp huấn luyện tự đối kháng (self-adversarial training), tạo ra dữ liệu đa dạng hơn và nâng cao khả năng tổng quát hóa của mô hình.

- **Tối ưu hóa siêu tham số:** Áp dụng thuật toán di truyền để lựa chọn siêu tham số tối ưu, tinh chỉnh hiệu suất của mô hình.

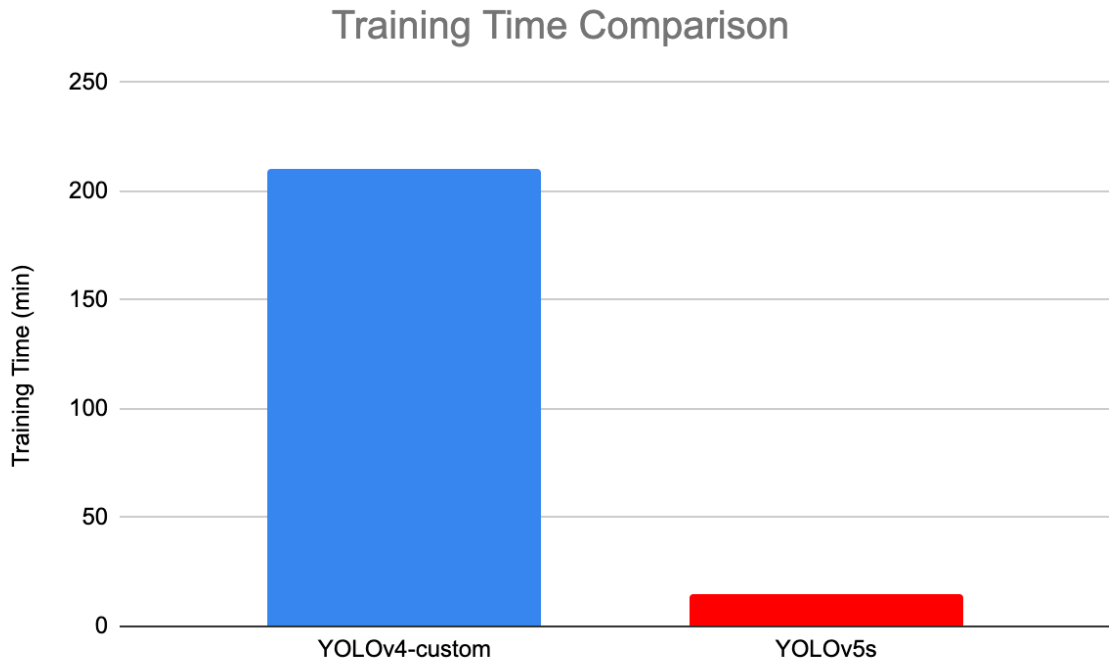
3.11.8.5 YOLOv5

Khác với các phiên bản trước, YOLOv5 không có bài báo nghiên cứu chính thức [3, 2, 1]. Đây cũng là phiên bản YOLO đầu tiên được triển khai bằng PyTorch thay vì Darknet.

Được phát hành bởi Glenn Jocher vào tháng 6 năm 2020, YOLOv5, tương tự như YOLOv4, sử dụng CSPDarknet53 làm backbone. Phiên bản này bao gồm năm kích thước mô hình khác nhau: YOLOv5s (nhỏ nhất), YOLOv5m, YOLOv5l và YOLOv5x (lớn nhất).

Một trong những cải tiến đáng chú ý trong kiến trúc YOLOv5 là việc tích hợp lớp Focus, được biểu diễn bởi một lớp duy nhất thay thế ba lớp đầu tiên của YOLOv3. Sự tích hợp này giúp:

- **Giảm số lượng lớp và tham số:** Giúp mô hình gọn nhẹ hơn.
- **Tăng tốc độ xử lý:** Cải thiện tốc độ xử lý thuận (forward) và nghịch (backward) mà không ảnh hưởng đáng kể đến mAP (mean Average Precision).



Hình 24. So sánh thời gian huấn luyện giữa YOLOv4-custom và YOLOv5s

3.11.8.6 YOLOv6

YOLOv6 (MT-YOLOv6), được phát triển bởi Meituan - một công ty thương mại điện tử Trung Quốc, hướng đến ứng dụng công nghiệp với thiết kế hiệu quả, thân thiện với phần cứng và hiệu suất cao [3, 2, 1].

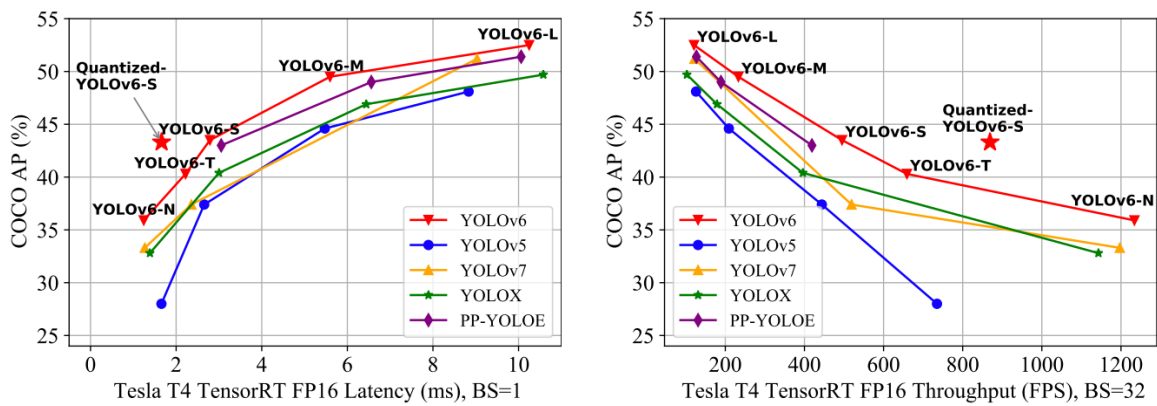
Được viết bằng PyTorch, phiên bản mới này không phải là một phần của YOLO chính thức nhưng vẫn được đặt tên là YOLOv6 vì backbone của nó được lấy cảm hứng từ kiến trúc YOLO một giai đoạn ban đầu.

YOLOv6 mang đến ba cải tiến đáng kể so với YOLOv5:

- **Thiết kế backbone và neck thân thiện với phần cứng:** Tối ưu hóa kiến trúc để hoạt động hiệu quả trên các thiết bị phần cứng khác nhau.
- **Decoupled head hiệu quả:** Tách biệt quá trình dự đoán bounding box và phân loại, giúp tăng cường độ chính xác.

- **Chiến lược huấn luyện hiệu quả hơn:** Áp dụng các kỹ thuật huấn luyện mới để tối ưu hóa hiệu suất mô hình.

YOLOv6 đạt kết quả ấn tượng về độ chính xác và tốc độ trên tập dữ liệu COCO [2] so với các phiên bản YOLO trước đó, như minh họa trong hình.



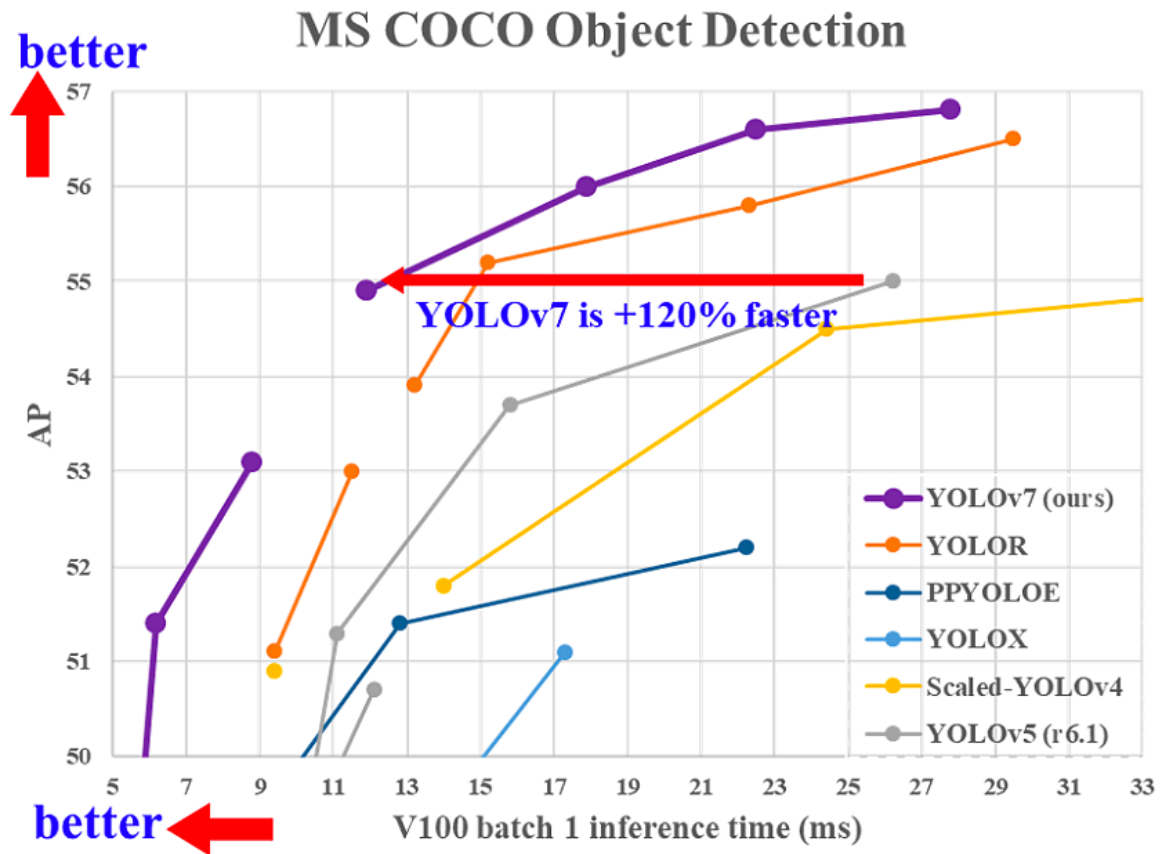
Hình 25. So sánh hiệu suất và độ trễ của các mô hình YOLO trên MS COCO

- **YOLOv6-N:** Đạt 35.9% AP trên COCO với tốc độ 1234 FPS trên GPU NVIDIA Tesla T4.
- **YOLOv6-S:** Đạt AP 43.3% với tốc độ 869 FPS, thiết lập một tiêu chuẩn mới về hiệu suất.
- **YOLOv6-M và YOLOv6-L:** Cũng đạt hiệu suất chính xác cao hơn, lần lượt là 49.5% và 52.3%, với tốc độ suy luận tương tự.

3.11.8.7 YOLOv7

YOLOv7, được ra mắt vào tháng 7 năm 2022 trong bài báo "Trained bag-of-freebies sets new state-of-the-art for real-time object detectors", đã tạo nên bước tiến đáng kể trong lĩnh vực phát hiện đối tượng [3, 2, 1]. Phiên bản này vượt qua tất cả các mô hình trước đó về độ chính xác và tốc độ.

Hình minh họa cho thấy thời gian suy luận của YOLOv7 nhanh hơn so với các mô hình object detection real-time khác.

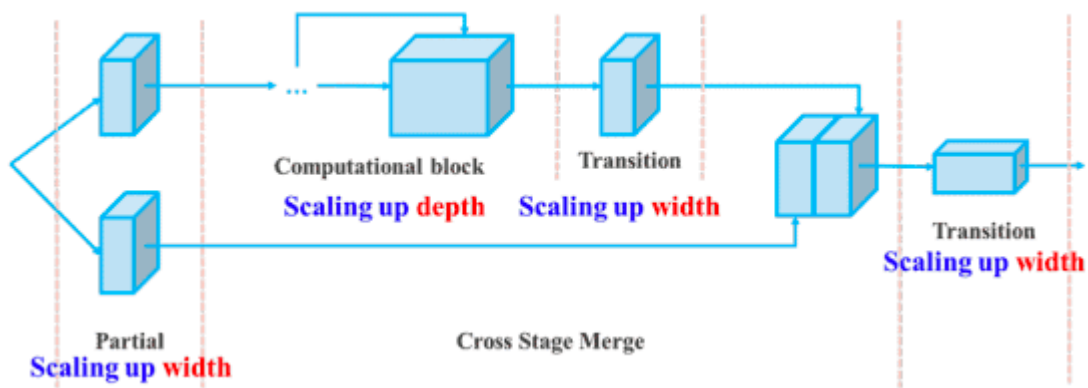


Hình 26. So sánh hiệu suất phát hiện đối tượng của các mô hình YOLO trên MS COCO

YOLOv7 mang đến những thay đổi lớn về (1) kiến trúc và (2) "trainable bag-of-freebies":

- **Cải tiến Kiến trúc:**

- **Mạng tổng hợp lớp hiệu quả mở rộng (E-ELAN):** YOLOv7 cải tiến kiến trúc bằng cách tích hợp E-ELAN, cho phép mô hình học các đặc trưng đa dạng hơn, từ đó cải thiện khả năng học.
- **Kết hợp kiến trúc:** YOLOv7 mở rộng kiến trúc bằng cách kết hợp kiến trúc từ các mô hình trước đó như YOLOv4, Scaled YOLOv4 và YOLO-R. Điều này cho phép mô hình đáp ứng nhu cầu về tốc độ suy luận khác nhau.



Hình 27. Kiến trúc mạng với các bước mở rộng độ sâu và chiều rộng

- **Trainable Bag-of-Freebies:** Thuật ngữ **bag-of-freebies** đề cập đến việc cải thiện độ chính xác của mô hình mà không làm tăng chi phí huấn luyện. Đây là lý do YOLOv7 không chỉ tăng tốc độ suy luận mà còn tăng độ chính xác phát hiện.

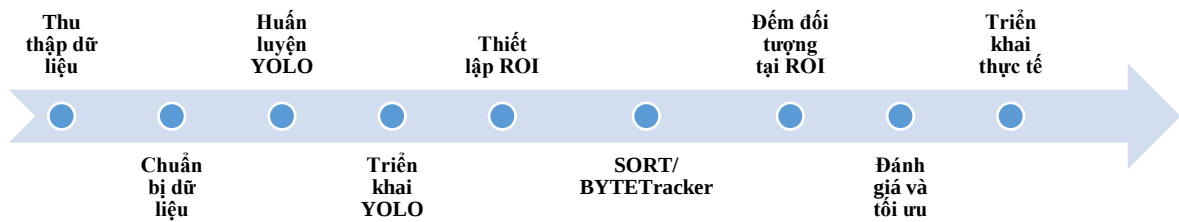
3.11.8.8 YOLOv8

Kiến trúc YOLOv8 kế thừa YOLOv5 nhưng có một số điều chỉnh nhỏ, đáng chú ý nhất là việc sử dụng mô-đun C2f thay cho mô-đun CSPNet. Mô-đun C2f (CSPNet followed by two convolutional networks) là một biến thể của CSPNet, kết hợp hiệu quả các đặc trưng ngữ cảnh với thông tin cấp cao để cải thiện độ chính xác phát hiện.

YOLOv8 được thiết kế theo hướng anchor-free, dựa trên mô hình YOLOv5 [14, 2, 1]. Điều này trái ngược với YOLOX, vốn cũng thử nghiệm anchor-free trên YOLOv5 nhưng cuối cùng lại sử dụng YOLOv3 vì cho rằng YOLOv5 đã được tối ưu hóa quá mức cho huấn luyện dựa trên anchor.

YOLOv8 cũng giới thiệu Decoupled head, cho phép xử lý độc lập các tác vụ objectness, phân loại và dự đoán bounding box. Cách tiếp cận này giúp mỗi nhánh tập trung vào nhiệm vụ riêng, từ đó cải thiện độ chính xác tổng thể. Lớp sigmoid được sử dụng làm lớp cuối cùng cho dự đoán objectness score, trong khi softmax được dùng cho phân loại.

Chương 4. Giải pháp đề xuất



Biểu đồ 1. Quy trình tổng quát

4.1 Phân tích dữ liệu

4.1.1 Dữ liệu test

Để xây dựng một mô hình deep learning hiệu quả cho việc theo dõi và phân tích luồng giao thông, việc phân tích kỹ lưỡng tập dữ liệu là bước không thể thiếu. Video mẫu city_traffic.mp4 được sử dụng làm đại diện cho dữ liệu giao thông đô thị mà mô hình sẽ học cách xử lý.

Thông số dữ liệu video mẫu:

- Độ dài: 3 phút.
- Độ phân giải: 1920x1080 (Full HD).
- Tần suất khung hình: 30FPS (khung hình mỗi giây).
- Kích thước tệp: Khoảng 100 MB.

Phân tích thống kê:

- Các loại phương tiện: ô tô, xe buýt và các loại xe khác.
- Số góc: 4
- Điều kiện giao thông: Video mẫu đại diện cho tình hình giao thông ở mức độ thấp, không có sự cố hoặc ùn tắc đáng kể, phản ánh điều kiện bình thường của giao thông đô thị.

4.1.2 Dữ liệu huấn luyện

Dữ liệu huấn luyện được sử dụng nhằm tinh chỉnh mô hình YOLO, giúp mô hình phù hợp hơn với mục tiêu nghiên cứu của nhóm. Bộ dữ liệu này bao gồm hai

thành phần chính: hình ảnh và các tệp chú thích kèm theo. Tổng cộng có 906 hình ảnh được trích xuất từ video "city_traffic.mp4" với tốc độ 10 khung hình trên giây (FPS), hình ảnh được resize về 640x640, số lượng ảnh được tăng thêm bằng cách xoay ảnh 15 độ về 2 phía. Các tệp chú thích chứa thông tin chi tiết về các đối tượng trong ảnh, bao gồm nhãn lớp và tọa độ hộp giới hạn, được chuẩn hóa theo định dạng yêu cầu của YOLO. Việc chuẩn bị dữ liệu kỹ lưỡng này giúp tăng cường độ chính xác và hiệu quả của mô hình trong quá trình nhận diện và phân loại đối tượng.

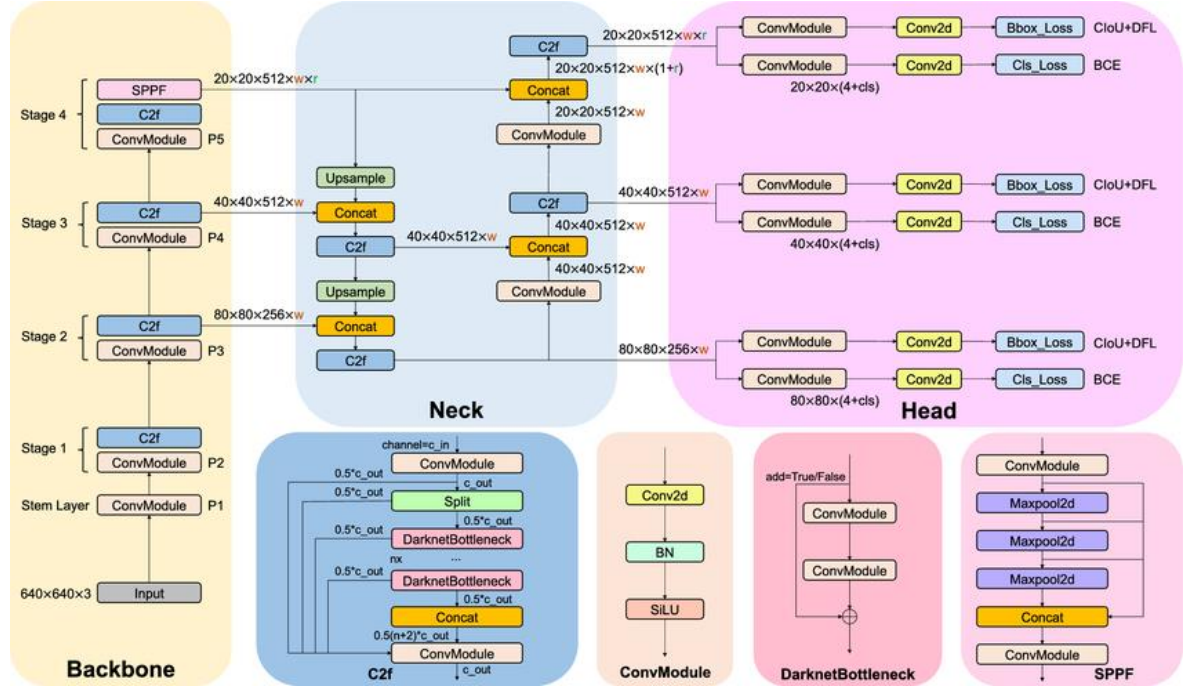
- **Tập dữ liệu:** Sau khi tăng cường ta có tập dữ liệu với 5412 ảnh với 3 class chính là 'car', 'bus' và 'truck' với nhãn lớp tương ứng là 0, 1, 2
- **Tập chú thích:** Tập chú thích là một thành phần quan trọng trong quá trình huấn luyện mô hình YOLO, chứa thông tin chi tiết về các đối tượng trong từng hình ảnh. Cấu trúc của tệp chú thích bao gồm các dòng, mỗi dòng mô tả một đối tượng trong ảnh theo định dạng chuẩn của YOLO. Mỗi dòng có năm thành phần: nhãn lớp, tọa độ x_center, tọa độ y_center, chiều rộng (width), và chiều cao (height).

<code><object-class> <x_center> <y_center> <width> <height></code>
--

- **Nhãn lớp (object-class):** Một số nguyên đại diện cho loại đối tượng, bắt đầu từ 0.
- **Tọa độ x_center (x_center):** Tọa độ x của tâm hộp giới hạn, được chuẩn hóa theo chiều rộng của ảnh (giá trị từ 0 đến 1).
- **Tọa độ y_center (y_center):** Tọa độ y của tâm hộp giới hạn, được chuẩn hóa theo chiều cao của ảnh (giá trị từ 0 đến 1).
- **Chiều rộng (width):** Chiều rộng của hộp giới hạn, được chuẩn hóa theo chiều rộng của ảnh (giá trị từ 0 đến 1).
- **Chiều cao (height):** Chiều cao của hộp giới hạn, được chuẩn hóa theo chiều cao của ảnh (giá trị từ 0 đến 1).

Ví dụ, một dòng trong tệp chú thích có thể như sau: 2 0.5 0.5 0.1 0.2, nghĩa là đối tượng thuộc lớp 2, với tâm hộp giới hạn ở giữa ảnh (0.5, 0.5), chiều rộng bằng 10% chiều rộng ảnh và chiều cao bằng 20% chiều cao ảnh.

4.2 Mô hình



Hình 28. Kiến trúc chi tiết của mô hình YOLO

4.2.1 Backbone

- **Input:** Hình ảnh đầu vào có kích thước là 640x640x3.
- **Stem Layer:** Bao gồm các ConvModule đầu tiên nhận dữ liệu từ lớp nhập liệu và bắt đầu quá trình trích xuất đặc trưng.
- **Các giai đoạn (Stage 1 đến Stage 4):**
 - **Stage 1:** Sử dụng các ConvModule kết hợp với C2F để tạo ra featured map cơ bản tại mức độ đầu tiên.
 - **Stage 2 đến 4:** Mỗi stage này có một cấu trúc tương tự nhau, bao gồm các lớp tích chập (ConvModule), được kết nối với các lớp trước đó thông qua phép nối (Concat) và có các lớp Upsample để tăng kích thước không gian của bản đồ đặc trưng, giúp cho việc phát hiện đối tượng ở các kích thước khác nhau.

4.2.2 Neck

- **Spatial Pyramid Pooling (SPPF):** Kết hợp các featured map từ các kích thước khác nhau để tạo ra một featured map duy nhất và mạnh mẽ hơn, sử dụng phép ghép (Concat) để tăng cường tính ngữ nghĩa và không gian của các đặc trưng.
- **Additional ConvModule:** Nhận featured map từ SPPF và thực hiện các phép tính tích chập thêm để chuẩn bị cho quá trình dự đoán cuối cùng trong Head.

4.2.3 Head

- **Phân loại và Định vị:** Head của YOLOv8 bao gồm các ConvModule và Conv2d để cuối cùng dự đoán bounding box và class của các đối tượng.
- **Hàm Loss:** Sử dụng các hàm mất mát khác nhau cho các tác vụ khác nhau:
 - **Binary Cross-Entropy (BCE):** Dùng cho phân loại đối tượng, đánh giá xác suất có đối tượng trong bounding box.
 - **CIoU-DFL Loss:** Dùng cho việc tính toán độ chính xác của bounding box, giúp mô hình dự đoán vị trí chính xác hơn của đối tượng.

4.3 Huấn luyện model trên tập dữ liệu được tạo

Các mô hình được huấn luyện bao gồm YOLOv8n, YOLOv8s, YOLOv8m và YOLOv8l [12]. Quá trình huấn luyện được thực hiện trong các môi trường Colab và Kaggle do khả năng cung cấp tài nguyên tính toán mạnh mẽ và dễ dàng truy cập. Phần mềm AnyLabeling và Roboflow được sử dụng để tạo tọa độ cho dữ liệu ảnh, vì chúng cung cấp các công cụ hữu ích và giao diện thân thiện cho việc gán nhãn và quản lý dữ liệu.

Thư viện YOLO từ Ultralytics được sử dụng để huấn luyện các mô hình này, nhờ vào tính linh hoạt và hiệu quả của thư viện trong việc triển khai các phiên bản

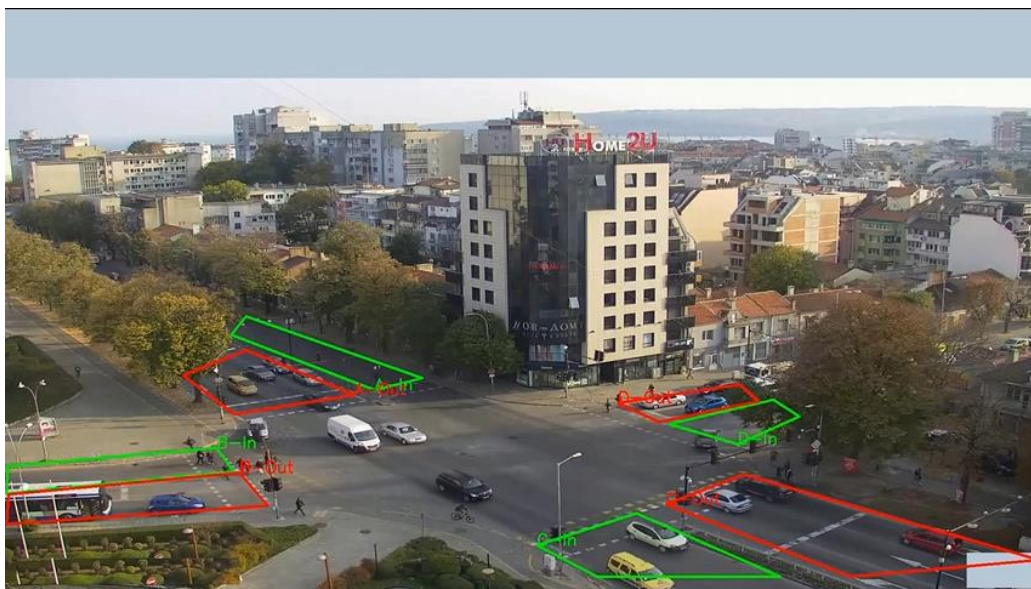
mới nhất của YOLO. Các mô hình được huấn luyện với 30 epochs và kích thước ảnh là 640x640. Bộ dữ liệu bao gồm 1813 hình ảnh trích xuất từ video "city_traffic" với tốc độ 10 khung hình trên giây (FPS), đảm bảo sự đa dạng và phong phú của dữ liệu. Vì là sử dụng thư viện YOLO nên các thông số như batch size, learning rate, ... đều là mặc định. Vì sử dụng thư viện YOLO từ Ultralytics, các thông số như batch size, learning rate, và các tham số khác đều được giữ ở giá trị mặc định.

4.4 Mô tả giải pháp thiết kế

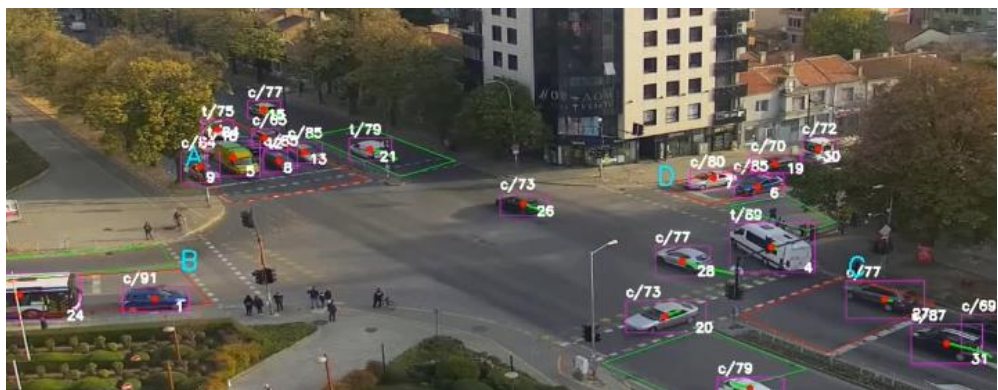
4.4.1 ROI

ROI (Region of Interest) là các khu vực trên khung hình mà chương trình sẽ tập trung để phân tích và theo dõi các đối tượng. Trong bài toán theo dõi phương tiện giao thông, các ROI thường là các làn đường hoặc khu vực đặc biệt mà các phương tiện cần được đếm và theo dõi. Thiết lập ROI giúp giảm thiểu khối lượng tính toán và tăng độ chính xác của việc phát hiện và theo dõi đối tượng.

Đối với cách vẽ ROI ở bài toán, ban đầu chúng em sẽ đánh dấu tọa độ các ngã đường, sau khi lấy được tọa độ thì tôi vẽ bằng bằng thư viện OpenCV, với đường nét màu xanh là làn xe chạy vào, đường nét màu đỏ là làn xe chạy ra.



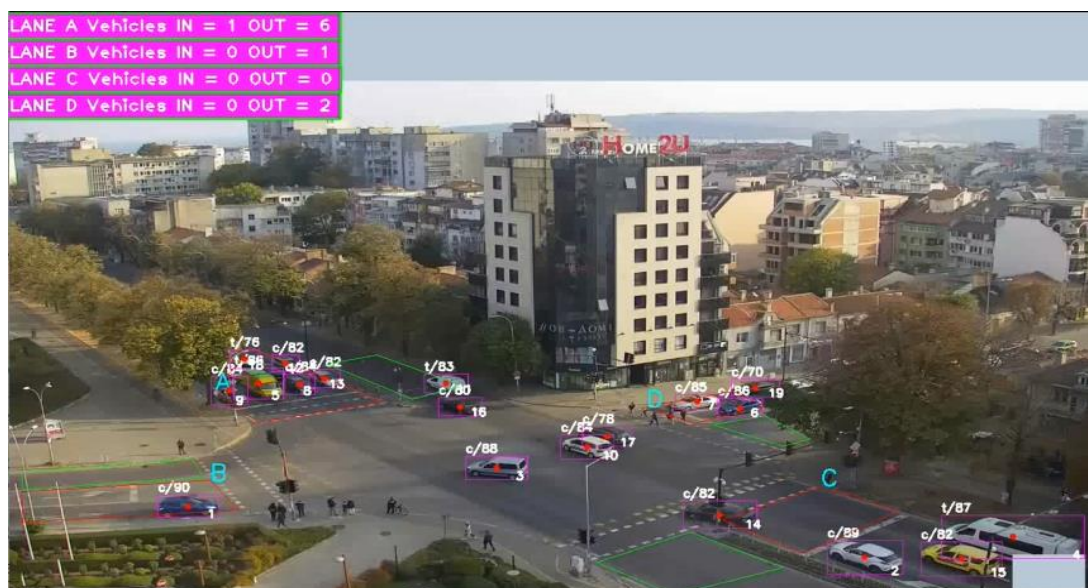
Hình 29. Giám sát luồng giao thông tại ngã tư sử dụng ROI



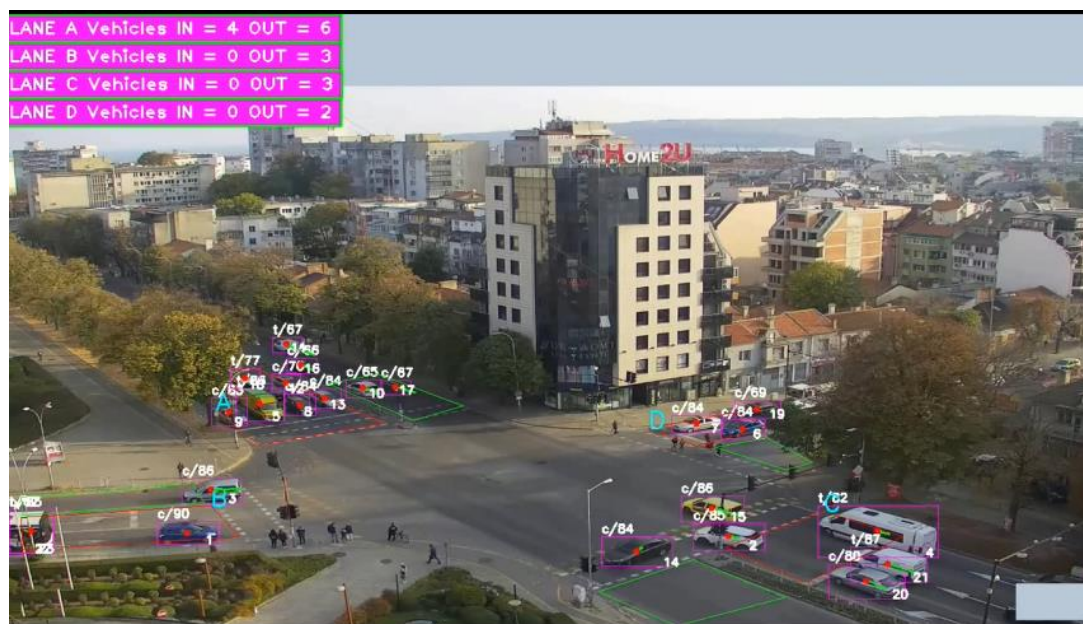
Hình 32. Giám sát luồng giao thông sử dụng BYTETracker để truy vết đối tượng

4.4.4 Đếm đối tượng khi vào ROI

Đếm đối tượng khi chúng di chuyển vào hoặc ra khỏi các ROI được xác định. Sử dụng các tọa độ của ROI để xác định khi nào một đối tượng đã đi vào hoặc ra khỏi một ROI cụ thể. Cập nhật bộ đếm cho mỗi ROI dựa trên số lượng đối tượng đi vào hoặc ra khỏi khu vực này



Hình 33. Đếm đối tượng khi vào ROI



Hình 34. Đếm đối tượng khi vào ROI

4.5 Đánh giá

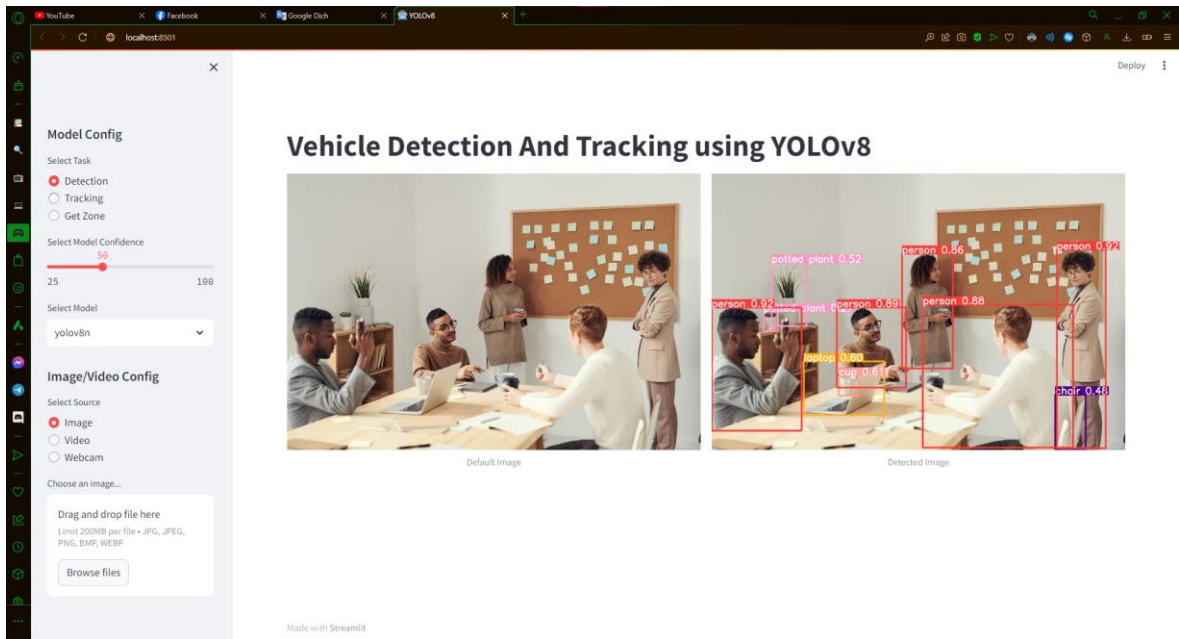
Để đánh giá hiệu quả của các mô hình, chúng tôi đã tiến hành huấn luyện và kiểm tra trên tập dữ liệu riêng. Bảng dưới đây tổng hợp các chỉ số hiệu suất (mAP) và tốc độ xử lý của các mô hình YOLOv8 với các kích thước khác nhau. Chỉ số mAP được thu thập trong quá trình huấn luyện mô hình trên Kaggle, trong khi tốc độ xử lý được đo đạc trên Google Colab với phần cứng CPU Intel Xeon 2.20GHz và GPU Tesla T4 15102MiB. Các kết quả được tính toán trung bình trên 5 lần thử nghiệm để đảm bảo độ chính xác và ổn định của các chỉ số.

Model	size (pixels)	mAP ^{val} 50	mAP ^{val} 50-95	Speed CPU PyTorch (ms)	Speed CPU ONNX (ms)	Speed GPU TensorRT (ms)	params (M)	FLOPs (B)
YOLOv8n	640	93.8	77.9	164.76	635.2	7.98	3.01	4.10
YOLOv8s	640	93.7	79.3	445.6	627.9	8.28	11.4	14.33
YOLOv8m	640	93.8	80.4	1103.6	1579.2	17.42	25.86	39.54
YOLOv8l	640	93.6	79.9	2096.7	2801.7	33.58	43.63	82.71
YOLOvx	640	94.0	79.3	2921.4	2801.7	53.5	68.16	129.07

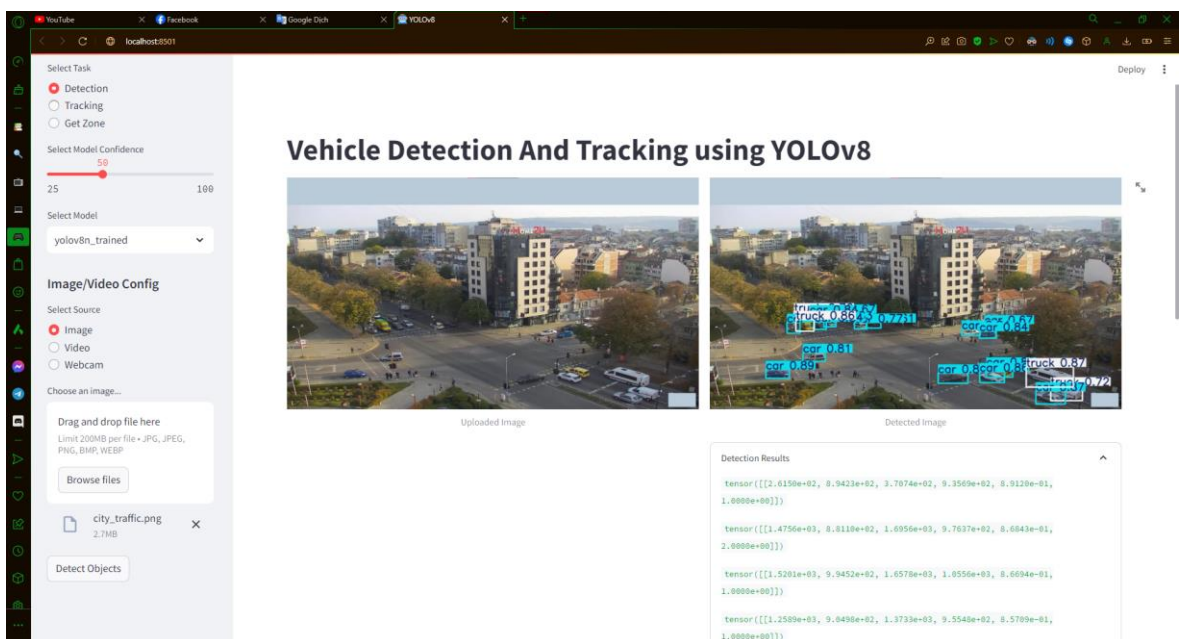
Bảng 5. Kết quả so sánh các mô hình YOLO

4.6 Thực nghiệm

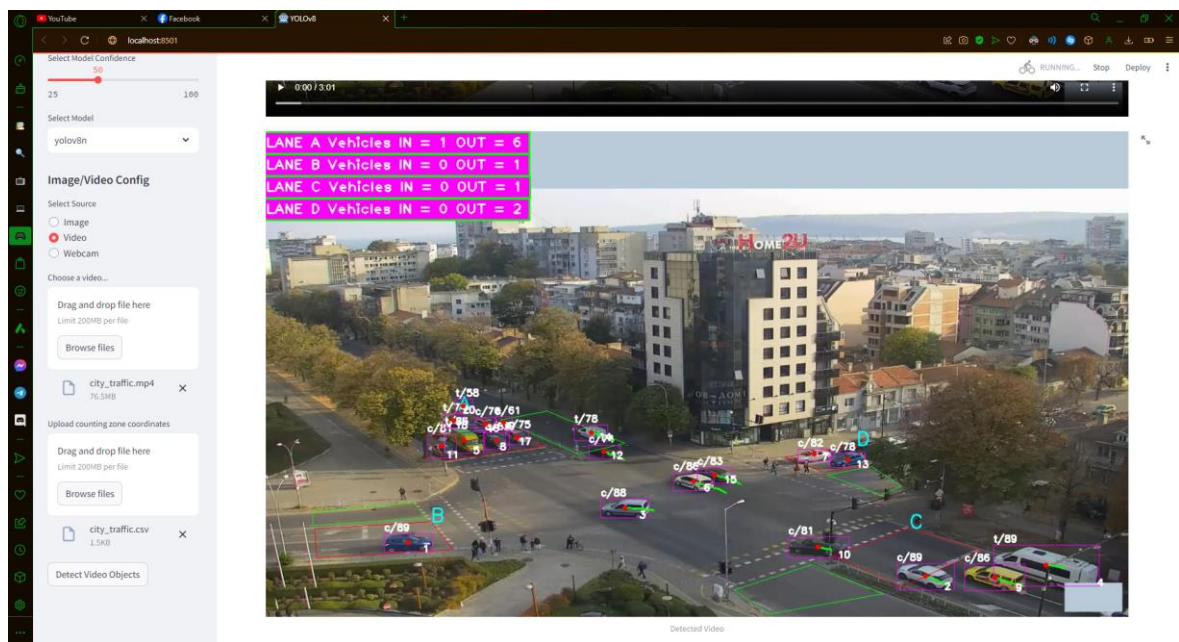
Ứng dụng hỗ trợ nghiên cứu được phát triển bằng thư viện Streamlit, tích hợp các chức năng quan trọng như: phát hiện (detection) và theo dõi (tracking) đối tượng trên hình ảnh và video tự chọn; điều chỉnh các thông số quan trọng như mức độ tin cậy (confidence). Dưới đây là một số hình ảnh minh họa:



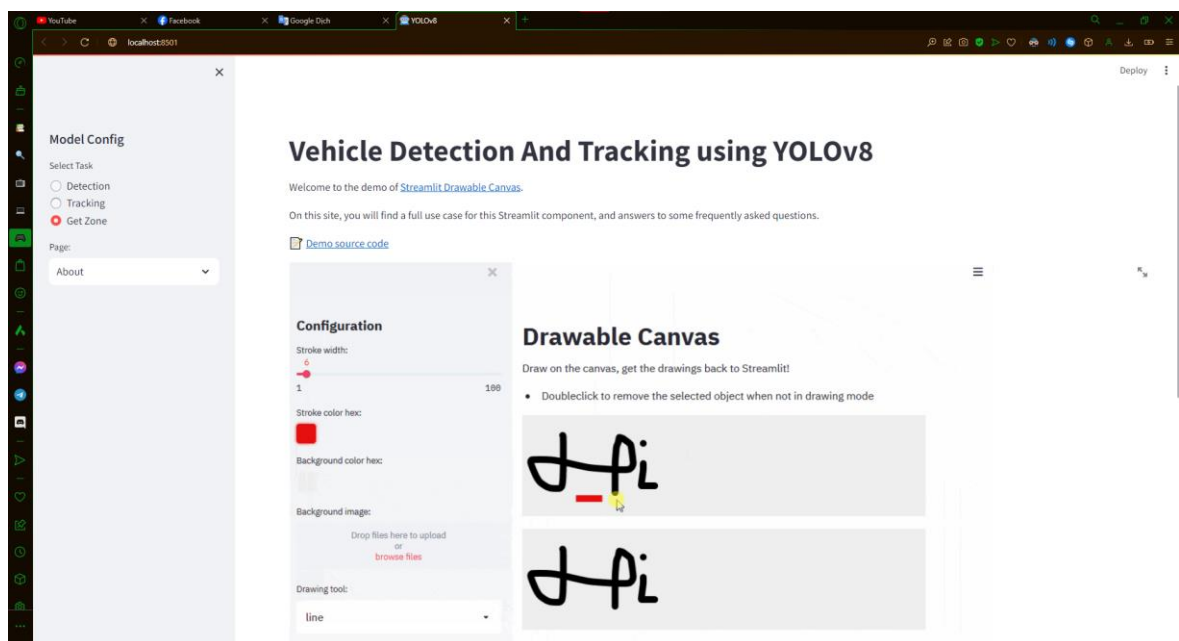
Hình 35. Giao diện chính



Hình 36. Chức năng detection trên ảnh



Hình 37. Chức năng tracking



Hình 38. Chức năng vẽ vùng đếm

KẾT LUẬN

Những điều đã làm được

Trong bài nghiên cứu này, chúng tôi đã đạt được các mục tiêu đề ra. Cụ thể, chúng tôi đã cài đặt thành công các mô hình YOLOv8 để phát hiện đối tượng (detection), và tiến hành huấn luyện mô hình sao cho phù hợp với yêu cầu của đề tài. Chúng tôi cũng đã kết hợp mô hình này với các thuật toán như SORT và ByteTrack để thực hiện theo vết đối tượng (tracking) một cách hiệu quả. Cuối cùng, chúng tôi đã hoàn thành và trình bày bài báo cáo nghiên cứu một cách đầy đủ và chi tiết.

Những điều chưa làm được

Mặc dù đã đạt được nhiều kết quả quan trọng, nghiên cứu này vẫn còn một số hạn chế và điểm cần cải thiện. Đầu tiên, việc tinh chỉnh và huấn luyện mô hình YOLOv8 chưa tối ưu hóa hoàn toàn, dẫn đến một số sai sót trong việc phát hiện và theo dõi đối tượng. Thứ hai, quá trình kết hợp các thuật toán như SORT và ByteTrack vẫn còn gặp khó khăn trong việc xử lý các tình huống phức tạp và đám đông. Thứ ba, mặc dù đã hoàn thiện bài báo cáo nghiên cứu, nhưng việc phân tích kết quả và đề xuất hướng phát triển trong tương lai chưa được khai thác một cách toàn diện. Cuối cùng, hiệu suất của hệ thống trong môi trường thực tế vẫn cần được kiểm tra và đánh giá thêm để đảm bảo tính ứng dụng cao.

Hướng phát triển

Dựa trên những kết quả đã đạt được và những hạn chế còn tồn tại, nghiên cứu này có thể tiếp tục phát triển theo các hướng sau:

1. **Tối ưu hóa mô hình YOLOv8:** Tiếp tục tinh chỉnh và tối ưu hóa mô hình YOLOv8 để giảm thiểu sai sót trong việc phát hiện và theo dõi đối tượng. Điều này có thể bao gồm việc thử nghiệm với các tham số khác nhau, áp dụng các kỹ thuật tối ưu hóa tiên tiến và sử dụng các tập dữ liệu huấn luyện lớn hơn và đa dạng hơn.

2. **Cải thiện thuật toán theo vết (tracking):** Nghiên cứu và phát triển các cải tiến hoặc kết hợp các thuật toán mới với SORT và ByteTrack để nâng cao hiệu suất trong việc xử lý các tình huống phức tạp và đám đông. Điều này có thể bao gồm việc tinh chỉnh các tham số hiện tại, tích hợp thêm thông tin ngữ cảnh hoặc sử dụng các kỹ thuật học sâu (deep learning).
3. **Phân tích và mở rộng dữ liệu huấn luyện:** Tăng cường và đa dạng hóa tập dữ liệu huấn luyện có thể giúp mô hình học được nhiều đặc điểm hơn và cải thiện khả năng tổng quát hóa. Việc này có thể bao gồm thu thập dữ liệu từ nhiều nguồn khác nhau và áp dụng các kỹ thuật tăng cường dữ liệu (data augmentation).
4. **Phân tích kết quả chi tiết và đề xuất hướng phát triển:** Thực hiện phân tích chi tiết hơn về kết quả nghiên cứu để hiểu rõ hơn về hiệu suất của mô hình và hệ thống trong các điều kiện khác nhau. Dựa trên kết quả phân tích này, đề xuất các hướng phát triển cụ thể và khả thi cho nghiên cứu trong tương lai.
5. **Đánh giá và triển khai trong môi trường thực tế:** Tiến hành kiểm tra và đánh giá hệ thống trong các tình huống thực tế để đảm bảo tính khả thi và hiệu suất. Việc này bao gồm thử nghiệm trong các môi trường khác nhau và điều chỉnh hệ thống dựa trên phản hồi thực tế.

Bằng cách tiếp tục phát triển theo các hướng trên, nghiên cứu này có thể đóng góp tích cực vào việc cải thiện hiệu quả và độ chính xác của hệ thống theo dõi luồng giao thông, từ đó mang lại lợi ích thiết thực cho xã hội.

TÀI LIỆU THAM KHẢO

- [1] soumyadip, "LearnOpenCV," [Trực Tuyến]. Có sẵn tại: <https://learnopencv.com/mastering-all-yolo-models/>. [Truy cập 2 tháng 7 2024].
- [2] J. Terven and D. Cordova-Esparza, "arxiv," [Trực Tuyến]. Có sẵn tại: <https://arxiv.org/abs/2304.00501>. [Truy cập 2 Tháng 7 2024].
- [3] P. Sharma, "Analytics Vidhya," [Trực Tuyến]. Có sẵn tại: <https://www.analyticsvidhya.com/blog/2018/12/practical-guide-object-detection-yolo-framework-python/>. [Truy cập 2 Tháng 7 2024].
- [4] B. Esme, "Bilgin's Blog," [Trực Tuyến]. Có sẵn tại: <http://bilgin.esme.org/BitsAndBytes/KalmanFilterforDummies>. [Truy cập 2 Tháng 7 2024].
- [5] A. Minisini and O. Kulkov, "Algorithms for Competitive Programming," [Trực Tuyến]. Có sẵn tại: <https://cp-algorithms.com/graph/hungarian-algorithm.html>. [Truy cập 2 Tháng 7 2024].
- [6] A. Bewley, Z. Ge, L. Ott, F. Ramos and B. Upcroft, "arxiv," [Trực Tuyến]. Có sẵn tại: <https://arxiv.org/abs/1602.00763>. [Truy cập 2 Tháng 7 2024].
- [7] luffca, "luffca," [Trực Tuyến]. Có sẵn tại: <https://www.luffca.com/2023/04/multiple-object-tracking-sort/>. [Truy cập 2 Tháng 7 2024].
- [8] B. T. Tung, "VIBLO," [Trực Tuyến]. Có sẵn tại: <https://viblo.asia/p/sort-deep-sort-mot-goc-nhin-ve-object-tracking-phan-1-Az45bPooZxY>. [Truy cập 2 Tháng 7 2024].
- [9] T. Sơn, "VIBLO," [Trực Tuyến]. Có sẵn tại: <https://viblo.asia/p/bytetrack-multi-object-detector-hien-dai-nhat-hien-nay-RQqKL33r57z>. [Truy cập 2 Tháng 7 2024].

- [10] B. LE, "Datature," [Trực Tuyến]. Có sẵn tại: <https://www.datature.io/blog/introduction-to-bytetrack-multi-object-tracking-by-associating-every-detection-box>. [Truy cập 2 Tháng 7 2024].
- [11] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu and X. Wang, "arxiv," [Trực Tuyến]. Có sẵn tại: <https://arxiv.org/abs/2110.06864>. [Truy cập 2 Tháng 7 2024].
- [12] T. KELDENICH, "Inside MachineLearning," [Trực Tuyến]. Có sẵn tại: <https://inside-machinelearning.com/en/yolov8-how-to-use/#boxzilla-13443>. [Truy cập 2 Tháng 7 2024].
- [13] T. KELDENICH, "Inside MachineLearning," [Trực Tuyến]. Available: <https://inside-machinelearning.com/en/bounding-boxes-python-function/>. [Truy cập 2 tháng 7 2024].
- [14] J. Solawetz and Francesco, "roboflow," [Trực Tuyến]. Có sẵn tại: <https://blog.roboflow.com/whats-new-in-yolov8/>. [Truy cập 2 tháng 7 2024].

PHỤ LỤC

Các thành phần thuật toán SORT

❖ Các hàm tiện ích

Thuật toán SORT sử dụng một số hàm tiện ích quan trọng sau:

- `linear_assignment()`: Gán nhãn liên kết giữa các phát hiện (detections) và các đối tượng đang được theo dõi (trackers) bằng cách giải bài toán lập kế hoạch tuyến tính.

```
def linear_assignment(cost_matrix):
    try:
        import lap
        _, x, y = lap.lapjv(cost_matrix, extend_cost=True)
        return np.array([[y[i],i] for i in x if i >= 0]) #
    except ImportError:
        from scipy.optimize import linear_sum_assignment
        x, y = linear_sum_assignment(cost_matrix)
        return np.array(list(zip(x, y)))
```

- `iou_batch()`: Tính toán giá trị IoU giữa các phát hiện và các đối tượng đang được theo dõi.

```
def iou_batch(bb_test, bb_gt):
    bb_gt = np.expand_dims(bb_gt, 0)
    bb_test = np.expand_dims(bb_test, 1)

    xx1 = np.maximum(bb_test[..., 0], bb_gt[..., 0])
    yy1 = np.maximum(bb_test[..., 1], bb_gt[..., 1])
    xx2 = np.minimum(bb_test[..., 2], bb_gt[..., 2])
    yy2 = np.minimum(bb_test[..., 3], bb_gt[..., 3])
    w = np.maximum(0., xx2 - xx1)
    h = np.maximum(0., yy2 - yy1)
    wh = w * h
    o = wh / ((bb_test[..., 2] - bb_test[..., 0]) * (bb_test[..., 3] -
bb_test[..., 1])
              + (bb_gt[..., 2] - bb_gt[..., 0]) * (bb_gt[..., 3] - bb_gt[...,
1])) - wh)
    return(o)
```

- `convert_bbox_to_z()` và `convert_x_to_bbox()`: Chuyển đổi định dạng biểu diễn bounding box giữa dạng $[x1, y1, x2, y2]$ và $[x, y, s, r]$.

```
def convert_bbox_to_z(bbox):  
    w = bbox[2] - bbox[0]  
    h = bbox[3] - bbox[1]  
    x = bbox[0] + w/2.  
    y = bbox[1] + h/2.  
    s = w * h  
    r = w / float(h)  
    return np.array([x, y, s, r]).reshape((4, 1))
```

- `associate_detections_to_trackers()`: đóng vai trò quan trọng trong việc gán các phát hiện mới vào các đối tượng đang được theo dõi hiện có. Hàm này sử dụng giá trị IoU để xác định mối quan hệ giữa các phát hiện mới và các đối tượng đang được theo dõi, từ đó cập nhật trạng thái của các đối tượng đang được theo dõi và khởi tạo các đối tượng mới cho các phát hiện chưa được gán.

```
def associate_detections_to_trackers(detections,trackers,iou_threshold
= 0.3):
    if(len(trackers)==0):
        return np.empty((0,2),dtype=int), np.arange(len(detections)),
np.empty((0,5),dtype=int)
    iou_matrix = iou_batch(detections, trackers)

    if min(iou_matrix.shape) > 0:
        a = (iou_matrix > iou_threshold).astype(np.int32)
        if a.sum(1).max() == 1 and a.sum(0).max() == 1:
            matched_indices = np.stack(np.where(a), axis=1)
        else:
            matched_indices = linear_assignment(-iou_matrix)
    else:
        matched_indices = np.empty(shape=(0,2))

    unmatched_detections = []
    for d, det in enumerate(detections):
        if(d not in matched_indices[:,0]):
            unmatched_detections.append(d)
    unmatched_trackers = []
    for t, trk in enumerate(trackers):
        if(t not in matched_indices[:,1]):
            unmatched_trackers.append(t)

    matches = []
    for m in matched_indices:
        if(iou_matrix[m[0], m[1]]<iou_threshold):
            unmatched_detections.append(m[0])
            unmatched_trackers.append(m[1])
        else:
            matches.append(m.reshape(1,2))
    if(len(matches)==0):
        matches = np.empty((0,2),dtype=int)
    else:
        matches = np.concatenate(matches,axis=0)
    return matches, np.array(unmatched_detections),
np.array(unmatched_trackers)
```


- `parse_args()`: để xử lý các tham số đầu vào từ dòng lệnh, chẳng hạn như hiển thị đầu ra của tracker, đường dẫn đến tệp phát hiện, ngưỡng IoU, số lượng khung hình tối đa để giữ đối tượng và số lượng phát hiện tối thiểu để khởi tạo đối tượng mới.

```
def parse_args():
    parser = argparse.ArgumentParser(description='SORT demo')
    parser.add_argument('--display', dest='display', help='Display
online tracker output (slow) [False]', action='store_true')
    parser.add_argument("--seq_path", help="Path to detections.",
type=str, default='data')
    parser.add_argument("--phase", help="Subdirectory in seq_path.",
type=str, default='train')
    parser.add_argument("--max_age",
                        help="Maximum number of frames to keep alive a
track without associated detections.",
                        type=int, default=1)
    parser.add_argument("--min_hits",
                        help="Minimum number of associated detections
before track is initialised.",
                        type=int, default=3)
    parser.add_argument("--iou_threshold", help="Minimum IOU for
match.", type=float, default=0.3)
    args = parser.parse_args()
    return args
```

❖ Lớp `KalmanBoxTracker()`

Lớp `KalmanBoxTracker` thực hiện bộ lọc Kalman để theo dõi vị trí và vận tốc của các đối tượng. Lớp này đóng vai trò quan trọng trong việc dự đoán vị trí của các đối tượng trong khung hình tiếp theo.

Khởi tạo: khởi tạo bộ lọc Kalman với hộp giới hạn đầu vào. Nó thiết lập các ma trận chuyển trạng thái (F), ma trận đo (H), ma trận nhiễu đo (R), ma trận nhiễu quá trình (Q) và vector trạng thái ban đầu (x).

- `__init__()`: Khởi tạo trình theo dõi bằng cách sử dụng bounding box ban đầu.

```
count = 0
def __init__(self, bbox):
    self.kf = KalmanFilter(dim_x=7, dim_z=4)
    self.kf.F = np.array([[1,0,0,0,1,0,0],[0,1,0,0,0,1,0],
                          [0,0,1,0,0,0,1],[0,0,0,1,0,0,0],
                          [0,0,0,0,1,0,0],[0,0,0,0,0,1,0],
                          [0,0,0,0,0,0,1]])
    self.kf.H = np.array([[1,0,0,0,0,0,0],[0,1,0,0,0,0,0],
                          [0,0,1,0,0,0,0],[0,0,0,1,0,0,0]])

    self.kf.R[2:,2:] *= 10.
    self.kf.P[4:,4:] *= 1000
    self.kf.P *= 10.
    self.kf.Q[-1,-1] *= 0.01
    self.kf.Q[4:,4:] *= 0.01

    self.kf.x[:4] = convert_bbox_to_z(bbox)
    self.time_since_update = 0
    self.id = KalmanBoxTracker.count
    KalmanBoxTracker.count += 1
    self.history = []
    self.hits = 0
    self.hit_streak = 0
    self.age = 0
```

- `update()`: cập nhật vector trạng thái với thông tin từ hộp giới hạn quan sát được.

```
def update(self, bbox):
    self.time_since_update = 0
    self.history = []
    self.hits += 1
    self.hit_streak += 1
    self.kf.update(convert_bbox_to_z(bbox))
```

- `predict()`: dự đoán trạng thái tiếp theo của đối tượng dựa trên các trạng thái trước đó, đồng thời cập nhật tuổi của đối tượng và lưu trữ lịch sử dự đoán.

```
def predict(self):
    if((self.kf.x[6]+self.kf.x[2])<=0):
        self.kf.x[6] *= 0.0
    self.kf.predict()
    self.age += 1
    if(self.time_since_update>0):
        self.hit_streak = 0
    self.time_since_update += 1
    self.history.append(convert_x_to_bbox(self.kf.x))
    return self.history[-1]
```

- `get_state()`: trả về bounding box dự đoán hiện tại

```
def get_state(self):
    return convert_x_to_bbox(self.kf.x)
```

❖ Lớp SORT

Lớp `Sort` thực hiện các bước chính của thuật toán SORT để theo dõi các đối tượng trong chuỗi khung hình. Trong quá trình khởi tạo, lớp này thiết lập các tham số quan trọng như `max_age` (số lượng khung hình tối đa để duy trì một đối tượng đang được theo dõi mà không có phát hiện mới), `min_hits` (số lượng phát hiện tối thiểu cần thiết để khởi tạo một đối tượng mới), và `iou_threshold` (ngưỡng IoU tối thiểu để xác định một phát hiện có khớp với một đối tượng đang được theo dõi hay không).

- `__init__()`: Khởi tạo các tham số.

```
def __init__(self, max_age=1, min_hits=3, iou_threshold=0.3):
    self.max_age = max_age
    self.min_hits = min_hits
    self.iou_threshold = iou_threshold
    self.trackers = []
    self.frame_count = 0
```

- `update()`: thực hiện các bước chính của thuật toán SORT.
 1. Dự đoán vị trí mới cho các đối tượng đang được theo dõi hiện tại.
 2. Gán các phát hiện mới vào các đối tượng đang được theo dõi hiện có bằng cách sử dụng hàm `associate_detections_to_trackers()`.
 3. Cập nhật trạng thái của các đối tượng đang được theo dõi với thông tin từ các phát hiện mới.
 4. Khởi tạo các đối tượng mới cho các phát hiện không được gán.
 5. Xóa các đối tượng đang được theo dõi không nhận được cập nhật trong một khoảng thời gian nhất định.
 6. Trả về danh sách các đối tượng đang được theo dõi, bao gồm thông tin về bounding box và ID của đối tượng.

```

def update(self, dets=np.empty((0, 5))):
    self.frame_count += 1
    trks = np.zeros((len(self.trackers), 5))
    to_del = []
    ret = []
    for t, trk in enumerate(trks):
        pos = self.trackers[t].predict()[0]
        trk[:] = [pos[0], pos[1], pos[2], pos[3], 0]
        if np.any(np.isnan(pos)):
            to_del.append(t)
    trks = np.ma.compress_rows(np.ma.masked_invalid(trks))
    for t in reversed(to_del):
        self.trackers.pop(t)
    matched, unmatched_dets, unmatched_trks =
associate_detections_to_trackers(dets, trks, self.iou_threshold)

    for m in matched:
        self.trackers[m[1]].update(dets[m[0], :])

    for i in unmatched_dets:
        trk = KalmanBoxTracker(dets[i, :])
        self.trackers.append(trk)
    i = len(self.trackers)
    for trk in reversed(self.trackers):
        d = trk.get_state()[0]
        if (trk.time_since_update < 1) and (trk.hit_streak >=
self.min_hits or self.frame_count <= self.min_hits):
            ret.append(np.concatenate((d, [trk.id+1])).reshape(1, -1))
            i -= 1

        if (trk.time_since_update > self.max_age):
            self.trackers.pop(i)
    if (len(ret)>0):
        return np.concatenate(ret)
    return np.empty((0, 5))

```

Chi tiết các phần chính trong đoạn mã

❖ Class STrack

Class STrack kế thừa từ class BaseTrack và sử dụng lọc Kalman để dự đoán và cập nhật trạng thái của đối tượng.

- **Thuộc tính:**

- *shared_kalman*: Đối tượng bộ lọc Kalman được chia sẻ.
- *_tlwh, score, tracklet_len, is_activated, mean, covariance, kalman_filter*: Các thuộc tính để lưu trữ thông tin về đối tượng và trạng thái của nó.

- **Phương thức:**

- `__init__()`: Khởi tạo đối tượng với các tham số ban đầu.

```
shared_kalman = KalmanFilter()
def __init__(self, tlwh, score):
    self._tlwh = np.asarray(tlwh, dtype=float)
    self.kalman_filter = None
    self.mean, self.covariance = None, None
    self.is_activated = False

    self.score = score
    self.tracklet_len = 0
```

- `predict()`: Dự đoán vị trí tiếp theo của đối tượng.

```
def predict(self):
    mean_state = self.mean.copy()
    if self.state != TrackState.Tracked:
        mean_state[7] = 0
    self.mean, self.covariance =
self.kalman_filter.predict(mean_state, self.covariance)
```

- `multi_predict()`: Dự đoán vị trí tiếp theo của nhiều đối tượng cùng lúc.

```
@staticmethod
def multi_predict(stracks):
    if len(stracks) > 0:
        multi_mean = np.asarray([st.mean.copy() for st in stracks])
        multi_covariance = np.asarray([st.covariance for st in
stracks])
        for i, st in enumerate(stracks):
            if st.state != TrackState.Tracked:
                multi_mean[i][7] = 0
        multi_mean, multi_covariance =
STrack.shared_kalman.multi_predict(multi_mean, multi_covariance)
        for i, (mean, cov) in enumerate(zip(multi_mean,
multi_covariance)):
            stracks[i].mean = mean
            stracks[i].covariance = cov
```

- o `activate()`: Khởi tạo một đối tượng mới.

```
def activate(self, kalman_filter, frame_id):
    """Start a new tracklet"""
    self.kalman_filter = kalman_filter
    self.track_id = self.next_id()
    self.mean, self.covariance =
self.kalman_filter.initiate(self.tlwh_to_xyah(self._tlwh))

    self.tracklet_len = 0
    self.state = TrackState.Tracked
    if frame_id == 1:
        self.is_activated = True
    self.frame_id = frame_id
    self.start_frame = frame_id
```

- o `re_activate()`: Kích hoạt lại một đối tượng đã mất.

```
def re_activate(self, new_track, frame_id, new_id=False):
    self.mean, self.covariance = self.kalman_filter.update(
        self.mean, self.covariance, self.tlwh_to_xyah(new_track.tlwh)
    )
    self.tracklet_len = 0
    self.state = TrackState.Tracked
    self.is_activated = True
    self.frame_id = frame_id
    if new_id:
        self.track_id = self.next_id()
    self.score = new_track.score
```


- `update()`: Cập nhật trạng thái của đối tượng khi có thông tin mới.

```
def update(self, new_track, frame_id):
    self.frame_id = frame_id
    self.tracklet_len += 1

    new_tlwh = new_track.tlwh
    self.mean, self.covariance = self.kalman_filter.update(
        self.mean, self.covariance, self.tlwh_to_xyah(new_tlwh))
    self.state = TrackState.Tracked
    self.is_activated = True

    self.score = new_track.score
```

- o `tlwh()`, `tlbr()`, `tlwh_to_xyah()`, `tlbr_to_tlwh()`, `tlwh_to_tlbr()`: Các phương thức để chuyển đổi và truy xuất các định dạng bounding box khác nhau.

```
@property
def tlwh(self):
    if self.mean is None:
        return self._tlwh.copy()
    ret = self.mean[:4].copy()
    ret[2] *= ret[3]
    ret[:2] -= ret[2:] / 2
    return ret

@property
def tlbr(self):
    ret = self.tlwh.copy()
    ret[2:] += ret[:2]
    return ret

@staticmethod
def tlwh_to_xyah(tlwh):
    ret = np.asarray(tlwh).copy()
    ret[:2] += ret[2:] / 2
    ret[2] /= ret[3]
    return ret

def to_xyah(self):
    return self.tlwh_to_xyah(self.tlwh)

@staticmethod
def tlbr_to_tlwh(tlbr):
    ret = np.asarray(tlbr).copy()
    ret[2:] -= ret[:2]
    return ret

@staticmethod
def tlwh_to_tlbr(tlwh):
    ret = np.asarray(tlwh).copy()
    ret[2:] += ret[:2]
    return ret
```

❖ Class BYTETracker

Lớp BYTETracker quản lý các đối tượng đang được theo dõi và thực hiện các thao tác như cập nhật, kết hợp, và quản lý trạng thái của chúng.

- **Thuộc tính:**

- `tracked_tracks`, `lost_tracks`, `removed_tracks`: Các danh sách để lưu trữ các đối tượng đang được theo dõi, đã mất, và đã bị loại bỏ.
- `frame_id`, `args`, `det_thresh`, `buffer_size`, `max_time_lost`, `kalman_filter`: Các thuộc tính để lưu trữ thông tin cấu hình và trạng thái hiện tại của hệ thống.

- **Phương thức:**

- `__init__()`: Khởi tạo đối tượng BYTETracker với các tham số cấu hình.

```
def __init__(self, args, frame_rate=30):
    self.tracked_tracks = []
    self.lost_tracks = []
    self.removed_tracks = []

    self.frame_id = 0
    self.args = args
    self.det_thresh = args.track_thresh + 0.1
    self.buffer_size = int(frame_rate / 30.0 * args.track_buffer)
    self.max_time_lost = self.buffer_size
    self.kalman_filter = KalmanFilter()
```

- `update()`: Cập nhật trạng thái của hệ thống theo dõi dựa trên các kết quả đầu ra từ mô hình phát hiện đối tượng.

```

def update(self, output_results, img_info, img_size):
    self.frame_id += 1
    activated_starcks = []
    refind_tracks = []
    lost_tracks = []
    removed_tracks = []

    if output_results.shape[1] == 6:
        scores = output_results[:, 4]
        bboxes = output_results[:, :4]
    else:
        output_results = output_results.cpu().numpy()
        scores = output_results[:, 4] * output_results[:, 5]
        bboxes = output_results[:, :4]
    img_h, img_w = img_info[0], img_info[1]
    scale = min(img_size[0] / float(img_h), img_size[1] /
float(img_w))
    bboxes /= scale

    remain_inds = scores > self.args.track_thresh
    inds_low = scores > 0.1
    inds_high = scores < self.args.track_thresh

    inds_second = np.logical_and(inds_low, inds_high)
    dets_second = bboxes[inds_second]
    dets = bboxes[remain_inds]
    scores_keep = scores[remain_inds]
    scores_second = scores[inds_second]

    if len(dets) > 0:
        detections = [STrack(STrack.tlbr_to_tlwh(tlbr), s) for
            (tlbr, s) in zip(dets, scores_keep)]
    else:
        detections = []
    unconfirmed = []
    tracked_tracks = []
    for track in self.tracked_tracks:
        if not track.is_activated:

```

```

        unconfirmed.append(track)
    else:
        tracked_tracks.append(track)

    strack_pool = joint_tracks(tracked_tracks,
self.lost_tracks)

    STrack.multi_predict(strack_pool)
    dists = matching.iou_distance(strack_pool, detections)
    matches, u_track, u_detection =
matching.linear_assignment(dists, thresh=self.args.match_thresh)
    for itracked, idet in matches:
        track = strack_pool[itracked]
        det = detections[idet]
        if track.state == TrackState.Tracked:
            track.update(detections[idet], self.frame_id)
            activated_starcks.append(track)
        else:
            track.re_activate(det, self.frame_id, new_id=False)
            refind_tracks.append(track)

    if len(dets_second) > 0:
        detections_second = [STrack(STrack.tlbr_to_tlwh(tlbr), s)
for
                                (tlbr, s) in zip(dets_second, scores_second)]
    else:
        detections_second = []
        r_tracked_tracks = [strack_pool[i] for i in u_track if
strack_pool[i].state == TrackState.Tracked]
        dists = matching.iou_distance(r_tracked_tracks,
detections_second)
        matches, u_track, u_detection_second =
matching.linear_assignment(dists, thresh=0.5)
        for itracked, idet in matches:
            track = r_tracked_tracks[itracked]
            det = detections_second[idet]
            if track.state == TrackState.Tracked:
                track.update(det, self.frame_id)

```

```

        activated_starcks.append(track)
    else:
        track.re_activate(det, self.frame_id, new_id=False)
        refind_tracks.append(track)

    for it in u_track:
        track = r_tracked_tracks[it]
        if not track.state == TrackState.Lost:
            track.mark_lost()
            lost_tracks.append(track)

    detections = [detections[i] for i in u_detection]
    dists = matching.iou_distance(unconfirmed, detections)
    dists = matching.fuse_score(dists, detections)

    matches, u_unconfirmed, u_detection =
matching.linear_assignment(dists, thresh=0.7)
    for itracked, idet in matches:
        unconfirmed[itracked].update(detections[idet],
self.frame_id)
        activated_starcks.append(unconfirmed[itracked])
    for it in u_unconfirmed:
        track = unconfirmed[it]
        track.mark_removed()
        removed_tracks.append(track)
    for inew in u_detection:
        track = detections[inew]
        if track.score < self.det_thresh:
            continue
        track.activate(self.kalman_filter, self.frame_id)
        activated_starcks.append(track)
    for track in self.lost_tracks:
        if self.frame_id - track.end_frame > self.max_time_lost:
            track.mark_removed()
            removed_tracks.append(track)

```

```

        self.tracked_tracks = [t for t in self.tracked_tracks if
t.state == TrackState.Tracked]
        self.tracked_tracks = joint_tracks(self.tracked_tracks,
activated_starcks)
        self.tracked_tracks = joint_tracks(self.tracked_tracks,
refind_tracks)
        self.lost_tracks = sub_tracks(self.lost_tracks,
self.tracked_tracks)
        self.lost_tracks.extend(lost_tracks)
        self.lost_tracks = sub_tracks(self.lost_tracks,
self.removed_tracks)
        self.removed_tracks.extend(removed_tracks)
        self.tracked_tracks, self.lost_tracks =
remove_duplicate_tracks(self.tracked_tracks, self.lost_tracks)

        output_tracks = [track for track in self.tracked_tracks if
track.is_activated]

        return output_tracks

```

❖ Các hàm hỗ trợ

- `joint_tracks(tlista, tlistb)`: Kết hợp hai danh sách các đường dẫn theo dõi (`tlista` và `tlistb`) thành một danh sách duy nhất.

```

def joint_tracks(tlista, tlistb):
    exists = {}
    res = []
    for t in tlista:
        exists[t.track_id] = 1
        res.append(t)
    for t in tlistb:
        tid = t.track_id
        if not exists.get(tid, 0):
            exists[tid] = 1
            res.append(t)
    return res

```

- `sub_tracks(tlista, tlistb)`: Lấy phần tử khác biệt giữa hai danh sách các đường dẫn theo dõi (`tlista` và `tlistb`).

```
def sub_tracks(tlista, tlistb):
    tracks = {}
    for t in tlista:
        tracks[t.track_id] = t
    for t in tlistb:
        tid = t.track_id
        if tracks.get(tid, 0):
            del tracks[tid]
    return list(tracks.values())
```

- `remove_duplicate_tracks(stracksa, stracksb)`: Loại bỏ các đường dẫn theo dõi trùng lặp giữa hai danh sách (`stracksa` và `stracksb`).

```
def remove_duplicate_tracks(stracksa, stracksb):
    pdist = matching.iou_distance(stracksa, stracksb)
    pairs = np.where(pdist < 0.15)
    dupa, dupb = list(), list()
    for p, q in zip(*pairs):
        timep = stracksa[p].frame_id - stracksa[p].start_frame
        timeq = stracksb[q].frame_id - stracksb[q].start_frame
        if timep > timeq:
            dupb.append(q)
        else:
            dupa.append(p)
    resa = [t for i, t in enumerate(stracksa) if not i in dupa]
    resb = [t for i, t in enumerate(stracksb) if not i in dupb]
    return resa, resb
```